**The University of Manchester**
**School of Natural Sciences**

# Word Soup Analysis

A Report Submitted for

**MATH30000: Double Project 2025/26 Full Year**

**Author:** Siti Abas

**Supervisor:** Dr. Neil Morrison

May 12, 2026

**Abstract**

This project investigates algorithmic strategies for solving the word-based puzzle game Word Soup. The game requires sequential word selection from a fixed grid of letters, where each decision affects both immediate score and future move availability. Although the objective appears to be score maximisation, the evolving grid structure introduces a trade-off between short-term reward and long-term solvability, making the problem computationally challenging.

Five algorithmic players are implemented with increasing levels of lookahead, ranging from a random baseline to 1-ply, 2-ply, and 3-ply greedy search, and a Hybrid policy incorporating rollout simulation. All players operate under identical game rules and scoring mechanisms, allowing controlled comparison of how additional foresight influences performance. The project also establishes a benchmarking framework to validate correctness against reference grids and to measure score, solvability, and runtime.

Experimental results show that increasing ply depth improves average score and grid clearance rates, but at a rapidly increasing computational cost. This highlights a fundamental trade-off between decision quality and computational feasibility. The Hybrid policy achieves an 87% grid clear rate compared to 6% for the Greedy-1, demonstrating that rollout-based lookahead substantially outperforms greedy search in both solvability and score.

# Contents

**1 Introduction** **5**
- 1.1 Motivation . . . 5
- 1.2 Aims and Research Questions . . . 6
- 1.3 Report Structure . . . 7
- 1.4 Illustrative Example of Human Play . . . 8

**2 Theoretical Background: Optimisation and Sequential Search** **15**
- 2.1 Strategy Games as Computational Problems . . . 15
- 2.2 Optimisation and Objective Functions . . . 17
- 2.3 Search Trees and Ply Depth . . . 19
- 2.4 Greedy Algorithms and Heuristics . . . 21
- 2.5 Dynamic Programming . . . 22
- 2.6 Simulation Search and Stochastic Method . . . 24

**3 Game Description: The Rules and Mechanics of Word Soup** **26**
- 3.1 Rules of the Game . . . 26
- 3.2 Scoring System . . . 28
- 3.3 Grid Evolution and Gravity . . . 31
- 3.4 Success Criteria . . . 33
- 3.5 Game State and Player Decisions . . . 34

**4 Algorithmic Players** **36**
- 4.1 Shared Components . . . 36
- 4.2 Random Player (Baseline) . . . 37
- 4.3 Greedy n-Ply Framework . . . 38
  - 4.3.1 1-Ply ($n = 1$) . . . 38
  - 4.3.2 2-Ply ($n = 2$) . . . 40
- 4.4 n-Ply Generalisation . . . 43
- 4.5 Heuristic / Hybrid Extensions . . . 46

**5 Experimental Setup** **49**
- 5.1 Dictionary Processing . . . 49

5.2 Grid Generation . . . 53
5.3 Metrics Collected . . . 54
5.4 Batch Configurations . . . 56
5.5 Benchmark Grids for Correctness . . . 57

**6 Results** **64**
6.1 Baseline Performance Comparison . . . 64
6.1.1 Clear Rate by Policy . . . 64
6.1.2 Average Score by Policy . . . 66
6.1.3 Runtime Cost of Lookahead . . . 71
6.2 Structural Properties of the Grid . . . 74
6.2.1 Predictors of Clearing . . . 74
6.2.2 Anomalous Failure: Grid `lfreq_6` . . . 80
6.2.3 Anomalous Failure: Grid `lfreq_37` . . . 84
6.3 Vowel Ratio as the Dominant Predictor of Clearing . . . 86
6.4 Efficiency and Trade-Off Analysis . . . 88
6.4.1 Score per Second . . . 88
6.5 Single-Game Case Study: Grid `lfreq_7` . . . 90

**7 Discussion** **98**
7.1 Interpretation of Results . . . 98
7.2 Comparison with Human Play . . . 100
7.3 Same Grid Comparison . . . 103
7.4 Limitations . . . 112
7.5 Future Work . . . 114
7.5.1 Implementation Improvements . . . 114
7.5.2 Monte Carlo Tree Search and Machine Learning . . . 117

**8 Conclusion** **122**

**9 Python Code for Players** **124**
9.1 Greedy Game Loop: `play_one_greedy_run` . . . 124
9.2 Greedy-1: `_greedy_best_choice` . . . 128
9.3 Greedy-2: `_best_two_move_plan` . . . 131

9.4 Greedy-3: `_best_three_move_plan` . . . . . 133
9.5 Hybrid Stage 1: `_select_hybrid_policy_ply` . . . . . 135
9.6 Hybrid move chooser: `_choose_move_hybrid` . . . . . 136
9.7 Hybrid Game Loop: `play_one_hybrid_run` . . . . . 138

# 1 Introduction

Word Soup is a word-based puzzle game in which players form valid words by connecting adjacent letters on a grid. Each selected word contributes to a cumulative score, and the structure of the grid evolves after every move due to tile removal and gravity-based shifting. Although the game appears simple at first glance, it naturally gives rise to a complex sequential decision problem in which early choices influence both immediate rewards and future possibilities.

## 1.1 Motivation




Figure 1: Example of a Word Soup grid with a word selection in progress. The highlighted path forms the word "TRENDY".

Many puzzle games can be interpreted as optimisation problems, where a player repeatedly selects actions from a large set of possible choices in order to maximise a numerical objective. As illustrated in Figure 1, Word Soup provides a particularly interesting example because each move alters the structure of the grid, thereby changing the set of future legal actions.

This relationship between immediate score maximisation and future board flexibility mirrors fundamental questions in optimisation and search theory. Greedy strategies prioritise local reward, whereas deeper lookahead methods attempt to account for longer-term consequences. Understanding how these approaches perform in a structured environment such as Word Soup

provides insight into broader issues in heuristic search and sequential decision-making.

Furthermore, the computational version of Word Soup developed in this project allows controlled experimentation. Unlike the smartphone application, the implemented framework enables systematic variation of grid size, letter distribution, dictionary constraints, and random seeds. This makes it possible to compare different computer players under identical initial conditions and to repeat experiments across large batches of grids. As a result, performance differences between greedy and multi-ply strategies can be analysed in a consistent and reproducible manner.

## 1.2 Aims and Research Questions

The primary aim of this project is to investigate how lookahead depth influences performance in the Word Soup game when implemented as a computational decision problem. In particular, the project evaluates whether increasing lookahead depth improves scoring and grid solvability and how these benefits compare to the associated computational cost.

To achieve this, several algorithmic computer players are implemented within a shared framework. These players differ only in their lookahead depth, ranging from greedy (1-ply) selection to deeper $n$-ply evaluation. All computer players are tested under controlled experimental conditions using an identical set of test grids and dictionaries.

The central research questions addressed in this project are:

1. Does increasing the lookahead depth lead to higher scores and a greater chance of clearing the grid?

2. Is there a point where increasing the lookahead depth further gives only small improvements compared to the extra computational costs?

3. How do different grid characteristics (for example, the proportion of

vowels or rare letters) affect the performance of the algorithms?

4. Can limited-depth lookahead strategies approach the performance of deeper search within the same grid setting?

### 1.3 Report Structure

The remainder of this report is organised as follows.

**Chapter 2** introduces the theoretical background required to frame Word Soup as an optimisation and search problem. Key concepts such as objective functions, search trees, ply depth, greedy algorithms, heuristic reasoning and simulation-based search are discussed to establish the foundations for the algorithmic design.

**Chapter 3** describes the game mechanics in detail, including the scoring system, gravity and horizontal shifting rules, termination conditions, and the definition of a game state. An illustrative example of human gameplay is provided to demonstrate how decisions affect the evolving grid.

**Chapter 4** presents the implementation of the algorithmic players. Starting from a random baseline, the framework progresses to a greedy $n$-ply structure, allowing systematic comparison of increasing lookahead depth.

**Chapter 5** outlines the experimental setup, including dictionary processing, grid generation protocols, evaluation metrics, and batch configurations used for benchmarking.

**Chapter 6** reports the experimental results, comparing different ply depths and analysing how grid characteristics influence performance.

**Chapter 7** discusses the interpretation of results, limitations of the current framework, and possible directions for future work.

Finally, **Chapter 8** concludes the report by summarising the main findings and their implications for limited-depth search strategies in sequential decision problems.

## 1.4 Illustrative Example of Human Play

To provide an understanding of the game mechanisms, this section presents a step-by-step example of how a typical human player interacts with a Word Soup board.




(a) Main menu of the Word Soup application.

(b) Initial grid configuration in Relaxed Mode before any move is made.

Figure 2: The Word Soup mobile application interface.

Figure 2a presents the initial interface of the Word Soup application. The

left panel shows the main menu, which provides standard navigation options such as starting a new game, adjusting game settings, and viewing records. For the purpose of this illustrative example, `Relaxed Mode` is selected. This mode removes time pressure and competitive constraints, allowing the focus to remain purely on understanding the game mechanics.

Figure 2b displays the starting grid configuration before any move has been made. The letters are generated according to a frequency-based distribution, discussed further later in this report. Each tile is colour-coded by letter value: yellow tiles indicate low-scoring letters, blue tiles represent mid-value letters, and red tiles indicate the highest-scoring letters, as detailed in Table 1. At this stage, the board represents the initial game state. No word has been selected, and the player is simply observing the arrangement of letters. A human player typically begins by scanning the grid to identify promising letter clusters, considering both immediate scoring opportunities and potential future combinations.




Figure 3: Example word selection: the player selects the word `WHEEL` by tracing adjacent letters.




Figure 4: Grid state after removing `WHEEL`: letters fall vertically to fill gaps, then the grid shifts horizontally to the chosen direction.

Figure 3 shows an example move where the player selects the word `WHEEL`. The word is formed by tracing a continuous path of adjacent letters on the grid, with each tile used at most once in the selection. Once the word is submitted, the selected tiles are removed, and the score for that word is added to the running total. For the word `WHEEL`, the value of the letters sums to 15 (W=4, H=4, E=1, E=1, L=5), giving a word score of $5 \times 15 = 75$ points (word length×letter value sum).

Figure 4 shows the resulting grid state immediately after the move. First, the remaining letters fall vertically to fill the empty spaces created by the

removal (gravity step). Next, the grid shifts horizontally towards the chosen side, which compresses the remaining tiles and reduces empty columns. This updated configuration becomes the new game state for the next word selection.




Figure 5: Second move selection: the player forms the word `LOCATE` by tracing adjacent letters.




Figure 6: Grid state after submitting word `LOCATE`. The selected tiles are removed, followed by vertical gravity and horizontal compression.

Figure 5 illustrates the second move following the removal of word `WHEEL`. In this step, the player selects the word `LOCATE`, again by tracing a connected sequence of adjacent letters. The word scores 72 points, computed as the sum of its letter values ($L = 5, O = 1, C = 3, A = 1, T = 1, E = 1$, giving a total of 12) multiplied by the word length of six. Once submitted, this score is added to the cumulative total displayed at the top of the screen.

Figure 6 shows the resulting configuration of the grid after `LOCATE` has been removed. As before, the update occurs in two stages. First, gravity is applied, causing remaining letters to fall vertically into empty spaces. Second, the grid undergoes horizontal movement, shifting tiles towards the chosen direction and eliminating empty columns. The total score increases accordingly, and the board transitions into a new state for subsequent decision-making.

Although `LOCATE` is not the highest scoring word available at this stage, it spans more letters across the grid, meaning its removal creates a larger structural change to the board. As letters fall and columns shift to fill the gaps left behind, previously non-adjacent letters may become neighbours, opening up new word paths that were not accessible.




Figure 7: Late-game configuration before termination. The remaining tiles are concentrated in a reduced region of the grid.




Figure 8: Final move and termination state. After the word ERN is submitted, no further valid words remain.

Figure 7 shows a late-stage configuration after several consecutive word removals. The grid has been substantially reduced, with remaining tiles clustered in a smaller region. At this stage, the number of available valid words is significantly lower than at the beginning of the game, reflecting the shrinking search space.

Figure 8 displays the final move. After submitting the word ERN, no further valid words can be formed from the remaining letters. The game therefore terminates due to the absence of legal actions. Notably, the grid is not fully cleared. As a result, the 500-point bonus awarded for complete clearance is

not achieved.

To have achieved the clearing bonus, the player would have needed to plan further ahead, prioritising moves that keep the remaining letters connected and accessible rather than simply selecting the highest scoring word at each step. In this game, the late stage configuration shown in Figure 7 suggests that earlier decisions left behind a fragmented arrangement of low-connectivity tiles which became increasingly difficult to incorporate into valid words. This illustrates the central trade off in Word Soup: maximising immediate score and achieving full board clearance are often conflicting objectives. A player focused purely on high value words may exhaust the accessible paths early, while a player that accounts for long-term grid structure may sacrifice some points in exchange for a greater chance of clearing the board. These observations motivate the algorithmic approaches explored in the remainder of this report, where different strategies are evaluated on both their scoring performance and their ability to clear the grid entirely.

# 2 Theoretical Background: Optimisation and Sequential Search

This chapter provides the theoretical background required to frame Word Soup as an optimisation and search problem. The aim is not to give a full review of all related methods, but to explain only the key ideas that guide the algorithm choices discussed later in the report.

Word Soup involves making a sequence of decisions under fixed rules, where each choice affects both immediate reward and future possibilities. Such problems are commonly studied within the fields of optimisation, heuristic search, and game-tree analysis. Understanding these frameworks helps clarify the trade-off between short-term gain and long-term performance that arise when designing algorithmic players.

The topics covered in this chapter focus on objective functions, search trees with limited lookahead, greedy decision-making, and the role of heuristics. These concepts provide the theoretical foundation for the players designs introduced in Chapter 4, ranging from random and greedy strategies to deeper lookahead approaches. In particular, they justify the structured progression from simple baseline strategies, such as random and greedy players, to more advanced lookahead approaches with increasing ply depth. This progression allows performance to be analysed systematically in terms of how additional foresight and search depth influence decision quality in Word Soup.

## 2.1 Strategy Games as Computational Problems

Strategy games can be naturally formulated as computational decision problems in which a player must select actions sequentially in order to maximise a given objective. At any point in the game, the current configuration defines a state, and each valid move corresponds to an action that transforms this state into a new one. The outcome of the game therefore depends not on a single decision, but on a sequence of interdependent choices made over time.

This sequential structure can be represented using a search tree, as illustrated in Figure 9. The root node corresponds to the current game state, while each outgoing edge represents a possible action. Each action leads to a new state, which in turn generates further possible actions. As this process continues, a branching structure emerges in which each path from the root represents a possible sequence of decisions.

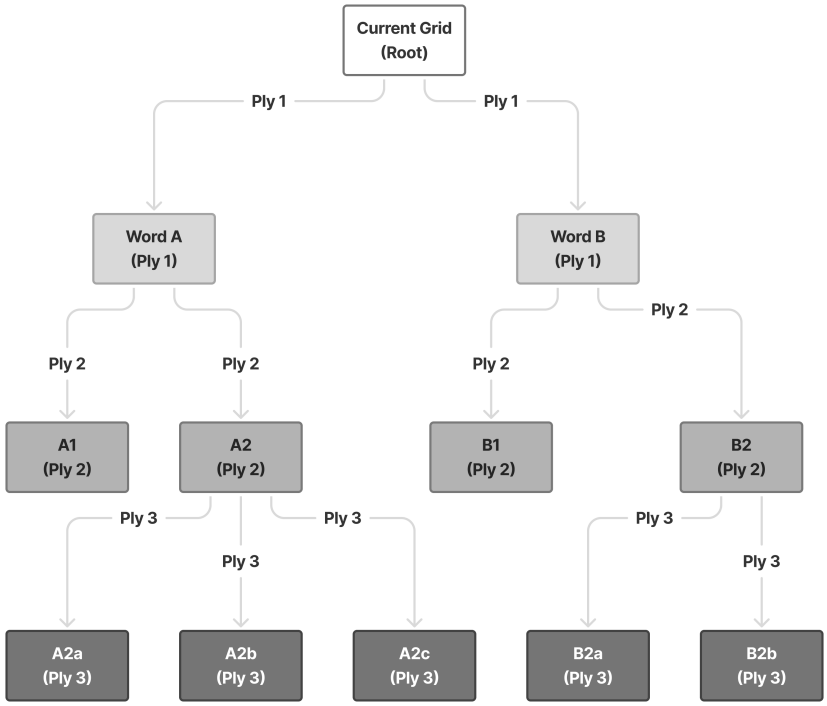



Figure 9: Illustration of a simplified search tree for Word Soup. Each node represents a possible grid state, and each edge corresponds to the selection of a valid word. The depth of the tree reflects the number of consecutive decisions considered.

In this representation, the depth of the tree corresponds to how far ahead the

decision process is considered. The first level represents immediate choices, while deeper levels represent future consequences of those choices. For example, selecting a word at the first level changes the grid configuration, which then determines the set of available words at the next level. In this way, each decision influences not only immediate reward but also the structure of the future possibilities.

A key challenge in such problems is that the number of possible states grows rapidly as more decisions are considered. Even with a moderate number of available actions at each step, the total number of possible consequences increases exponentially with depth. This makes it computationally infeasible to explore the entire search tree in order to find the optimal sequence of actions.

As a result, practical approaches rely on approximate methods that restrict how much of the tree is explored. These include strategies that consider only immediate outcomes, as well as methods that explore the tree to a limited depth in order to balance decision quality with computational cost. This trade-off between optimality and efficiency is central to the design of algorithmic computer players for Word Soup and motivates the use of search-based methods with limited lookahead, which are discussed in Section 2.3

## 2.2 Optimisation and Objective Functions

In optimisation problems, an objective function defines what it means for a solution to be considered good or optimal. In the context of Word Soup, the global objective for a fixed initial grid can be naturally defined as the total score achieved over the entire game, including any clear-board bonus. This provides a clear and static performance criterion at the level of a complete playthrough.

However, the optimisation process is inherently sequential. At each step, the player must select a word without knowing the full long-term conse-

quences of that choice. While the final objective (total score) is fixed, the local objective for choosing the next move is not immediately obvious. Each decision yields an immediate reward but also changes the grid configuration, thereby altering future word availability, solvability, and the possibility of clearing the board.

A natural short-term objective is score maximisation. Each selected word contributes a numerical score based on its letter values, and the total score accumulated over a game provides a simple and measurable performance criterion. Optimising for score alone encourages the selection of long words and high-value letters, particularly in the early stages of play.

Nevertheless, maximising immediate score does not necessarily lead to the best overall outcome. Some high-scoring word choices can fragment the grid or isolate difficult letter combinations, causing the game to terminate early. This is particularly important in Word Soup because clearing the grid yields a substantial bonus that can exceed the score of several typical word selections. As a result, a strategy that greedily selects high-value words may achieve strong short-term rewards but fail to secure the clear-board bonus, leading to a lower final score overall.

From an optimisation perspective, grid clearance is therefore not a separate objective, but a critical component of maximising the global objective of total score. This introduces a fundamental tension between maximising immediate reward and preserving future solvability under a fixed global objective. A strategy that prioritises immediate score may reduce future feasible moves, whereas a strategy that preserves favourable grid structure may accept lower short-term gains in order to enable higher cumulative rewards later.

From a formal optimisation viewpoint, the global objective remains a single quantity for a fixed initial grid, namely the total cumulative score including the clear-board bonus. However, the difficulty arises at the local decision

level. When selecting the next word, the algorithm does not directly observe how that choice will affect the final score, since future word availability and the possibility of clearance depend on the evolving grid structure. Consequently, the local objective function used during planning is only an approximation of the true global objective. A move that maximises immediate score may reduce future solvability and prevent grid clearance, thereby lowering the eventual total score despite appearing optimal in the short term. This mismatch between a well-defined global objective and an uncertain local evaluation is a key characteristic of sequential decision problems such as Word Soup.

Therefore, Word Soup is best formulated as a sequential optimisation problem in which algorithmic players must balance immediate reward against future consequences. The quality of a decision cannot be assessed solely by its instant score, but must consider its impact on the trajectory of future states. This perspective naturally motivates the use of search-based methods with limited lookahead, which are discussed in the following section.

## 2.3 Search Trees and Ply Depth

Sequential decision-making problems are naturally represented using search trees, where each node corresponds to a possible game state and each edge represents a valid action. In the context of Word Soup, a state is defined by the current grid configuration, and each action corresponds to selecting and removing a valid word. Starting from an initial grid, successive word selections generate a branching structure of possible future states.

In principle, an optimal strategy could be obtained by exploring the entire search tree and selecting the sequence of actions that maximises the final score. However, this is not computationally feasible in practice. At each stage of the game, the number of available words can be large, and every move alters the grid through letter removal and gravity effects. As a result, the branching factor is high and the structure of the tree evolves dynamically,

causing the total number of possible future states to grow exponentially.

To make the problem tractable, search-based methods restrict how far into the future decisions are evaluated. This is formalised using the concept of `ply depth`. A single ply corresponds to one decision made by the player. A 1-ply strategy evaluates only the immediate outcome of each available move, while a 2-ply strategy evaluates each move together with the best subsequent move that follows. More generally, a $k$-ply strategy considers sequences of $k$ consecutive decisions.

Increasing the ply depth allows the algorithm to account for longer-term consequences of its choices. In Word Soup, this is particularly important because early decisions can significantly influence the structure of the grid. For example, selecting a high-scoring word may remove valuable letters or fragment the grid, reducing the availability of future words and making it more difficult to clear the board. A shallow strategy (e.g. Greedy-1) may fail to detect these long-term effects, whereas deeper lookahead (e.g. Greedy-2 or Greedy-3) can reveal whether a move preserves or restricts future opportunities.

However, increasing ply depth comes at a substantial computational cost. Each additional level of lookahead multiplies the number of states that must be evaluated, leading to a rapid increase in runtime. This creates a fundamental trade-off between decision quality and computational efficiency. Shallow strategies are fast but potentially short-sighted, while deeper strategies provide better-informed decisions at the expense of higher computational cost.

In this project, ply depth is used as the primary mechanism for controlling this trade-off. The algorithmic players introduced in Chapter 4 differ mainly in the depth of lookahead they employ. By comparing strategies with different ply depths under identical conditions, it becomes possible to study how additional foresight affects performance, including total score, grid clear-

ance, and computational efficiency. Figure 9 illustrates how ply depth maps onto the branching structure of the search tree.

## 2.4 Greedy Algorithms and Heuristics

In sequential decision problems, it is often computationally infeasible to evaluate all possible future states. As a result, simpler decision rules are commonly used to approximate optimal behaviour. One of the most widely used approaches is the `greedy algorithm`, which selects the action that maximises an immediate objective at each step without explicitly considering long-term consequences. In this project, Greedy-1 is the direct implementation of this 1-ply rule, selecting at each step the word with the highest immediate score subject to the board-clearing priority.

In the context of Word Soup, a natural greedy strategy is to select the word that yields the highest immediate score. This corresponds to a 1-ply decision rule in which each available move is evaluated solely based on its direct reward, though additional prioritisation criteria are introduced in the implemented policies. Such strategies are computationally efficient, as they avoid exploring deeper levels of the search tree and require only a single evaluation per candidate action.

However, greedy decision-making is inherently short-sighted. Since it focuses only on immediate reward, it may ignore the impact of a decision on future states. In Word Soup, this limitation is particularly significant due to the dynamic nature of the grid. Selecting a high-scoring word may remove useful letters, fragment the grid structure, or isolate difficult letter combinations, thereby reducing the availability of future words and increasing the likelihood that the game terminates early. As a result, a purely greedy strategy may achieve strong short-term gains but fail to maximise the total score over the entire game.

To address this limitation, heuristic methods are often introduced to pro-

vide a more informative evaluation of candidate actions. A `heuristic` can be viewed as an approximate objective function that incorporates additional features beyond immediate reward. Instead of evaluating a move solely by its score, a heuristic may also consider properties of the resulting grid, such as word availability, structural connectivity, or the presence of difficult letters.

In Word Soup, heuristics can be used to balance the trade-off between immediate score and future solvability. For example, a heuristic may penalise moves that excessively fragment the grid or remove high-frequency letters that are likely to support future word formation. By incorporating such considerations, heuristic-based strategies aim to approximate the long-term objective more effectively than purely greedy methods, while still remaining computationally tractable.

In this project, a random player is used as a baseline to provide a reference point for performance. Unlike greedy or search-based strategies, the random player selects valid words uniformly without considering either immediate score or future consequences. This provides a reference point against which more structured approaches can be evaluated.

Greedy strategies therefore represent the first level of improvement over this baseline, as they incorporate an explicit objective by prioritising immediate score. More advanced methods, such as multi-ply lookahead and heuristic-based evaluation, further extend this idea by incorporating information about future states. By comparing these approaches under identical conditions, it becomes possible to assess how increasing levels of decision-making sophistication influence overall performance.

## 2.5 Dynamic Programming

Dynamic programming is a general optimisation technique used for problems that can be decomposed into overlapping problems with an optimal structure. Instead of solving the same problem repeatedly, the method stores pre-

viously computed results and reuses them when the same state reappears. This can greatly reduce computational cost when many different decision paths lead to identical intermediate configurations.

In principle, Word Soup can be viewed as a sequential optimisation problem over game states, so it is natural to ask whether dynamic programming could be applied. A full dynamic programming formulation would require each grid configuration to be treated as a state, together with the best achievable future score from that state onward. If the same state were reached through different sequences of word selections, its optimal value could be computed once and then reused.

However, in this project dynamic programming is not adopted as the main solution method. The primary reason is that the effective state space of Word Soup is extremely large. Each move changes the grid through letter removal, vertical collapse within columns, and horizontal shifting when columns become empty. As a result, even small differences in word choice can lead to distinct downstream configurations. Although overlapping problems may occur in principle, they are not expected to arise frequently enough in the practical search space explored here to make full storing and reusing previously computed results efficient.

A further difficulty is that the value of a state is tied to a large branching set of possible future words, which must still be enumerated in order to determine the optimal continuation. Consequently, dynamic programming does not remove the central challenge of the problem, namely the combinatorial explosion in the number of reachable states. Storing and comparing all encountered configurations would also introduce substantial memory overhead, especially for larger grids.

For these reasons, the methods used in this project focus instead on random baselines, greedy scoring rules, and limited-depth lookahead. These approaches do not attempt to solve the entire optimisation problem exactly,

but they provide a practical balance between decision quality and computational feasibility. Dynamic programming remains a useful theoretical point of comparison, as it highlights why exact optimisation is difficult for Word Soup and why approximate search-based methods are more appropriate in this setting. Specifically, the rapid divergence of grid states following each word removal means that previously computed state values cannot be reliably reused, which confirms that the limited-depth lookahead methods implemented in this project are appropriate choices.

## 2.6 Simulation Search and Stochastic Method

The search methods discussed in the preceding sections (greedy selection and fixed depth lookahead) share a common limitation: they evaluate each candidate move by reasoning forward only a small number of steps. The quality of a move is therefore assessed based on a short horizon, which may not capture consequences that only become apparent after many subsequent decisions.

An alternative approach is to estimate the value of a move by simulating the remainder of the game from that point, rather than reasoning analytically about future states. This is the central idea behind simulation-based search, and more specifically Monte Carlo methods. Instead of computing an exact value for each possible continuation, the algorithm runs a number of full-game simulations starting from a candidate state, and uses the average outcome across those simulations as an estimate of that state's value. The randomness involved in how the simulation plays out is what makes this a stochastic approach: no single simulation is exact, but the average over many simulations converges towards a reliable estimate.

Monte Carlo Tree Search (MCTS) is a widely studied algorithm that applies this idea within a search tree framework [3]. Rather than expanding the full game tree, MCTS selectively simulates promising paths and updates its estimates incrementally. The key advantage of this approach over

fixed-depth lookahead is that it can evaluate entire game trajectories, including outcomes far beyond the horizon that a greedy or shallow search can reach. This makes it especially well suited to problems where long-term consequences determine whether a sequence of decisions is ultimately successful, such as whether a board can be fully cleared. The Hybrid policy developed in this project draws on the same underlying principle of simulation-based evaluation, though it does not implement the full MCTS selection and back propagation cycle; the comparison is examined in Section 7.5.2.

# 3 Game Description: The Rules and Mechanics of Word Soup

This section formally defines the rules and mechanics of the Word Soup game studied in this project. The aim is to describe the game in a precise and implementation-independent manner, so that all algorithmic players operate under the same assumptions and constraints.

## 3.1 Rules of the Game

Word Soup is a word-search puzzle played on a rectangular grid of $12 \times 9$ letters. In this project, we consider a finite variant of the game in which the grid is generated once at the start and no new letters are introduced during play. The player repeatedly selects and removes valid words until the grid becomes unsolvable.

A word is formed by selecting a sequence of adjacent letters on the grid. Adjacency is defined to include horizontal, vertical, and diagonal connections. Each grid cell may be used at most once within a single word. All selected words must be at least three letters long and must correspond to valid English words contained in a predefined dictionary.




Figure 10: An example of a valid word selection in Word Soup. Letters are connected through horizontal, vertical, or diagonal adjacency, and each cell may be used at most once per word. The score for each word is computed as the product of its length and the sum of its letter values.

When a valid word is selected, all of its letter are removed from the grid. The affected cells become empty, creating gaps in the grid structure. After each removal, a gravity mechanism is applied to restore the grid to a stable configuration.

The gravity mechanism proceeds in two stages. First, within each column, letters fall downward to fill any empty spaces below them. Second, if any column becomes fully empty, all columns to its right shift horizontally to close the gap, compacting the grid leftward. No new letters are introduced at any stage, so the grid monotonically decreases in size as the game progresses.

The game continues in a turn-based manner following this cycle of word formation, removal, and grid stabilisation. The game terminates when no valid English word can be found anywhere on the grid. If all letters have been removed by this point, the grid is said to be fully cleared; otherwise, the grid is considered unsolvable.

## 3.2 Scoring System

Each valid word selected by the player contributes to the total score. The score awarded for a word depends on its length and the individual letter values defined by the game's own scoring system. Each letter has a fixed point value as shown in Table 1. The total score for a word is computed as the product of its length and the sum of the point values of its letters, so that longer words yield disproportionately higher scores.

Table 1: Letter scoring system used in Word Soup

| Points | Letters |
|---|---|
| 1 | A E O R S T |
| 2 | D G N U |
| 3 | B C I M P |
| 4 | F H V W Y |
| 5 | K L |
| 8 | J X |
| 10 | Q Z |

In addition to word-level scoring, a fixed bonus of 500 points is awarded if the player successfully clears the entire grid. This clear-board bonus rewards strategies that prioritise long-term solvability rather than short-term score maximisation. The total score for a game is therefore the sum of all individual word scores plus the 500-point bonus when applicable.

Figure 11 shows an example of a Word Soup grid used in gameplay. The game interface visually encodes letter values using different colours, allowing players to quickly identify high-value tiles. In particular, yellow tiles correspond to 1 point letters (e.g., A, E, O, R, S, T), green tiles to 2 points (e.g., D, G, N, U), blue tiles to 3 points (e.g., B, C, I, M, P), purple tiles to 4 points (e.g., F, H, V, W, Y), orange tiles to 5 points (e.g., K, L), pink tiles to 8 points (e.g., J, X), and red tiles to 10 points (e.g., Q, Z).

The letter point values used in Word Soup are similar to those in `Scrabble` but are not identical. In standard Scrabble, the score for a word is the sum of its letter values, with additional multipliers applied via premium board squares [8]. Word Soup does not use premium squares or blank tiles. In-

stead, the word length acts as a direct multiplier, meaning that longer words yield disproportionately higher rewards regardless of their position in the grid. Rare letters such as Q, X, Z, and J retain high point values in both systems, reflecting their infrequent use in English.

This colour-coding provides an immediate visual indication of letter value during gameplay, reinforcing the scoring scheme defined in Table 1. As a result, high-value letters such as Q and Z (red) and J and X (pink) are strategically important, as they can significantly increase the score of individual words despite being relatively rare on the grid.

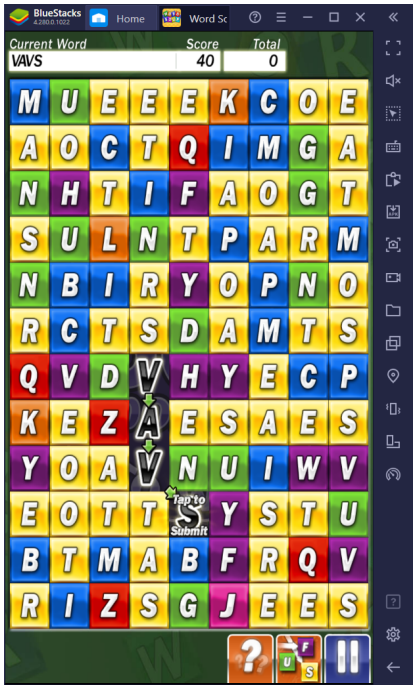



Figure 11: Example Word Soup grid illustrating colour-coded letter values and scoring interface.

### 3.3 Grid Evolution and Gravity

As letters are removed and gravity is applied, the grid evolves dynamically throughout the game. Word removals can significantly alter the local and global structure of the grid, changing both the availability of future words and the overall shape of the board.

Since columns may be removed entirely, the grid width can decrease over time. This shrinking behaviour introduces additional strategic considerations, as early word choices can strongly influence the difficulty of later stages of the game. Furthermore, a player can in principle exploit the gravity mechanism deliberately: by selecting words that cause specific letters to fall into new positions, it becomes possible to form longer or higher-scoring words that were not previously reachable, and to arrange the remaining letters in a configuration that makes full grid clearance more viable.




(a) Before: the word *VAVS* is selected along a vertical path in column 4 and 5.




(b) After: *VAVS* is removed and gravity is applied. Letters in the columns fall downward, leaving two empty cells at the top. The available words in that region change as a result.

Figure 12: An example of grid evolution after one move in Word Soup. Removing *VAVS* scores 40 points and triggers the gravity mechanism, which reshapes the local grid structure and alters the set of words reachable in subsequent moves.

### 3.4 Success Criteria

Two definitions of success are considered in this project. The first is score maximisation, where the objective is to achieve the highest possible total score under the game rules. The second is grid clearance, where success is defined as removing every letter from the grid before the game becomes unsolvable. Both measures capture different aspects of player performance and reflect distinct strategic priorities. Score maximisation rewards imme-

diate efficiency and the ability to identify high-value words quickly, while grid clearance rewards foresight, planning, and the capacity to maintain a playable grid over many turns.

Expert human players tend to prioritise grid clearance over pure score maximisation. In a sample of ten games played by an experienced human player, a clear rate of approximately 80% was observed, with a mean total score of approximately 1949 points including the clear-board bonus. This suggests that skilled human play naturally favours long-term solvability, accepting lower immediate word scores in order to preserve the structure of the grid for future moves as discussed further in Section 7.2.

These two objectives are not always aligned. High-scoring words may leave behind awkward letter configurations that prevent full clearance, while low-risk choices that focus on keeping the grid playable may sacrifice score in order to maintain solvability. This tension creates a meaningful trade-off between short-term gain and long-term success. In practice, players and algorithms must balance these competing goals, deciding when to prioritise immediate points and when to preserve the grid's structure for future opportunities. Understanding this balance is central to evaluating the effectiveness of different strategies and provides the foundation for the algorithmic approaches explored in later sections.

## 3.5 Game State and Player Decisions

At any point during play, the state of the game is fully determined by the current configuration of the letter grid. This configuration specifies the position of all remaining letters and implicitly defines the set of valid words that can be formed at that stage. No hidden information is present, and the player has complete access to the grid at all times.

A single decision in the game consists of selecting one valid word from the current grid. This choice determines both the immediate score obtained and

the resulting grid state after letter removal and gravity are applied. Since word selection directly affects future word availability, each decision has both short-term and long-term consequences.

The game can therefore be viewed as a sequential decision-making process, where each move transforms the current grid into a new state with different constraints and opportunities. The scale of this decision problem is significant: at any given move, a player may have hundreds or thousands of valid words to choose from, yet selecting any one of them permanently alters which words remain reachable in all future moves. Since the grid shrinks monotonically and no letters are ever reintroduced, every decision is irreversible. This property distinguishes Word Soup from many combinatorial games, as there is no opportunity to recover from a poor early choice. The order in which words are selected is therefore critical, since early decisions may open up or permanently foreclose later possibilities. This structure makes Word Soup particularly great to algorithmic approaches that reason about future grid states rather than acting on immediate reward alone.

This formulation provides a clear distinction between the rules of the game and the strategies used to play it. While the game mechanics define how the grid evolves after each move, the choice of which word to select at each step is left entirely to the player. The following chapter focuses on how different algorithmic players exploit this structure to make decisions under the same fixed rules.

# 4 Algorithmic Players

An algorithmic player is a coded strategy that selects moves automatically according to a fixed set of rules. This chapter introduces a series of algorithmic players designed to play Word Soup under the fixed rules described in Chapter 3. Each player differs in how it evaluates possible moves and how far it reasons about future grid states. The objective is not only to compare performance, but to understand how increasing levels of foresight influence decision quality in a sequential, state-dependent game.

All algorithmic players operate deterministically on a given grid configuration and share the same underlying representation of the game state. The primary distinction between players lies in their decision-making depth, ranging from a purely greedy strategy that evaluates only the immediate move to multi-ply strategies that anticipate the effects of future word selections. By structuring the players in this progressive manner, it becomes possible to isolate the impact of lookahead depth on both score and grid solvability.

Throughout this chapter, the term "player" refers to an algorithmic strategy rather than a human participant.

## 4.1 Shared Components

Despite their differences, all algorithmic players in this project rely on a common set of components that define how the game state is processed and evaluated. These shared components ensure consistency across players and allow differences in performance to be attributed solely to decision-making strategy rather than implementation details.

At the core of each player is a representation of the current grid configuration. From this representation, the algorithm enumerates all valid words of at least three letters that can be formed according to the adjacency and reuse rules defined in Chapter 3. This word enumeration step produces the set of legal moves available at a given state. For the simplest policy, all

enumerated words are considered directly. For policies that reason further ahead, the candidate set is pruned to the highest scoring words before lookahead is applied, reducing the computational cost without discarding the most promising options.

Each algorithm also shares a common scoring function, which assigns a numerical value to a candidate word using the same formula defined in section 3.2: the product of the word's length and the sum of the value of its letters. This means the function only measures how good a word is right now, without thinking about how that choice might affect later moves. The way each player uses this score depends on how far ahead it plans: a simple player picks the word with the highest immediate score, while a deeper one also considers how that choice will shape future opportunities.

Finally, all players apply the same grid update mechanism after a word is selected. This includes letter removal, vertical gravity within columns, and horizontal column shifting when empty columns occur. By enforcing identical state transition rules, the algorithms differ only in how they choose actions and in the depth of search they apply before committing to a move.

These shared components form the basic framework that all the different player algorithms rely on. They ensure that every player starts from the same rules and mechanics before adding more advanced decision-making layers.

## 4.2 Random Player (Baseline)

The random player is included as a baseline strategy to provide a reference point for evaluating decision quality in Word Soup. Unlike all other algorithmic players considered in this project, the random player does not attempt to optimise score or preserve future solvability. Its decisions are made without any form of strategic evaluation.

At each turn, the random player enumerates all valid words that can be

formed from the current grid configuration. One word is then selected uniformly at random from this set and applied to the grid. No preference is given to word length, letter value, or potential impact on future grid states.

Since the random player operates under the same game rules and state transition mechanics as all other players, any variation in its performance compared to other strategies can be directly linked to its lack of informed decision-making.

The presence of this baseline allows improvements observed in greedy and lookahead-based players to be interpreted meaningfully. In particular, it distinguishes whether observed gains in score or grid clearance result from purposeful strategic reasoning or are merely the outcome of random variation.

## 4.3 Greedy n-Ply Framework

### 4.3.1 1-Ply ($n = 1$)

The greedy-1 player represents the simplest form of informed decision-making in Word Soup. Unlike the random player, which selects words without preference, this strategy evaluates available moves using the game's scoring function and always chooses the word that yields the highest immediate reward. The term "1-ply" indicates that the algorithm considers only the current move and does not reason about subsequent grid states.

At each turn, the algorithm enumerates all valid words that can be formed from the current grid configuration. Each candidate word is assigned a score based solely on its letter values. The player then selects the word with the maximum score and applies it to the grid, after which the standard gravity and grid update rules are enforced. To ensure reproducibility, the algorithm follows a consistent tie-breaking procedure: when multiple words share the same score, it selects the one that appears first in a predetermined ordering, such as alphabetical order or by reading sequence on the grid.

By prioritising immediate score maximisation, the greedy-1 player introduces a clear improvement over random selection. It consistently exploits high-value letters and longer words, leading to higher average scores in early stages of the game. However, because the algorithm does not consider how a word choice affects future move availability, it may produce decisions that are locally optimal but globally harmful.

For example, consider a late-game grid where only a small number of letters remain. Suppose two short words are available, each worth a modest score, and playing them in sequence would clear the grid entirely and earn the 500-point bonus. A third word is also available on the current board that scores higher than either short word individually, but removing it leaves behind a letter combination that cannot form any valid word. The greedy-1 player selects the high-scoring word immediately, forfeiting the clear bonus and ending the game with letters still on the board. A player that considers even one move ahead would recognise that the two modest words together, plus the bonus, yield a substantially higher total.

In particular, selecting a high-scoring word can fragment the grid or isolate low-connectivity letters, reducing the number of valid words available in later turns. This limitation highlights the tension between short-term gain and long-term solvability and motivates the introduction of deeper lookahead strategies. The following section extends this approach by considering pairs of consecutive decisions in order to anticipate the immediate future impact of a move.

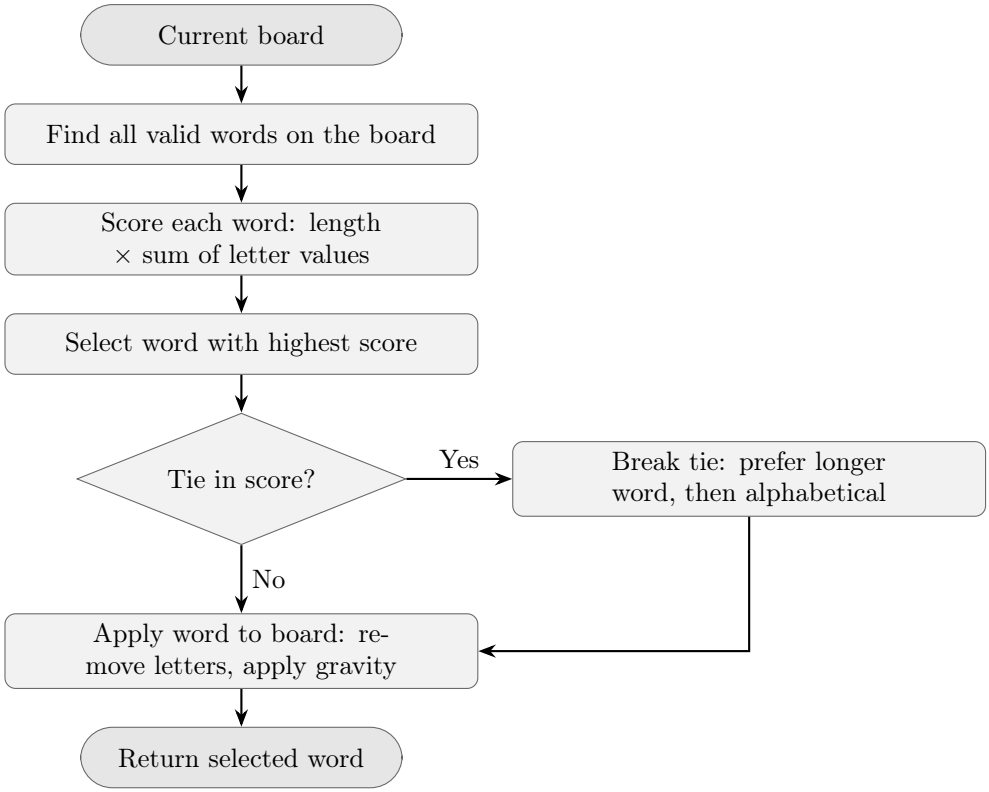



Figure 13: Decision process of the 1-ply greedy player. At each turn, the algorithm selects the word with the highest immediate score, with ties broken by word length and then alphabetical order. No lookahead is performed.

The degree to which this limitation affects outcomes depends on structural properties of the grid, such as vowel ratio and letter connectivity, which are examined in Chapter 6.

### 4.3.2 2-Ply ($n = 2$)

The greedy-2 player extends the greedy-1 player by introducing limited lookahead in the decision-making process. Rather than selecting a word based solely on its immediate score, this player evaluates the consequences of a choice by considering the best possible second move that could follow. In this context, "2-ply" refers to a two-step lookahead consisting of the current

move and the immediate subsequent move.

At each turn, the algorithm considers all valid words that can be selected from the current grid. For each candidate first word, the resulting grid state is computed after letter removal and gravity are applied. From this updated grid, the algorithm then identifies the highest-scoring valid word that could be played next using the same greedy evaluation as the 1-ply player. The combined score of this word pair is used as the evaluation criterion for the initial decision.

To illustrate, suppose the current grid contains a word $w_1$ scoring 40 points, after which the best available follow-up word scores 60 points, giving a two-move total of 100 points. A second option $w_2$ scores only 30 points immediately, but the resulting grid state allows a follow-up word worth 80 points, giving a two-move total of 110 points. The greedy-1 player selects $w_1$ because it has the higher immediate score. The greedy-2 player selects $w_2$ because the combined score over two moves is greater, demonstrating how even a single step of lookahead can reverse the preference ordering between candidates.

By selecting the first word that maximises the total score over two consecutive moves, the greedy-2 player looks ahead one step to consider the immediate future impact of its choice. This allows the algorithm to avoid choices that appear attractive in isolation but severely restrict the next available move. As a result, decisions hold up better against grid breakups and early isolation of tricky letter combinations.

Compared to the greedy-1 player, this strategy represents a qualitative improvement in decision quality. It introduces foresight while remaining computationally manageable, as only one additional layer of lookahead is required. Even though the algorithm still only looks one move ahead, adding this second step helps it make choices that not only give an immediate benefit but also keep good options open for the next move.

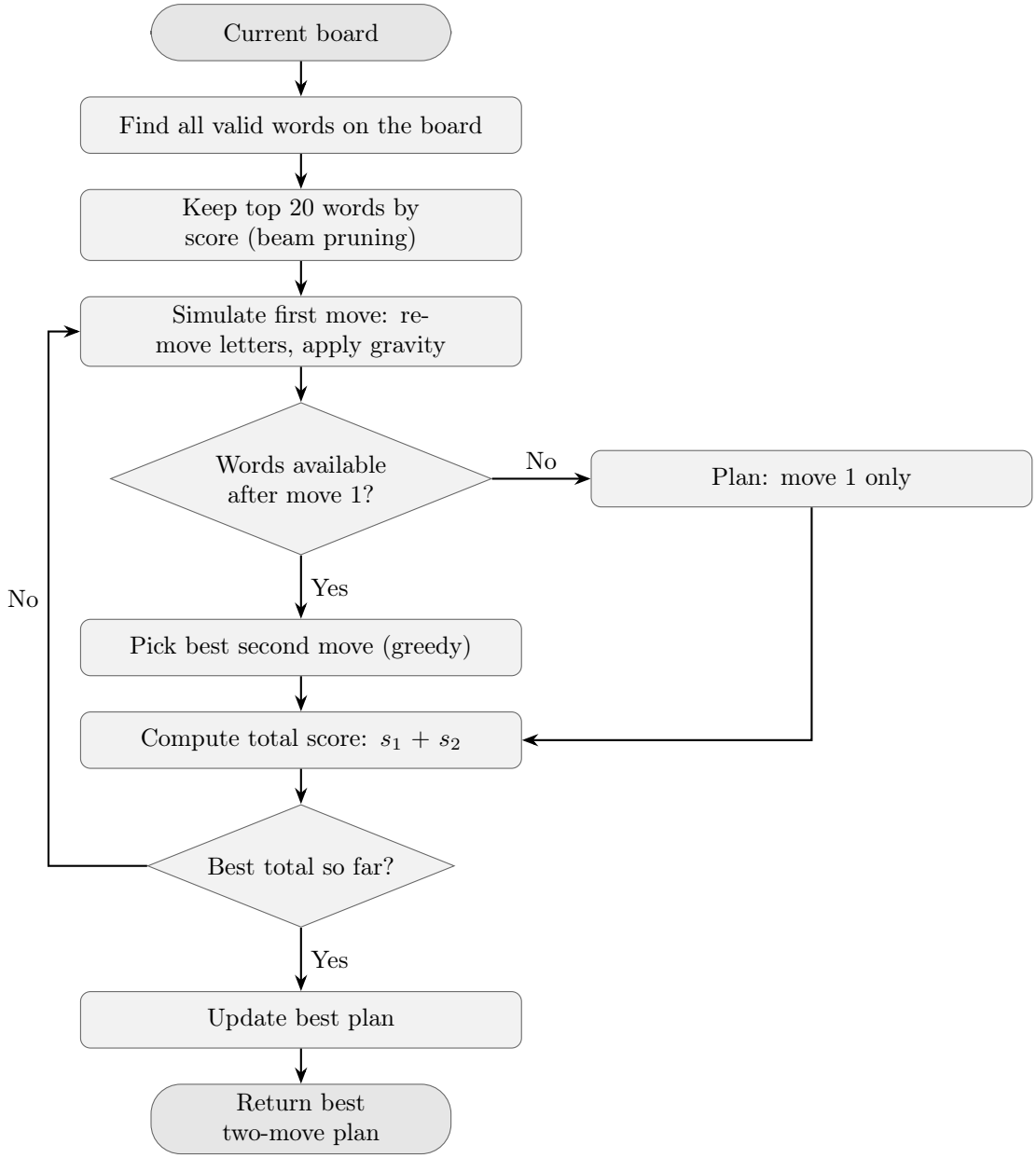



Figure 14: Decision process of the greedy-2 player. At each turn, the algorithm simulates all candidate first moves and picks the word pair that maximises the combined two-move score.

Figure 14 summarises the decision process of the greedy-2 player. Starting from the current board, all valid words are identified and the candidate set is pruned to the top 20 by score. For each candidate, the resulting board state is simulated and the best available second move is determined using the same greedy criterion as the 1-ply player. The combined score of the two moves is used to rank all first-move candidates, and the plan with the highest total is committed. If no second move is available after a first move, that candidate is evaluated on its immediate score alone.

The limitations of this approach arise when the impact of a decision extends beyond two moves. In such cases, a locally optimal word pair may still lead to unfavourable grid structures later in the game. This observation motivates the introduction of deeper lookahead strategies, which consider longer sequences of decisions and are discussed in the following subsection.

### 4.4 n-Ply Generalisation

The greedy lookahead strategy can be generalised to an $n$-ply framework, where sequences of $n$ consecutive decisions are evaluated before selecting the initial move. The parameter $n$ determines the depth of lookahead and controls how far into the future the algorithm evaluates consequences.

At each turn, all valid words available in the current grid are considered as candidate first moves. For each candidate, the resulting grid state is computed. The procedure is then applied recursively for up to $n$ moves, generating a sequence of decisions. The cumulative score across these $n$ moves is used to evaluate the quality of the initial word choice. The word that maximises this cumulative evaluation is selected.

When $n = 1$, the strategy reduces to the standard greedy algorithm that selects the highest-scoring immediate word. When $n = 2$, the algorithm evaluates pairs of moves. When $n = 3$, it evaluates triples of moves. Larger values of $n$ are possible in principle, but the number of candidate sequences

grows exponentially with depth, making exhaustive evaluation computationally infeasible beyond small values of $n$.

Figure 15 illustrates the decision process for the 3-ply case.

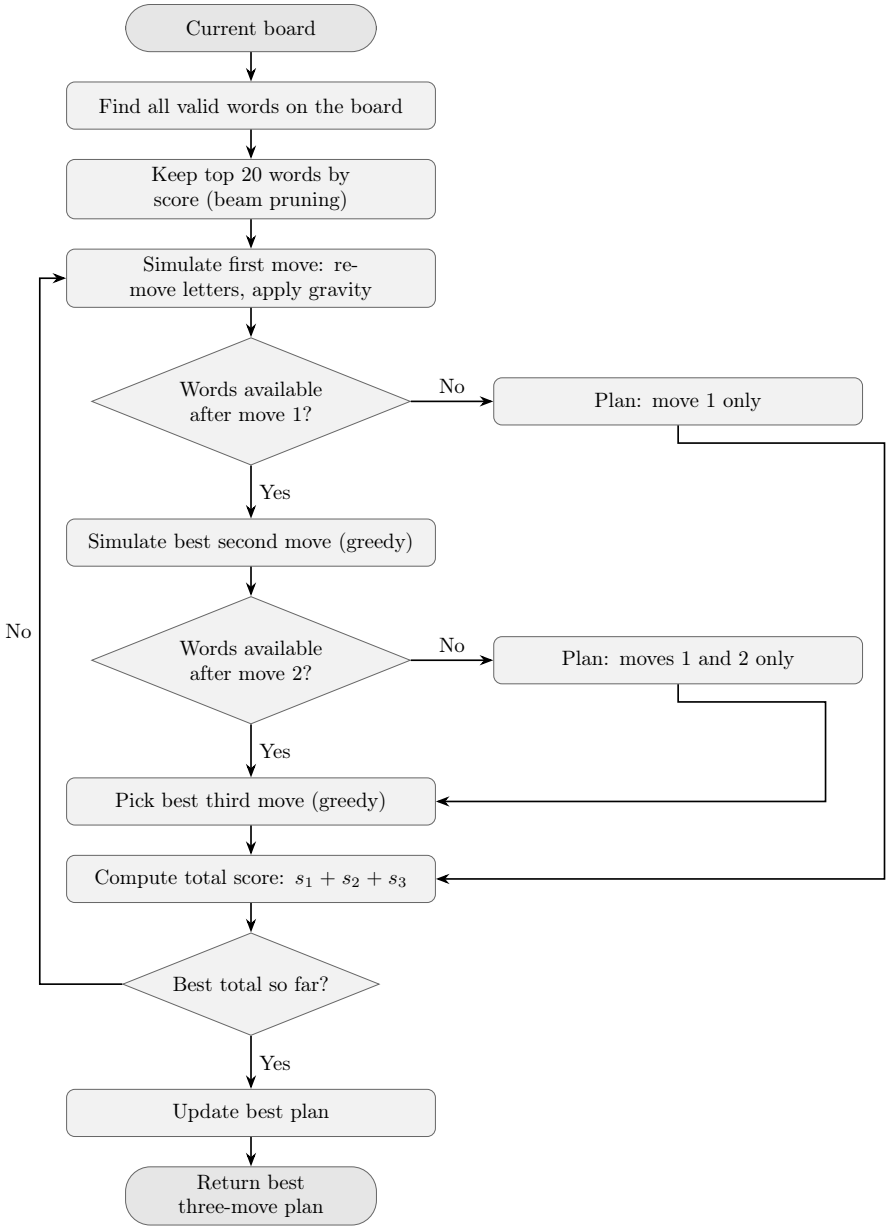



Figure 15: Decision process of the 3-ply greedy player. At each turn, the algorithm evaluates all candidate first moves over a three-move lookahead and selects the word that maximises the combined score across up to three consecutive moves.

In this study, implementations are developed and tested for $n \in 1, 2, 3$. This allows systematic analysis of how increasing lookahead depth influences score, solvability, and runtime.

### 4.5 Heuristic / Hybrid Extensions

In addition to the core players described previously, a hybrid strategy was implemented to explore whether a combination of shallow evaluation, selective search, and adaptive decision-making could improve performance while maintaining computational efficiency.

The hybrid player operates in two stages. First, all legal moves are evaluated using a shallow scoring rule based on immediate reward. From this set, only the top-$k$ candidates are retained for further consideration. This acts as a pruning mechanism, reducing the number of moves that require deeper evaluation and therefore limiting computational cost. In this implementation, $k$ is fixed at 20, which provides a balance between retaining promising candidates and controlling runtime.

In the second stage, the shortlisted moves are evaluated more deeply using a rollout strategy, which is the process of playing out a sequence of future moves from a candidate position to estimate its long term value. Rather than fixing a single lookahead depth, the hybrid player adapts its decision policy based on the proportion of tiles remaining on the board. When the board is relatively full, a greedy-1 policy is used to maintain efficiency. As the game progresses and the number of remaining tiles decreases, the player switches to 2-ply and eventually 3-ply evaluation. In this implementation, threshold values of approximately 50% and 20% of the initial tile count are used to determine these transitions.

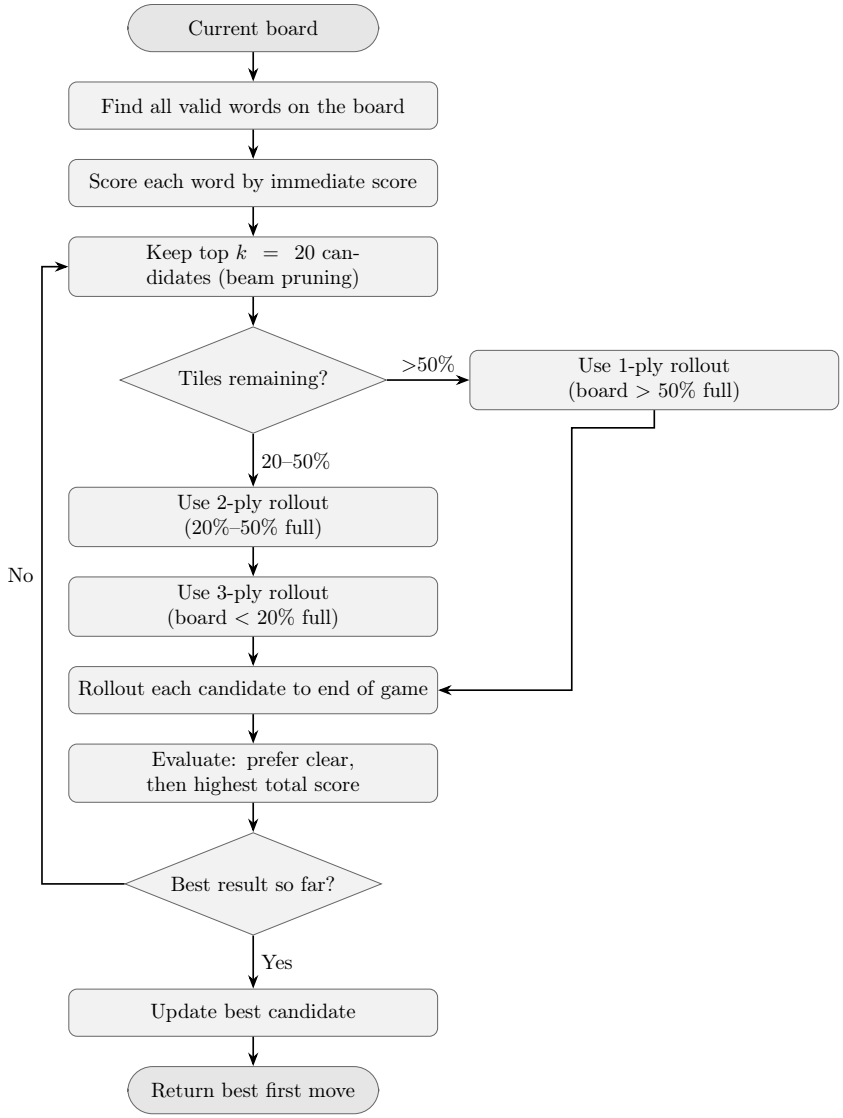



Figure 16: Decision process of the Hybrid player. Candidates are first pruned to the top 20 by immediate score. The rollout depth adapts based on the proportion of tiles remaining: 1-ply when the board is relatively full, switching to 2-ply and 3-ply as the game progresses. The candidate whose rollout produces the best outcome (prioritising grid clearance over raw score) is selected.

This adaptive mechanism reflects the changing nature of the problem. In

the early stages of the game, the branching factor is large and the grid offers many viable paths forward, so individual decisions have relatively small long-term impact and shallow evaluation is sufficient. In contrast, in later stages, the grid becomes more constrained and each move can significantly affect the possibility of clearing the board, making deeper lookahead more valuable.

Finally, move selection prioritises outcomes that result in a complete board clear, followed by maximisation of total projected score. This reflects the scoring structure of the game, in which clearing the board yields a substantial bonus and is therefore a key objective.

Overall, the hybrid approach can be interpreted as a compromise between purely greedy strategies and full-depth search. It avoids the computational cost of exhaustive exploration while incorporating additional information about future states, allowing it to make more informed decisions than standard greedy methods.

# 5 Experimental Setup

## 5.1 Dictionary Processing

The Word Soup solver requires a dictionary of valid English words in order to determine whether a sequence of letters on the grid forms a legitimate word. For this project, the dictionary file `dictionary_actualy_game.txt` was provided by the project supervisor as a pre-filtered list containing one word per line, with a total of 278,688 valid entries and a minimum word length of three letters. This is considerably larger than the word lists used in simpler word games such as Wordle, which operates on a fixed set of five-letter words, and reflects the open-ended and combinatorial nature of the Word Soup search space. Since the file had already been prepared for computational use, no additional cleaning or filtering was required during the experiments.

In general, however, dictionary preprocessing is an important step when using external word lists. Raw dictionaries often contain capitalised entries, punctuation, hyphenated words, or other symbols that are not suitable for grid-based letter matching. When such dictionaries are used, typical preprocessing steps include converting all entries to lowercase, removing words containing non-alphabetic characters, and ensuring that each line represents a single valid word. These steps ensure consistency and prevent invalid matches during the search process.

One further observation is that the dictionary may contain pairs of words that are reverses of one another, such as `BAT` and `TAB`. Since the Word Soup scoring formula does not distinguish between a word and its reverse, one word from each such pair could in principle be removed from the dictionary without affecting the solver's ability to achieve any particular score. This reverse word pruning was not applied in the current implementation, as the priority was correctness over dictionary minimisation, but it represents a straightforward optimisation that could reduce the number of candidates considered during the grid search.

After loading the dictionary, the words are inserted into a Trie (prefix tree) data structure [5]. Originally proposed by Fredkin (1960), the Trie is a well-established data structure in computer science, widely used in applications that require fast prefix-based retrieval, such as autocomplete systems and natural language processing pipelines. In this project, the Trie enables efficient prefix and word lookups during the grid search. In particular, two operations are required: checking whether a given string is a valid prefix of any dictionary word, and checking whether a string corresponds to a complete dictionary word.

The prefix operation is especially important during the depth-first search used to discover words on the grid. If a partial sequence of letters does not correspond to any prefix in the dictionary, the search branch can be terminated immediately, allowing large parts of the search space to be pruned early.

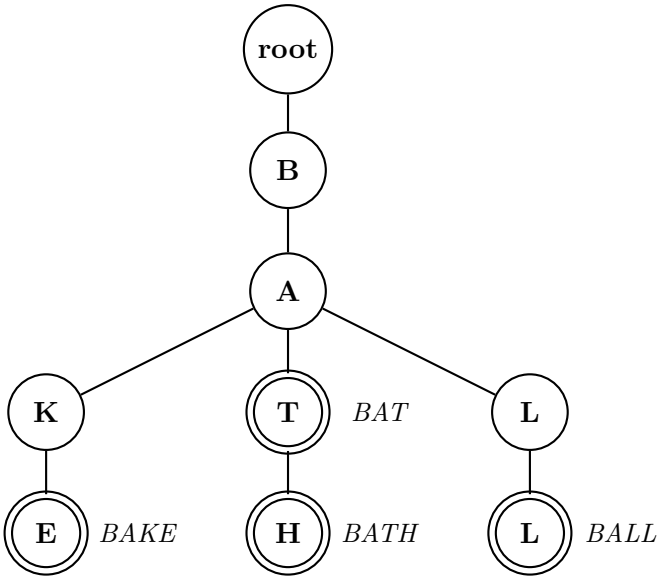



Figure 17: A partial Trie constructed from the words `BAKE`, `BAT`, `BATH`, and `BALL`. Each node represents a letter; the root is empty. Nodes marked with a double circle indicate the end of a valid word. During the grid search, if a partial path does not exist in the Trie, the branch is pruned immediately.

**Word score distribution.** To characterise the scoring landscape of the dictionary, the score of every word was computed using the Word Soup formula: word length multiplied by the sum of its letter values. Table 2 summarises the distribution across score bands, and Figure 18 shows the full histogram.

Table 2: Word count and proportion by score band across the full dictionary ($n = 278{,}688$ words).

| **Score band** | **Words** | **Proportion** |
|---|---|---|
| Below 100 | 59,414 | 21.32% |
| 100 – 149 | 55,000 | 19.74% |
| 150 – 199 | 51,463 | 18.47% |
| 200 – 299 | 63,127 | 22.65% |
| 300 – 447 | 35,342 | 12.68% |
| 448 – 628 (top 5%) | 11,439 | 4.10% |
| 629 or above (top 1%) | 2,903 | 1.04% |

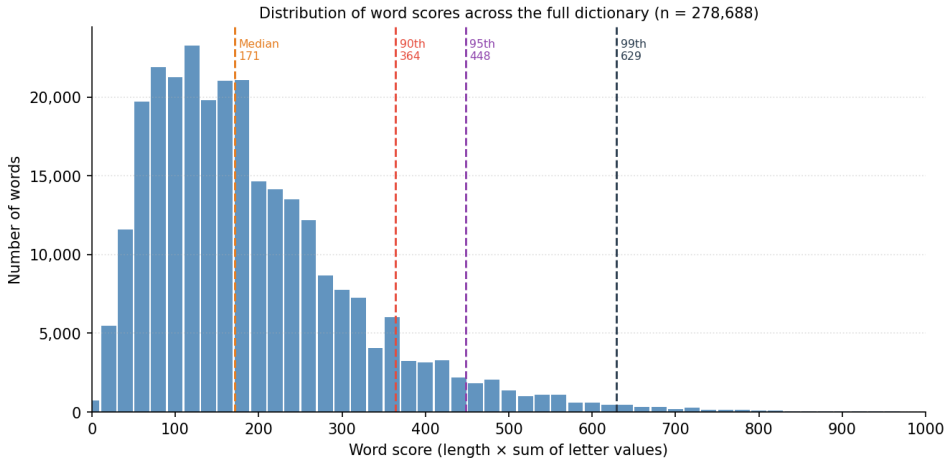



Figure 18: Distribution of word scores across the full dictionary ($n$ = 278,688). The distribution is right-skewed: the majority of words score below 200 points, while the upper tail is driven by long words containing high-value letters. Dashed lines mark the median (171), 90th (364), 95th (448), and 99th (629) percentiles.

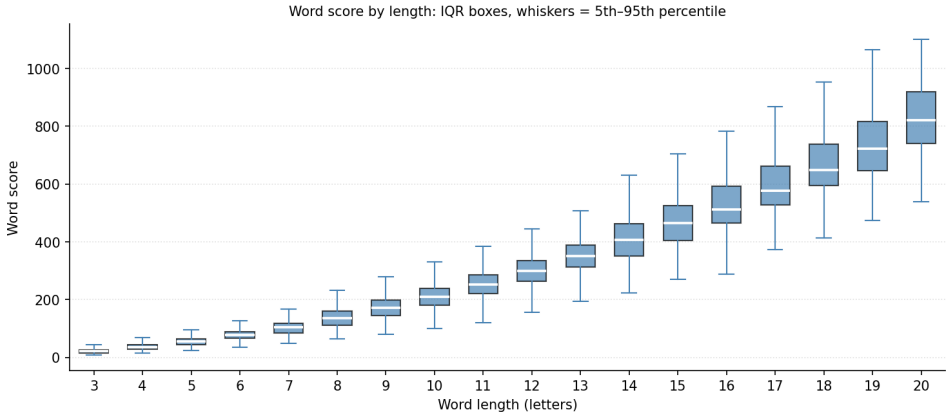



Figure 19: Word score by word length. Each box shows the interquartile range; whiskers extend to the 5th and 95th percentiles. Mean score rises steeply with length: 3 letter words average 22 points, while 15 letter words average 468 points and 20 letter words average 826 points.

The distribution is strongly right-skewed. Most words (78.7%) score 100

points or above, and the median score is 171 points, close to the mean of 200 points. The upper tail is sparse: only the top 5% of words score 448 points or above, and the top 1% score 629 or above. The highest-scoring words in the dictionary reach scores above 1,400 points (e.g. `immunoelectrophoretically`, score 1,475), though such words require letter combinations that are extremely unlikely to be reachable on any given grid.

Figure 19 makes clear that word length is the dominant driver of score. A 3 letter word scores at most 69 points, while even an average 15 letter word scores 468 points, approaching the value of the 500 point clearing bonus. This has two implications for the algorithms. First, all greedy and Hybrid policies are incentivised to prefer longer words, since the scoring formula multiplies letter values by length. Second, the number of genuinely high value targets is small: fewer than 14,342 words (5.14%) score above 448 points, and many of these will be eliminated by the initial grid filter because their required letters are not present on the board. The viable set of high value targets on any given grid is therefore small enough to enumerate explicitly, which is what motivates the targeted long word construction direction discussed in Section 7.5.

## 5.2 Grid Generation

To evaluate the different Word Soup players under controlled conditions, starting grids were generated before each experiment. All experiments use the full game configuration with a grid size of $12 \times 9$.

Each starting board was generated using a letter-frequency sampling method. In this method, letters are sampled according to their frequency in the project dictionary, where letters that appear more often in dictionary words are more likely to be selected. This produces grids with a realistic mix of letters and tends to generate boards where valid words can actually be found. The letter frequencies used here are derived from the project dictionary rather than from observed distributions in the original Word Soup game, as the latter

is not publicly documented. This approach is a reasonable approximation, since the dictionary was sourced from the game itself and is expected to reflect a similar letter composition.

Randomness was controlled using a fixed seed of `123`. A fixed seed means the same sequence of grids is produced every time the program runs, so results can be reproduced and compared consistently.

For the evaluation, 100 starting grids were generated and tested against each player. This number was chosen as a practical balance between statistical coverage and computational cost: 10 grids would be too few to draw reliable conclusions, while 1000 grids would result in prohibitively long runtimes given the higher ply policies. The five players considered are Random, Greedy-1, Greedy-2, Greedy-3, and Hybrid. Each player was evaluated on the same 100 grids under identical conditions, giving a total of 500 simulated games. All grid generation and word validation use the same dictionary file, `dictionary_actual_game.txt`, described in Section 5.1. Using a shared dictionary ensures that word availability is consistent across all experiments.

## 5.3 Metrics Collected

For each simulated game, a set of metrics was recorded to allow comparison across the five player policies. The metrics fall into three categories: grid characteristics measured before the game begins, in game statistics averaged across all moves during play, and performance outcomes recorded after the game ends.

Table 3 summarises all metrics collected, grouped by when they are recorded during the experiment.

Table 3: Summary of Metrics Collected

| Metric | Type | Description | Typical Range |
|---|---|---|---|
| *Grid Characteristics (recorded before game)* | | | |
| `startWords` | Grid | Number of valid words formable in a single move at the start | 1501 – 3964 |
| `startTopScore` | Grid | Score of the highest-scoring word at the start | 171 – 448 |
| `vowelRatio` | Grid | Proportion of cells containing a vowel | 0.27 – 0.47 |
| `rareRatio` | Grid | Proportion of cells containing Q, X, Z, or J | 0.00 – 0.04 |
| *In-Game Statistics* | | | |
| `branchingAvg` | In-game | Average number of valid words available per turn | 417 – 1408 |
| `ppMoveAvg` | In-game | Average score earned per move | varies by policy |
| `wordLenAvg` | In-game | Average length of words chosen by the player | varies by policy |
| `moveMax` | In-game | Score of the highest-scoring single move made | 65 – 561 |
| *Performance Outcomes (recorded after game)* | | | |
| `cleared` | Outcome | 1 if all letters were cleared, 0 otherwise | binary |
| `cellsLeft` | Outcome | Number of letters remaining when the game ended | 0 – 22 |
| `scoreNoBonus` | Outcome | Total score without the clearance bonus | 682 – 2767 |
| `total_score_with_bonus` | Outcome | Total score including the 500-point clearance bonus | 682 – 3267 |
| `moves` | Outcome | Total number of words played during the game | 10 – 26 |
| `scorePerCell` | Outcome | Total score divided by number of starting cells (108) | 7.49 – 22.32 |
| *Runtime (recorded per experiment run)* | | | |
| `runtime_sec` | Runtime | Total time for the player to complete the game | varies by policy |
| `grid_gen_sec` | Runtime | Time taken to generate the starting grid at runtime | <1 s |

## 5.4 Batch Configurations

Five separate batches were run, one for each player policy: Random, Greedy-1, Greedy-2, Greedy-3, and Hybrid. Each batch consisted of 100 simulated games, giving a total of 500 games across the full experiment.

All five batches used the same fixed random seed of `123`. The seed is set once at the start of each batch run, and grids are generated sequentially at runtime using that fixed RNG state. This means every batch produces the same sequence of 100 starting grids in the same order, which is functionally equivalent to pre-generating and storing the grids before the experiment. As a result, any difference in performance between the players is due to the player policy itself and not differences in the grids they were given.

For all batches, the minimum word length was set to 3 letters. This is consistent with the standard game rules and was kept fixed across all configurations. Results for each game were saved to a CSV file, one per policy: `random_100.csv`, `greedy1_100.csv`, `greedy2_100.csv`, `greedy3_100.csv`, and `hybrid_100.csv`.

Table 4 summarises the configuration used for each batch.

Table 4: Batch Configuration Summary

| **Player** | **Lookahead Depth** | **Games Run** | **Seed** |
|---|---|---|---|
| Random | None | 100 | 123 |
| Greedy-1 | 1-ply | 100 | 123 |
| Greedy-2 | 2-ply | 100 | 123 |
| Greedy-3 | 3-ply | 100 | 123 |
| Hybrid | Rollout | 100 | 123 |
| **Total** | — | **500** | — |

## 5.5 Benchmark Grids for Correctness

Before running large batch experiments, a set of benchmark grids was used to validate implementation correctness. These benchmarks act as controlled tests to check that

- only valid dictionary words are selected,
- the scoring function is applied consistently, and
- the grid update mechanics (letter removal, gravity, and column shifting) produce the correct next state.

This validation step is essential because experimental results are only meaningful if the underlying simulator and players behave correctly.

The benchmark comparison includes edge-cases grids (no playable words; exactly one playable word) as well as fixed test grids. For each benchmark, Greedy-$n$ players where $n \in 1, 2, 3$ were run under identical rules and dictionary settings, and the resulting word sequence, total score, and clearance outcome were recorded. Where a reference outcome was available, results were compared against two sources of ground truth. First, expected outcomes were derived manually from the known grid structure, such as verifying that a grid with exactly one valid word produces exactly one move. Second, and more importantly, outputs were verified against an independent implementation provided by the project supervisor, which was written separately and had not been seen during development. Agreement with this unseen reference code provides stronger evidence of correctness than self-verification alone, as it rules out the possibility of systematic errors shared between the implementation and its own test cases.

Table 5: Benchmark grid results: best word(s) found and total score for each greedy variant. ⋆ denotes a fully cleared grid.

| Grid | Greedy-1 | Greedy-2 | Greedy-3 |
|---|---|---|---|
| AP<br>AQUIZ | QUIZ **100** | QUAI 64 + ZAP 42 = **606*** | QUAI 64 + ZAP 42 = **606*** |
| LYNX<br>PUMA<br>ONYX<br>IBEX | PLUMY **85** | YUPON 60 + LIBYAN 108 = **168** | PLUMY 85 + ANNEX 70 + BOXY 64 = **219** |
| TEST<br>RULE<br>EXAM<br>QUIZ | EQUALS **120** | EXULT 85 + QUIZMASTER 330 = **415** | REST 16 + QUALM 105 + ZEUXITE 182 = **803*** |
| JAM<br>TEA<br>MILK | KELIM **85** | KELIM 85 + JAM 36 = **121** | KET 21 + JAIL 68 + MAK 21 = **610*** |
| RYE<br>CORN<br>MAIZE<br>WHEAT | COZENER **133** | COZENER 133 + WHEATY 90 = **223** | COZENER 133 + WHEATY 90 + MAIR 32 = **755*** |
| JOG<br>SWIM<br>CYCLE | MELICS **96** | CYCLE 80 + JOWS 56 = **136** | JIG 39 + MOW 24 + CYCLES 102 = **665*** |
| MANX<br>BERMUDA<br>GIBRALTAR | RADULAE **91** | DAMMAR 66 + BLUNTER 105 = **171** | GIBBER 78 + DATUM 45 + LARNAX 108 = **231** |

⋆ Grid fully cleared by this algorithm.

During validation, two implementation issues were discovered that affected the interpretation of the greedy lookahead results. Both issues were corrected before the main experiments were run. The results shown in Table 5 reflect the corrected implementation.

First, the initial implementation of the greedy-2 player was accidentally performing extra lookahead work and behaving partially like the 3-ply version. Concretely, the code was evaluating beyond the intended two-move horizon

during planning, even though the third move was not actually executed. This produced misleading behaviour: Greedy-2 appeared smarter than it should be while also incurring unnecessary computation. After correcting this, Greedy-2 now evaluates exactly two consecutive moves and commits only to that planned pair, matching the intended definition of a 2-ply lookahead strategy.

Second, the greedy-3 player initially optimised on the cumulative score of the best three words, which is not necessarily aligned with the true objective of Word Soup when a clear-board bonus is present. In Word Soup, a sequence of moves that clears the grid can receive a large bonus (e.g., +500), meaning the globally optimal plan may involve choosing slightly lower-scoring words early if they enable full clearance. This creates counterexamples where the single highest-scoring first move (e.g., QUIZ word) leads to strong immediate points but blocks clearance, while an alternative sequence with a lower initial payoff clears the board and wins overall once the bonus is counted.

To resolve this mismatch without restructuring the full search logic, the evaluation was updated with a bonus-detector applied at the end of each greedy simulation: after the planned sequence of moves for a given Greedy-$n$ player is completed (e.g., one move for Greedy-1, two for Greedy-2, and three for Greedy-3), the remaining tiles are checked. If the grid is empty, the clearance bonus is added to the evaluated score for that line; otherwise, no bonus is added. This preserves the committed block of behaviour of each Greedy-$n$ strategy while making the evaluation consistent with the game's scoring rules. The benchmark grids in Table 5 confirm that after these fixes, Greedy-$n$ players (where $n \in 1, 2, 3$) behave consistently under identical dictionary and rule settings, and match expected word sequences and outcomes on controlled test cases.

The corrected behaviour is reflected in the benchmark traces shown in Figures 20 and 21, where word sequences and resulting board states match the expected outcomes recorded in Table 5.

To illustrate the solver behaviour concretely, Figures 20 and 21 trace the step-by-step execution of Greedy-2 on Grid 1 (`AP, AQUIZ`) and Greedy 3 on Grid 2 (`LYNX, PUMA, ONYX, IBEX`) respectively. These two grids were selected because they represent opposite outcomes: Grid 1 is fully cleared within two moves, while Grid 2 terminates with residual letters, demonstrating that clearance is not guaranteed even when multiple words are found.

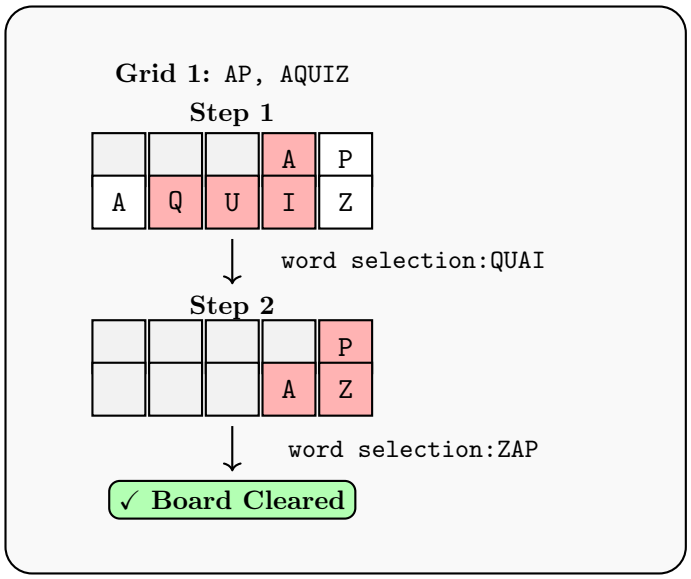



Figure 20: Step-by-step simulation for Grid `AP, AQUIZ` under Greedy-2. Highlighted cells indicate the selected word each round. Letters collapse downward and rightward after each removal. Refer to Table 5 for the corresponding scores.

Grid 1 illustrates a case where the Greedy-1 choice is suboptimal. The highest-scoring single word at the start is `QUIZ` (score 100), which Greedy-1 selects. However, removing `QUIZ` leaves only the letters `A`, `A` and `P`, which do not form a valid word, so the game ends without clearing the board. Greedy-2 instead evaluates pairs of moves and finds that `QUAI` followed by `ZAP` yields a combined score of $64 + 42 = 606$ and fully clears the board, earning the 500-point clearance bonus. Another two-move solution exists

(`AQUA` then `ZIP`), but it scores lower because `ZIP` is a three-letter word while `QUAI` is four letters; since the Word Soup scoring formula multiplies letter values by word length, using `I` in the shorter word `ZIP` yields a lower score for that move. This example demonstrates concretely why lookahead depth matters: a locally optimal first move can prevent board clearance entirely.

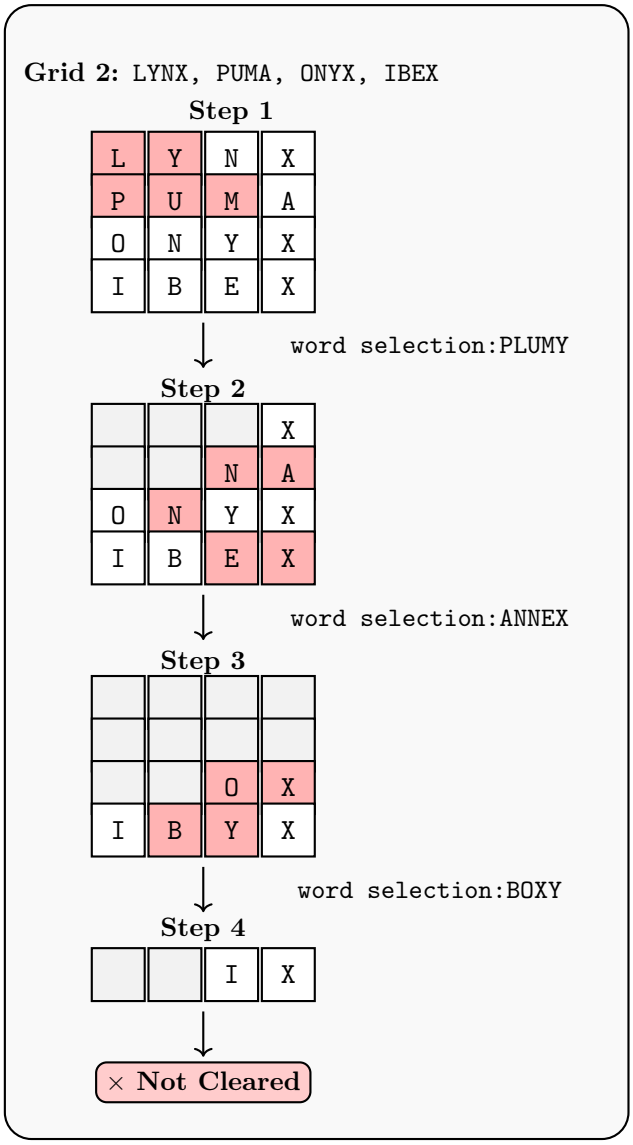



Figure 21: Step-by-step simulation for Grid `LYNX, PUMA, ONYX, IBEX` under Greedy-3. Highlighted cells indicate the selected word each round. Letters collapse downward and rightward after each removal. Refer to Table 5 for the corresponding scores.

Grid 2 illustrates a more complex case where clearance fails despite multiple words being found. The grid contains four horizontal words (`LYNX`, `PUMA`,

ONYX, IBEX), and a (hypothetically speaking) 4-ply strategy that plays all four in sequence would fully clear the board. However, the Greedy-$n$ players do not simply select the first $n$ of these four-letter words, because longer words score higher and are therefore preferred when available.

Greedy-1 selects PLUMY as its single best move, since the five-letter word scores 85 points, more than any individual four-letter word. After PLUMY is removed, the remaining letters no longer support a clean sequence of non-overlapping words. Greedy-3 also begins with PLUMY, as it remains the highest scoring first move even under 3-ply lookahead, and then plays ANNEX and BOXY. This leaves only I and X on the board, which do not form a valid word, so the game ends without clearing. Notably, Greedy-2 does not use PLUMY at all: it finds that the pair YUPON and LIBYAN yields a higher combined two-move score of 168, demonstrating that the Greedy-2 optimal pair is not simply the top two moves from Greedy-1's perspective.

This grid also shows that there is no three-word sequence that clears the board. The four horizontal words would require a 4-ply plan to exploit, which lies beyond the lookahead horizon of all three greedy players. This is precisely the type of case where deeper search or rollout-based planning, as used by the Hybrid policy, has the potential to find a clearing path that greedy strategies miss.

# 6 Results

## 6.1 Baseline Performance Comparison

This section presents a direct comparison of all five policies — Random, Greedy-1, Greedy-2, Greedy-3, and Hybrid, across the 100 test grids generated with batch seed 123. The comparison is organised into three subsections: clear rate (Section 6.1.1), average score (Section 6.1.2), and runtime cost of lookahead (Section 6.1.3). Together, these three metrics capture the core trade-off between computational effort and playing strength.

### 6.1.1 Clear Rate by Policy

The clear rate is the proportion of the 100 test grids on which a policy successfully removed all tiles from the board, earning the 500-point bonus. It is the primary performance metric in this project because clearing the board is the explicit objective of the game. In all but pathological cases, an optimal policy would prioritise clearing the grid: the 500-point bonus exceeds the score of all but a tiny fraction of words in the dictionary, so a strategy that clears reliably will outperform one that does not.

Table 6 reports the clear rate for each policy, and Figure 22 illustrates the results. The Hybrid policy achieved the highest clear rate at 87%, followed by Greedy-3 at 10%, Greedy-2 at 8%, Greedy-1 at 6%, and the Random baseline at 1%. While the greedy policies improve incrementally with lookahead depth, the gap between the best greedy policy (Greedy-3 at 10%) and the Hybrid policy (87%) is striking: Hybrid clears more than eight times as many grids as Greedy-3. This confirms that rollout simulation, which estimates a move's long-term value by simulating a sequence of future moves from that position, provides a qualitatively different level of board-clearing ability that fixed-depth lookahead alone cannot achieve.

Table 6: Clear rate for each policy across 100 test grids (batch seed 123).

| Policy | Grids Cleared | Clear Rate |
|---|---|---|
| Random | 1 | 1% |
| Greedy-1 | 6 | 6% |
| Greedy-2 | 8 | 8% |
| Greedy-3 | 10 | 10% |
| Hybrid | 87 | 87% |

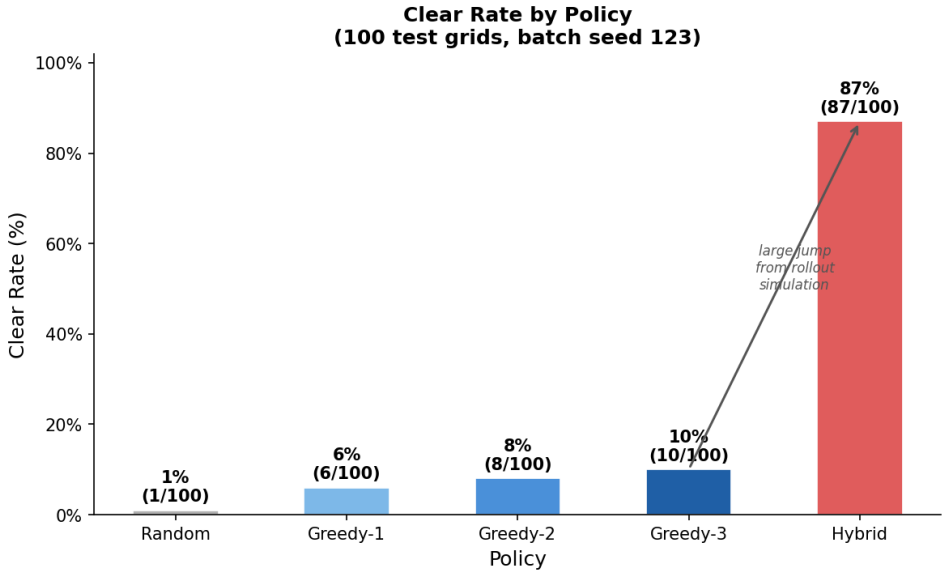



Figure 22: Clear rate by policy across 100 test grids. The Hybrid policy achieves the highest clear rate (87%), and each increase in lookahead depth produces a consistent improvement.

The Random policy cleared only 1 of the 100 grids, confirming that undirected play is almost entirely ineffective at board clearing. The three Greedy policies achieved similar and modest clear rates of 6%, 8%, and 10% for Greedy-1, Greedy-2, and Greedy-3 respectively, suggesting that increasing fixed-depth lookahead alone provides only marginal benefit. The Hybrid policy cleared 87 grids, representing a substantial jump over all Greedy variants. This gap highlights the difference in mechanism: whereas Greedy-3

plans three moves ahead and then resets, the Hybrid policy first shortlists the most promising candidate moves using beam search, then simulates the entire remainder of the game for each candidate before committing to a move. This gives it a fundamentally better view of which choices lead to full board clearance, rather than only what appears locally optimal. The structural reasons why 13 grids remained uncleared are examined in Section 6.2.1.

### 6.1.2 Average Score by Policy

Figure 23 presents the mean raw score (without the 500-point clear bonus) achieved by each policy across the 100 test grids, alongside a comparison of mean scores with and without the bonus applied. Figure 24 shows the corresponding score distributions, and Figure 25 illustrates the marginal gain in mean score at each step up the policy hierarchy.

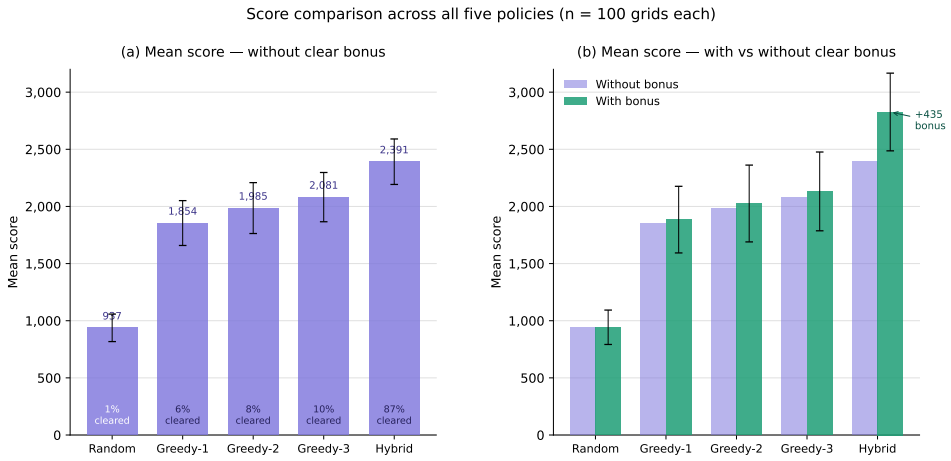



Figure 23: Score comparison across all five policies ($n = 100$ grids each). Figure (a) shows mean raw score without the clear bonus; figure (b) contrasts mean scores with and without the clear bonus applied. The +435 annotation on the Hybrid bar reflects the mean bonus contribution across all 100 grids, which is lower than the fixed 500 point bonus because 13 grids were not cleared.

The results show a clear and consistent ordering of policies by mean raw score. The Random policy achieves a mean of 937.7 points, which serves as

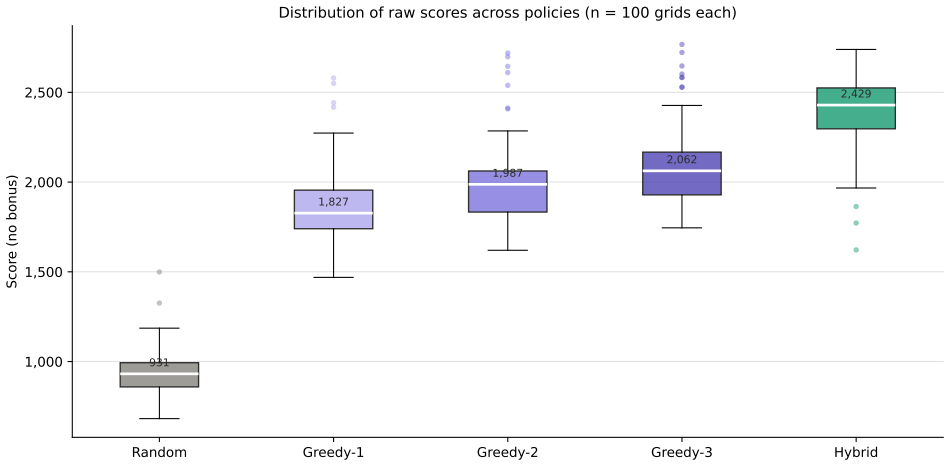



Figure 24: Distribution of raw scores (no bonus) across all five policies ($n = 100$ grids each). Median values are labelled within each box.

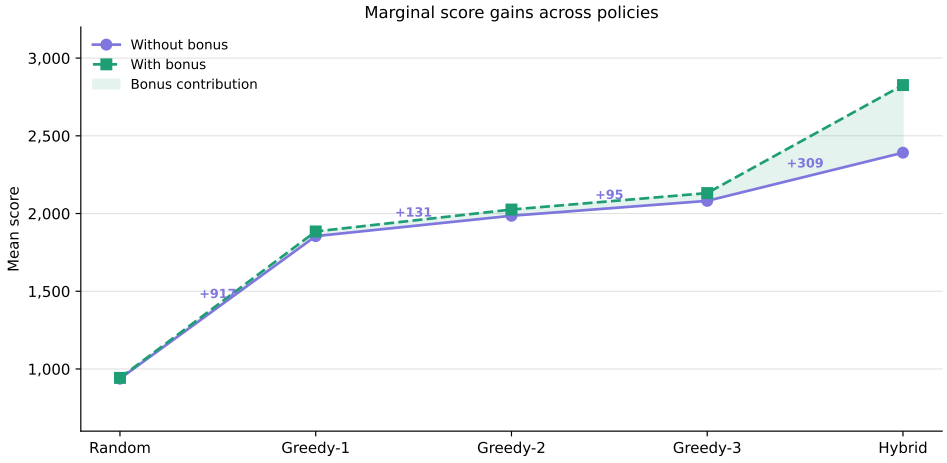



Figure 25: Marginal gains in mean score between consecutive policies. The shaded region indicates the contribution of the clear bonus to the mean total score.

the baseline. Greedy-1 raises this to 1854.7 points (an increase of approximately 917 points) demonstrating that even a single-ply greedy strategy, which selects the highest-scoring word available at each move, dramatically outperforms undirected play. This large initial gain reflects the substantial benefit of preferring longer and higher-value words over random selection.

Beyond Greedy-1, the marginal gains from additional lookahead depth diminish progressively. Greedy-2 achieves a mean of 1985.7 points, a gain of 131 points over Greedy-1, while Greedy-3 reaches 2081.5 points, a further gain of only 96 from Greedy-2. This pattern of diminishing returns is consistent with the general behaviour of lookahead search in sequential decision problems: deeper search provides increasingly marginal improvements in immediate score as the greedy objective is already well-optimised at shallower depths. It should be noted, however, that this observation is limited to ply depths of one, two, and three. At greater depths, the search space expands substantially, and a deeper greedy policy might construct longer words that yield a dramatic score increase. Whether the diminishing returns pattern would persist for Greedy-4 or beyond remains an open question not explored in this project.

The Hybrid policy achieves the highest mean total score of 2826.4 points when the clear bonus is included, which contributes an average of 435 points across the 100 grids. Its mean raw score, from word selections alone and excluding the bonus, is 2391.4 points, which is higher than Greedy-3's mean raw score of 2081.5 points. This advantage arises because clearing the board requires all 108 tiles to be played, which naturally accumulates more total word score than games that terminate early with tiles remaining. All policies in this study, including the greedy family, are designed to prioritise board-clearing: the lookahead evaluation used by Greedy-1, Greedy-2, and Greedy-3 ranks sequences that clear the board above those that do not, before considering raw score. However, the Hybrid policy pursues this objective more aggressively through full rollout simulation, rather than looking ahead a fixed number of moves, it simulates the remainder of the game from each

candidate move and selects the line most likely to result in a complete clear. This stronger clearing mechanism is what drives the Hybrid policy's higher raw scores on average, since clearing the board requires all tiles to be used, which naturally yields more total scored words.

The effect of the 500 point clear bonus is visible but moderate for the greedy policies, since their clear rates are low $(6 - 10\%)$. For Greedy-1, the mean score rises from 1854.7 to 1884.7 with the bonus, a mean contribution of just 30 points. For Greedy-3 it rises from 2081.5 to 2131.5, a contribution of 50 points. By contrast, the Hybrid policy's 87% clear rate makes the bonus highly impactful: its mean total score including the bonus is 2826.4 points, a bonus contribution of approximately 435 points, as annotated in Figure 23(b). This confirms that for the Hybrid policy, clearing the board is not merely a secondary objective; it is the primary driver of its score advantage over the greedy family. Indeed, even without the bonus, the Hybrid policy's mean raw score of 2391.4 points exceeds that of every other policy, and its mean total score with the bonus (2826.4 points) exceeds every other policy by a substantial margin. This confirms that Hybrid's score advantage is not solely an artefact of the bonus: it achieves the highest mean total score of 2826.4 points across all 100 grids, a margin that no other policy approaches.

Score variability is broadly similar across all policies except Random, which has the narrowest spread (interquartile range, $IQR \approx 135$ points). The greedy policies have IQRs in the range 215–238 points, and the Hybrid policy has an IQR of 228 points. The boxplot in Figure 24 reveals that the Hybrid distribution is shifted upward and compact relative to the greedy policies, with few low outliers. This tighter concentration at high scores suggests that the Hybrid policy performs consistently well across grid instances, rather than relying on occasional high-scoring grids. Greedy-2 and Greedy-3, by contrast, exhibit several high outliers above 2500 points, indicating that on particularly favourable grids, short lookahead can still achieve high scores, but with no guarantee of consistent performance.

A notable asymmetry in the nature of outliers is visible in Figure 24, which plots raw scores without the clear bonus. For the greedy policies and Random, high scoring outliers correspond to the rare grids that were cleared, since completing the board requires all tiles to be played and naturally accumulates more score. For the Hybrid policy, the direction is reversed: low scoring outliers correspond to the 13 grids it failed to clear, where play terminated early. The fact that this asymmetry is visible even in the raw scores, before any bonus is applied, confirms that clearing the board yields a structurally higher score regardless of the bonus.

Increasing lookahead depth produces consistent but diminishing improvements in raw score, while the Hybrid policy achieves a qualitatively different result by combining rollout-based clearing with score optimisation. The bonus analysis confirms that the Hybrid policy's overall score lead is substantially amplified by its high clear rate, a feature that none of the greedy policies considered here is able to replicate. Summary statistics are reported in Table 7, and the marginal gains at each step of the policy hierarchy are visualised in Figure 25.

Although the Hybrid policy leads on average, it is not the highest raw scorer on every individual grid. Across the 100 test grids, there are 20 grids on which a greedy policy achieves a higher raw score than Hybrid, with Greedy-3 and Greedy-2 accounting for all such cases. This occurs because the Hybrid policy prioritises board clearance above immediate score maximisation: when simulating future moves, it selects the line most likely to result in a full clear rather than the line that maximises points along the way. On grids where a greedy policy happens to find a high scoring path that also clears the board, it can outscore Hybrid on raw points. Notably, of the 20 grids where Hybrid is not the top raw scorer, 14 are grids that Hybrid successfully cleared, meaning its lower raw score on those grids reflects a deliberate trade-off in move selection rather than a failure to play well. The remaining 6 are among the 13 grids Hybrid failed to clear entirely. This confirms that Hybrid's average

score lead is robust and structural, even if not universal at the individual grid level.

Table 7: Summary of score statistics by policy ($n = 100$ grids each).

| Policy | Mean (no bonus) | Mean (with bonus) | IQR | Marginal gain |
|---|---|---|---|---|
| Random | 937.7 | — | 134.8 | — |
| Greedy-1 | 1854.7 | 1884.7 | 215.0 | +917.0 |
| Greedy-2 | 1985.7 | 2025.7 | 228.8 | +131.0 |
| Greedy-3 | 2081.5 | 2131.5 | 238.0 | +95.8 |
| Hybrid | 2391.4 | 2826.4 | 227.8 | +694.9 |

**Does Hybrid sacrifice score to achieve clearing?** A natural question is whether the Hybrid policy's focus on board clearing comes at the cost of raw score performance. On the ten grids that both Greedy-3 and Hybrid successfully cleared, Greedy-3 achieves a mean raw score of 2573.2 points compared to Hybrid's 2441.7 points. This difference arises because on grids where clearing is easy enough for Greedy-3 to manage, Greedy-3 can afford to prioritise higher-scoring words within its fixed three-move lookahead. Hybrid, by contrast, commits to the clearing path that its rollout simulation identifies as most likely to succeed, even when a marginally higher-scoring alternative exists. However, this trade-off is overwhelmingly favourable. On the 77 grids that Hybrid cleared but Greedy-3 did not, Hybrid achieves a mean raw score of 2445.8 points to Greedy-3's 2023.8 points, simply because clearing the board requires all tiles to be scored. The 500-point bonus then adds a further reward. In total, including the bonus, Hybrid's mean score of 2826.4 points far exceeds Greedy-3's 2131.5 points. The small raw-score deficit on shared-clearing grids is therefore an acceptable cost for the substantially broader clearing ability Hybrid provides.

### 6.1.3 Runtime Cost of Lookahead

Table 8 reports the mean, standard deviation, and maximum runtime per game for all five policies across the 100 test grids. Figure 26 shows the full

runtime distribution as box plots on a log scale, and Figure 27 plots mean runtime against clear rate for each policy.

Table 8: Runtime per game for all five policies (100 grids, batch seed 123).

| Policy | Mean (s) | SD (s) | Max (s) | Clear Rate |
|---|---|---|---|---|
| Random | 0.24 | 0.06 | 0.43 | 1% |
| Greedy-1 | 0.36 | 0.14 | 0.85 | 6% |
| Greedy-2 | 8.34 | 4.56 | 22.01 | 8% |
| Greedy-3 | 101.29 | 56.60 | 351.87 | 10% |
| Hybrid | 21.20 | 5.23 | 36.15 | 87% |

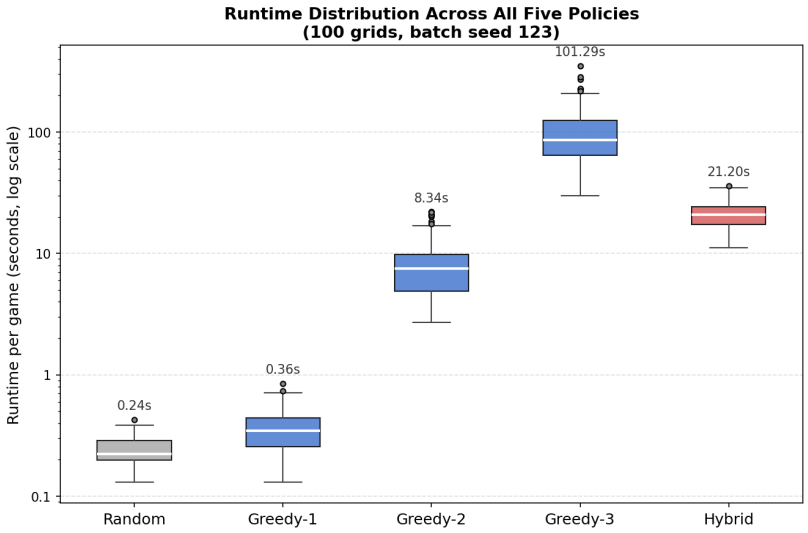



Figure 26: Runtime distribution across all five policies on a log scale (100 grids, batch seed 123). Mean runtime per game is annotated above each box.

The Random and Greedy-1 policies are the fastest, completing each game in a mean of 0.24 and 0.36 seconds respectively. The small difference between them reflects the fact that both policies evaluate only the immediately available moves without any forward planning. Extending lookahead to two moves (Greedy-2) increases mean runtime by a factor of approximately 23, to 8.34 seconds, and extending further to three moves (Greedy-3) raises it to

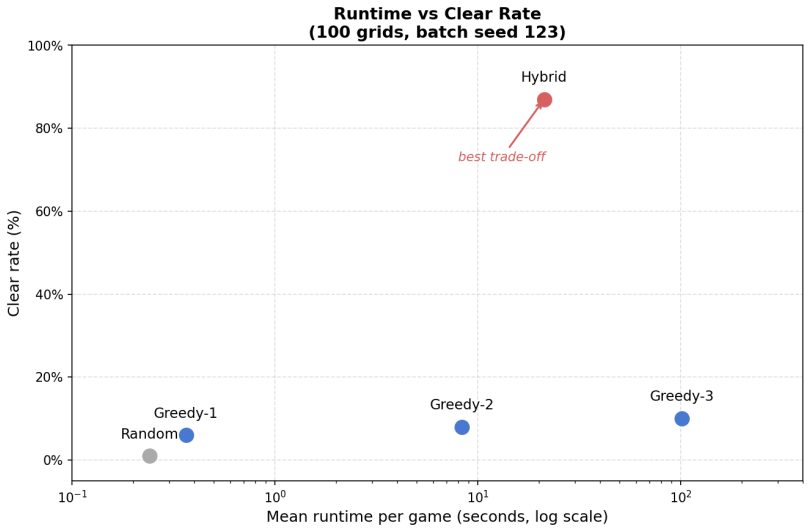



Figure 27: Mean runtime per game plotted against clear rate for each policy (log scale on horizontal axis). The Hybrid policy achieves the best trade-off, reaching an 87% clear rate at a mean runtime of 21.2 seconds per game.

101.3 seconds, a factor of 278 over Greedy-1. The large variance in Greedy-3 runtime (standard deviation 56.6 seconds, maximum 351.9 seconds) reflects sensitivity to the branching factor: grids with more playable words at each turn require the exhaustive search to explore substantially more candidate sequences. At the start of a game, the number of valid words on a full 12×9 grid averages 2,564 across the test set (ranging from 1,501 to 3,964), and a three-ply search must evaluate sequences drawn from this pool at each step, causing the search space to grow rapidly with the initial branching factor.

Despite this cost, the improvement in clear rate from Greedy-1 to Greedy-3 amounts to only 4 percentage points (from 6% to 10%), confirming that fixed-depth lookahead beyond one ply is an inefficient strategy for this game.

The Hybrid policy offers a qualitatively different trade-off. At a mean runtime of 21.2 seconds per game, comparable to Greedy-2 and approximately five times faster than Greedy-3, it achieves an 87% clear rate. This is visible

in Figure 27: Hybrid occupies the upper region of the runtime (clear rate space), well separated from all three Greedy variants. The lower variance in Hybrid runtime (standard deviation 5.2 seconds) compared to Greedy-3 also reflects a more computationally predictable strategy: rather than exhaustively searching a fixed-depth tree, the Hybrid policy uses full rollout simulation guided by a shortlist of top-$k$ candidates (where $k = 20$), which bounds the per-move computation more consistently across grids. This design sacrifices some immediate score optimality in favour of a strategy focused on long-term board clearance. The experimental results show that this trade-off is favourable: the Hybrid policy not only clears the board at a far higher rate than any greedy variant, but also achieves the highest mean raw score across all 100 grids, confirming that the sacrifice in local optimality does not reduce overall performance.

## 6.2 Structural Properties of the Grid

The preceding section showed that clear rate varies considerably across policies, suggesting that certain grid configurations are inherently more resistant to clearing regardless of the strategy applied. This section investigates which structural features of a grid predict whether the Hybrid policy — the strongest performer — succeeds in clearing it. Three properties are examined in turn: vowel composition, word availability and branching factor, and their joint effect on late-game tractability.

### 6.2.1 Predictors of Clearing

Although the Hybrid policy achieved the highest clear rate of 87%, 13 grids remained uncleared. This section examines the structural properties of those 13 grids to identify what makes a grid resistant to clearing.

#### Vowel Ratio as the Primary Predictor

The most consistent distinguishing feature between cleared and uncleared grids is the vowel ratio at the start of the game. Failed grids had a mean

vowel ratio of 0.328, compared to 0.379 for cleared grids. As shown in Figure 28, 8 of the 13 failed grids had a vowel ratio below 0.33, whereas only 9 of the 87 cleared grids fell below that threshold. A consonant-heavy board reduces the number of valid word paths available, particularly in the late game when fewer tiles remain and adjacency constraints become more restrictive. This problem is further compounded by the fact that valid words tend to incorporate vowels disproportionately, so each move played gradually depletes the vowel supply. As the game progresses, the remaining tiles become increasingly heavy with consonants regardless of the starting ratio, and the board reaches an unplayable state before it can be fully cleared.

**Rare Letter Ratio** The proportion of rare letters (Q, X, Z, J) on the grid was also examined as a potential predictor of clearing outcome. Failed grids had a slightly higher mean rare letter ratio of 0.015, compared to 0.013 for cleared grids. However, the difference is small and the distributions overlap considerably: rare letters appeared in 11 of the 13 failed grids and in 66 of the 87 cleared grids. This suggests that rare letter ratio alone is not a reliable predictor of clearing failure. A single Q or Z tile does not necessarily prevent clearing, because such tiles can still form valid words along adjacency paths if the surrounding letters are cooperative. The rarity effect is therefore better understood as a secondary contributor that compounds the consonant-heavy problem rather than an independent cause of failure.

### Near-Miss Behaviour

The failures are, in most cases, near-misses rather than complete breakdowns. As shown in Figure 29, 7 of the 13 failed grids left only 1 or 2 cells remaining at the end of play. In total, 9 of the 13 failed grids left 5 or fewer cells. This indicates that the Hybrid policy was playing effectively throughout the game and only encountered an unsolvable state in the final moves. The unsolvability arises because the last remaining tiles form an isolated cluster with no valid adjacency paths, a late-game trap that the rollout simulation cannot foresee because it lies beyond the planning horizon.

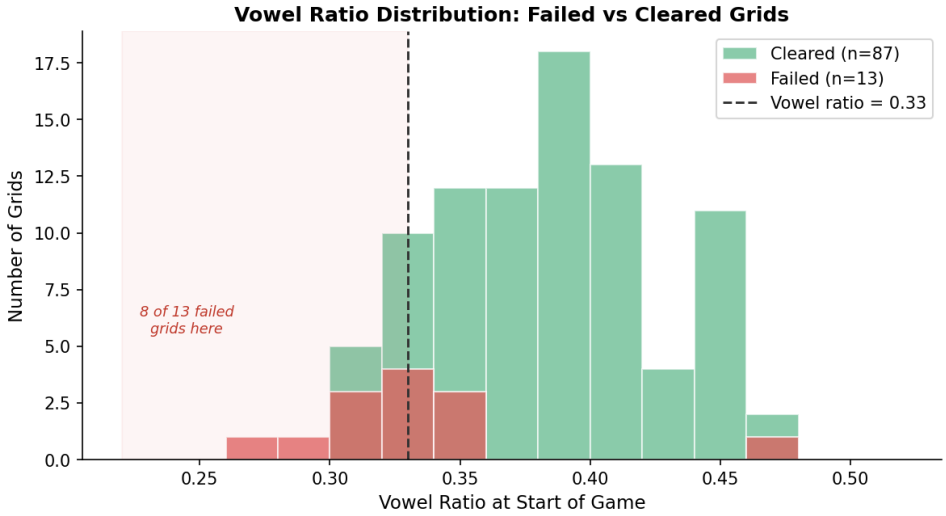



Figure 28: Distribution of vowel ratio across cleared and failed grids. The dashed line marks a vowel ratio of 0.33; 8 of the 13 failed grids fall below this threshold.

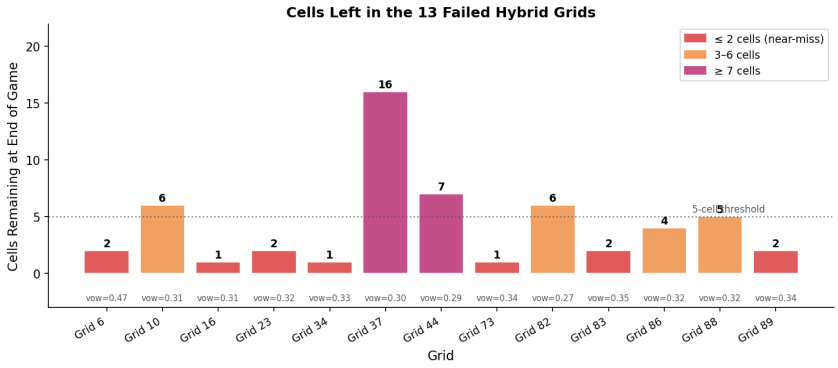



Figure 29: Number of cells remaining at the end of play for each of the 13 failed Hybrid grids. The vowel ratio of each grid is annotated below each bar. The dashed line at 5 cells serves as a visual boundary separating near miss failures (5 or fewer cells left) from more substantial failures. Seven grids left at most 2 cells, indicating near-miss failures rather than systematic breakdowns.

### Branching Factor and Word Availability

Beyond the vowel ratio, failed grids also exhibited lower word availability and lower average branching factors. The mean number of playable words at the start of the game was 2,363 for failed grids compared to 2,594 for cleared grids, and the mean branching factor across the game was 733 versus 847 respectively (Figure 30). Importantly, a low branching factor is not simply a consequence of a low vowel ratio: the branching factor reflects the full adjacency structure of the grid, which depends on how all letters are spatially arranged, not just the vowel composition alone. These differences confirm that failed grids were structurally harder: the player faced fewer legal moves at each turn, which compounds the effect of a low vowel ratio by reducing the chance of finding a word that avoids a late-game dead-end.

Notably, the average points per move was higher for failed grids (155.8) than for cleared grids (139.0). **This rules out poor word selection as a cause of failure**. The Hybrid policy was still identifying and playing high-scoring words; it simply could not avoid the structural trap imposed by the grid.

### Scatter Analysis and Outliers

Figure 31 plots vowel ratio against branching factor and against average points per move for all 100 grids. Failed grids (marked with crosses) cluster in the lower-left region of the left figure, confirming that low vowel ratio and low branching factor together are the strongest joint predictors of failure. The right figure shows no clear separation by points per move, which is consistent with the finding above that the policy's word selection quality is not the cause of failure.

### Individual Grid Profiles

Figure 32 presents a normalised feature profile for all 13 failed grids. The majority share a common pattern: low vowel ratio alongside otherwise moderate or high values for branching factor and points per move, consistent

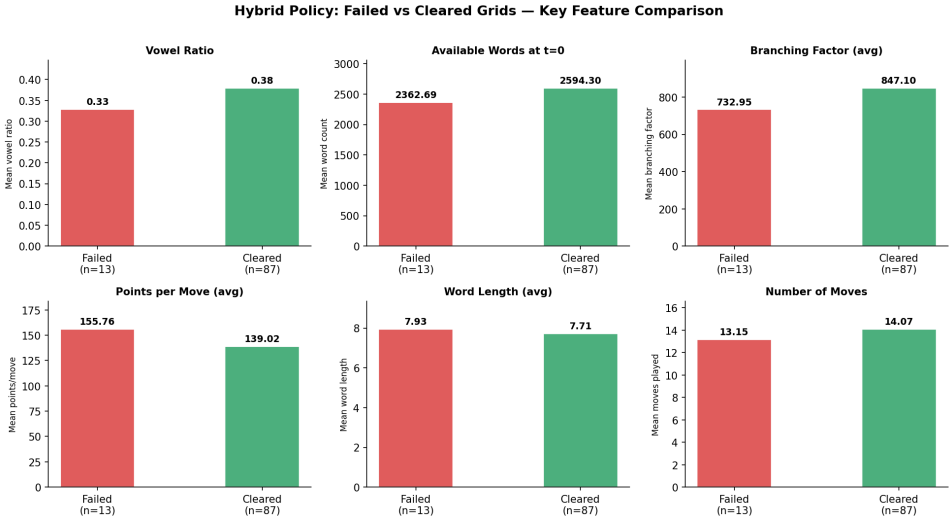



Figure 30: Mean values of six key features for failed and cleared Hybrid grids. Failed grids have lower vowel ratios, fewer starting words, and lower branching factors, but higher average points per move. This is likely because consonant-heavy grids tend to contain more high-value letters, which raises the per-move score independently of whether the grid can be cleared.

with the near-miss interpretation above. One clear outlier is Grid `lfreq_82`, which has the lowest vowel ratio (0.27), lowest branching factor (543), and lowest points per move (101) in the failed set. This grid represents a genuinely difficult instance where the policy struggled throughout play rather than only at the final step. The remaining 12 failures are better characterised as late-game traps: the policy played effectively for most of the game but arrived at an unresolvable board state in the final moves, as evidenced by the low cells-remaining counts shown in Figure 29.

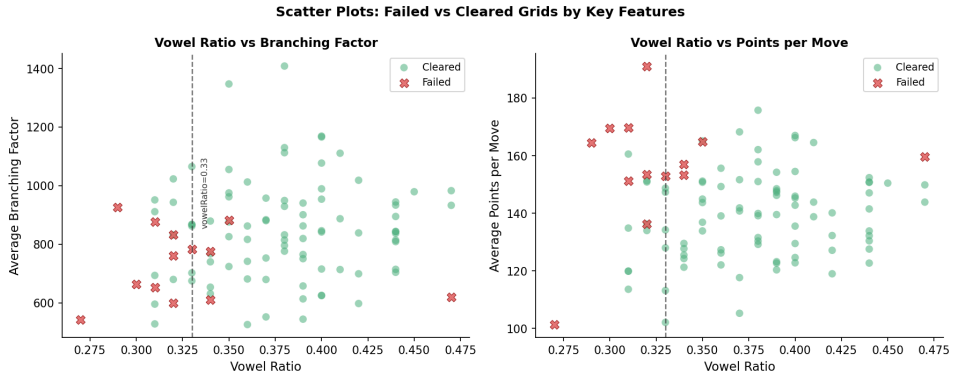



Figure 31: Scatter plots of vowel ratio against branching factor (left) and average points per move (right). Failed grids are shown as red crosses. The dashed vertical line marks a vowel ratio of 0.33. Failed grids concentrate in the low-vowel, low-branching region but are not separated by points per move.

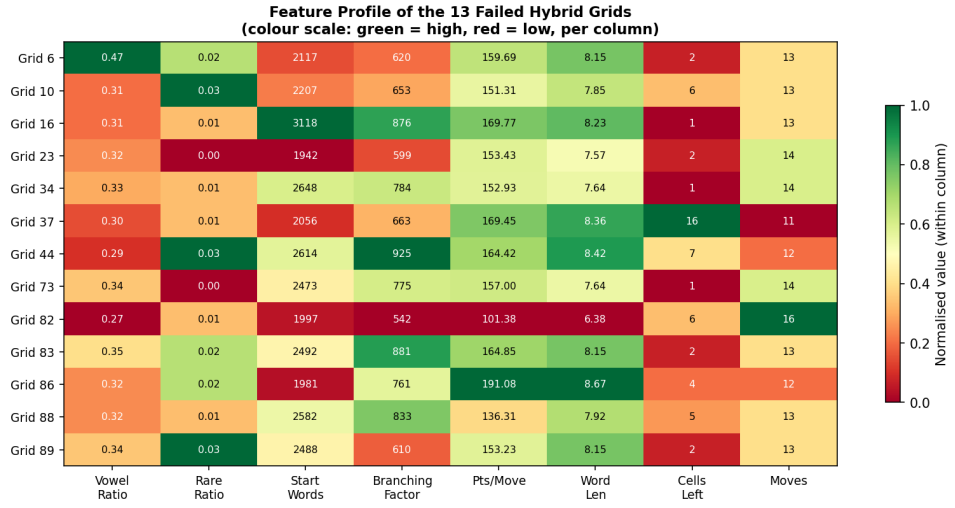



Figure 32: Normalised feature profile of the 13 failed Hybrid grids. Each cell is colour-coded relative to the column range (green = high, red = low). Grid `lfreq_82` stands out as a structural outlier with consistently low values across all features.

Table 9 summarises the key patterns across the 13 failed grids.

Table 9: Summary of failure patterns across the 13 uncleared Hybrid grids.

| Pattern | Count | Proportion |
|---|---|---|
| Vowel ratio $< 0.33$ | 8 | 62% |
| Cells remaining $\leq 2$ (near-miss) | 7 | 54% |
| Cells remaining $\leq 5$ | 9 | 69% |
| Branching factor below overall mean | 10 | 77% |

In summary, the dominant failure mode for the Hybrid policy is a consonant-heavy grid that reaches an unplayable state in the final one or two moves. The rollout simulation cannot prevent this because the dead-end only becomes apparent beyond its planning horizon. Several directions could address this limitation. One approach is vowel conservation: penalising moves that consume vowels disproportionately to preserve grid balance into the late game. Another is to weight the scoring heuristic to prioritise rare or awkward letters such as Q, X, and Z earlier in the game, removing tiles that are hardest to incorporate before the adjacency structure becomes too restricted. A third direction is to extend the rollout horizon specifically when the remaining cell count falls below a threshold, triggering deeper search only at the critical late-game stage where dead-ends arise. These directions are discussed further in the context of future improvements in Chapter 7.

### 6.2.2 Anomalous Failure: Grid `lfreq_6`

Among the 13 grids that the Hybrid policy failed to clear, grid `lfreq_6` stands apart as the most structurally anomalous. Every other failed grid is consistent with the dominant failure mode identified in Section 6.2.1: a low vowel ratio (below 0.33) that limits word-forming potential throughout the game. Grid `lfreq_6` violates this pattern entirely. Its vowel ratio is 0.47, the highest of any failed grid in the batch, exceeding the second-highest failed

grid (vowel ratio 0.35) by a margin of 0.12, and sitting well above the mean vowel ratio of cleared grids (0.38). By the primary predictor identified in this chapter, `lfreq_6` should have been one of the easier grids to clear.

Table 10: Feature profile of `lfreq_6` compared to the means of cleared and failed grids across the 100-grid batch.

| Feature | `lfreq_6` | Failed mean | Cleared mean |
|---|---|---|---|
| Vowel ratio | 0.47 | 0.328 | 0.379 |
| Rare-letter ratio | 0.02 | 0.015 | 0.013 |
| Initial word count | 2,117 | 2,363 | 2,594 |
| Avg. branching factor | 620.5 | 732.9 | 847.1 |
| Avg. points per move | 159.69 | 155.76 | 139.02 |
| Avg. word length | 8.15 | 7.93 | 7.71 |
| Cells remaining | 2 | 4.23 | 0 |

Table 10 presents the full feature profile of `lfreq_6`. Several values are immediately notable. Despite the high vowel ratio, the initial word count of 2,117 is below the mean of failed grids (2,363) and well below the cleared grid mean (2,594). The average branching factor of 620.5 is the lowest of any grid with vowel ratio above 0.44 in the entire batch; among the 13 grids in that high-vowel peer group, the next lowest branching factor is 704.3, and the group mean is 864.2. Grid `lfreq_6` is the only member of this peer group that was not cleared. Furthermore, the policy played well throughout: the average word length of 8.15 letters and average points per move of 159.69 both indicate that the Hybrid policy was finding and playing strong words at every step. The failure cannot be attributed to poor word selection. The game ended with just 2 tiles remaining, making this a near-miss failure rather than a systematic breakdown.

**The vowel clustering hypotheses.** The combination of high vowel ratio and low word availability points to a spatial distribution problem rather than a letter frequency problem. When vowels are distributed evenly across

the grid, each vowel tile contributes to consonant–vowel adjacency relationships across multiple regions of the board, maximising the number of valid words that can be formed at each step. When vowels cluster together, however, they form locally rich patches while leaving other regions of the board dominated by consonants. Those consonant-heavy regions produce few valid words on their own and become increasingly isolated as the game progresses and tiles are removed. The final dead-end which is the two tiles remaining with no valid word between them is the natural endpoint of this process: the policy cleared the vowel-rich regions successfully but could not avoid being left with residual consonant tiles that had no playable neighbours.

This hypothesis is supported by the `words per vowel unit` ratio,

$$\text{words per vowel unit} = \frac{\texttt{startWords}}{\texttt{vowelRatio}}$$

defined as the initial word count divided by the vowel ratio. For `lfreq_6`, this value is 4,504 – the third lowest of all 100 grids, placing it at the 3rd percentile of the batch. The cleared grid mean is 6,927 and the failed grid mean (excluding `lfreq_6`) is 7,542. A low words per vowel ratio is precisely what would be expected if vowels are clustered which also means the vowel count is high, but those vowels are not generating a proportionate number of accessible words because they are not adjacent to enough distinct consonants.

Figure 33 illustrates both dimensions of the anomaly. The left panel plots vowel ratio against average branching factor for all 100 grids. Grid `lfreq_6` (gold star) lies in the right portion of the plot – consistent with cleared grids on the vowel axis – but sits far below its cleared peers on the branching factor axis, closer to the failed cluster. The right panel plots vowel ratio against words per vowel unit. Here `lfreq_6` is the clear outlier: it has the highest vowel ratio among failed grids, yet its word availability per vowel unit is lower than almost every other grid in the batch, whether cleared or failed.

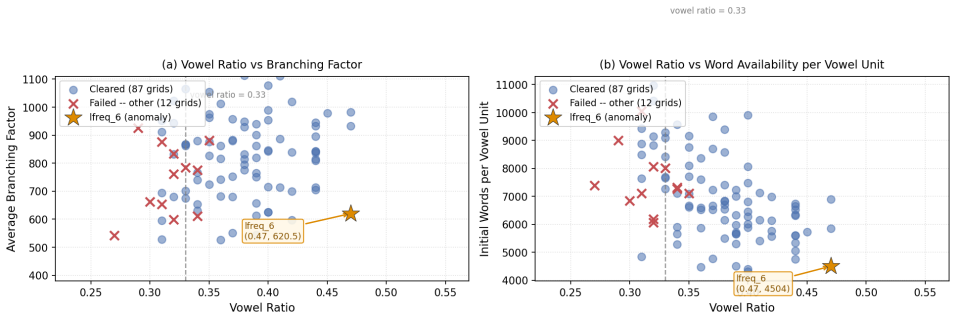



Figure 33: Grid `lfreq_6` anomaly profile across all 100 grids. `Left`: vowel ratio against average branching factor. `Right`: vowel ratio against initial words per vowel unit. In both figures, `lfreq_6` (gold star) sits in the high-vowel region of the horizontal axis but occupies an unusually low position on the vertical axis relative to its cleared peers, indicating that its vowels are not generating the word availability that would normally accompany a ratio of 0.47.

**Implications for the vowel ratio predictor.** This case demonstrates that vowel ratio is a necessary but not sufficient condition for clearing. A high vowel ratio is strongly associated with clearability across the batch where the cleared grid mean is 0.38 and no grid with a ratio below 0.27 was cleared. But `lfreq_6` shows that vowels can be present in sufficient quantity while being spatially arranged in a way that undermines their utility. The words per vowel ratio captures this failure mode more precisely than vowel ratio alone. It measures not just whether vowels are present, but whether those vowels are generating accessible vocabulary. A more complete model of grid difficulty would incorporate a spatial measure of vowel distribution alongside the aggregate vowel ratio. That said, `lfreq_6` is a single case, and deriving a spatial metric would require access to the grid layout at the time of analysis, which is not available from the summary statistics alone.

### 6.2.3 Anomalous Failure: Grid `lfreq_37`

Among the 13 grids that the Hybrid policy failed to clear, one stands apart from the rest. Grid `lfreq_37` left 16 cells remaining at the end of play and far above the group mean of 4.23 cells. Every other failed grid left at most 7 cells, most of which qualify as near-misses. `lfreq_37` represents a qualitatively different failure.

**Is it vowel ratio?** Low vowel ratio is the primary predictor of failure identified in Section 6.2.1, and `lfreq_37` does have a low vowel ratio of 0.300. However, two other failed grids have lower vowel ratios which are `lfreq_82` at 0.270 and `lfreq_44` at 0.290. Yet both left far fewer cells (6 and 7 respectively). Vowel ratio alone does not account for the severity of `lfreq_37`'s failure.

**A compound explanation: few moves and long words.** As shown in Figure 34, `lfreq_37` is the outlier on both the cells-remaining axis and the moves axis simultaneously. The Hybrid policy made only 11 moves on this grid which is the fewest of any failed grid and those moves had an average word length of 8.36 letters, the second longest among failed grids. Together, this means the policy removed 92 of 108 cells in just 11 turns, clearing large connected chunks of tiles on each move. On a consonant-heavy grid where adjacency options are already limited, this pattern of aggressive long-word selection can isolate the remaining tiles into disconnected clusters before the game ends. Once those clusters lose all valid adjacency paths, no further moves are possible regardless of the letters present.

The branching factor of `lfreq_37` (663.1) was not exceptionally low relative to other failed grids (group mean: 733.0), confirming that the grid was structurally accessible at the start. The failure was not caused by persistent difficulty throughout the game. Instead, the specific sequence of word selections collapsed the adjacency graph prematurely. The board was cleared at speed, but the 16 remaining tiles ended in an unplayable configuration.

This contrasts with Grid `lfreq_82`, discussed in Section 6.2.1, which exhibited consistently poor structural properties from the first move. `lfreq_82` failed due to persistent difficulty while `lfreq_37` failed due to an early structural collapse of the grid. These represent two distinct types of failure within the 13 uncleared grids: structurally resistant grids, and grids that become unplayable as a consequence of the policy's own word-selection pattern. The latter type, illustrated by lfreq_37, suggests that penalising long-word sequences on consonant-heavy grids could reduce the risk of premature adjacency collapse, an avenue discussed further in Chapter 7.

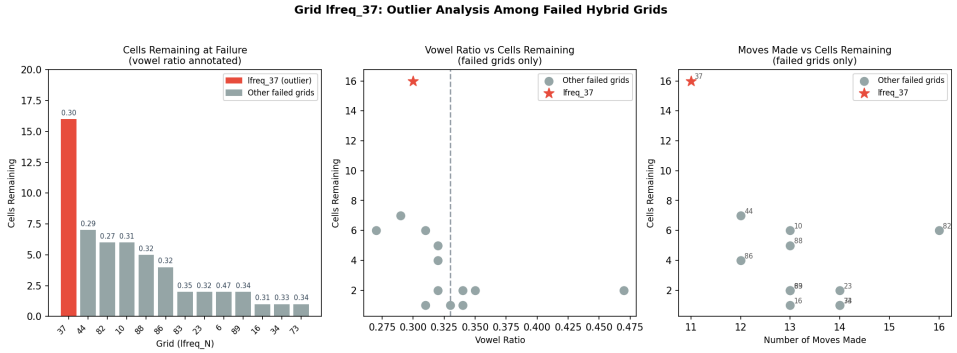



Figure 34: Left: cells remaining at failure for all 13 failed Hybrid grids, with vowel ratio annotated. `lfreq_37` (red) is a clear outlier with 16 cells remaining. Centre: vowel ratio vs cells remaining, showing that the two grids with lower vowel ratios did not fail as severely. Right: moves made vs cells remaining. `lfreq_37` made the fewest moves of any failed grid.

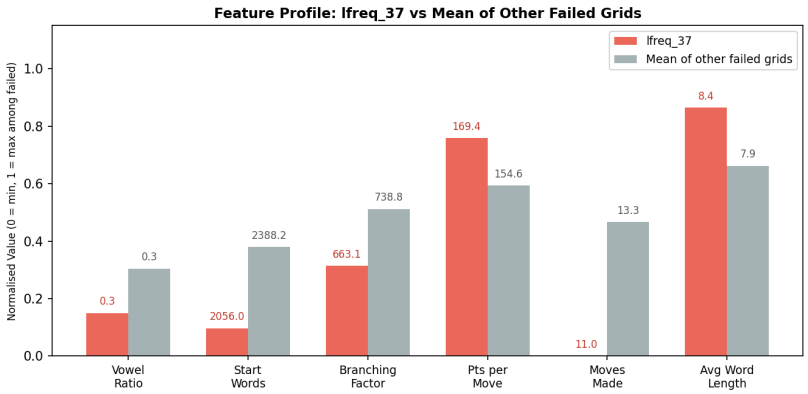



Figure 35: Normalised feature profile of `lfreq_37` (red) against the mean of the other 12 failed grids (grey). Raw values are annotated. `lfreq_37` stands out with the fewest moves and near-longest average word length, while its branching factor is not notably low.

## 6.3 Vowel Ratio as the Dominant Predictor of Clearing

The analysis in Section 6.2.1 identified vowel ratio as the primary structural predictor of failure among the 13 grids that Hybrid could not clear. This section examines whether the same relationship holds across all 100 grids using the full dataset rather than only the failed cases.

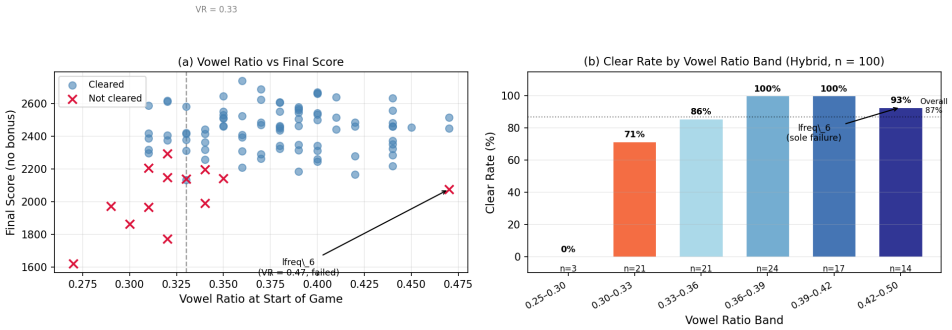



Figure 36: Vowel ratio and clearing outcome across all 100 Hybrid games. (a) Scatter plot of vowel ratio against final score. Cleared grids (blue) span the full range of vowel ratios, while failed grids (red crosses) cluster toward the left. Grid `lfreq_6` is the lone failure in the high-vowel region. (b) Clear rate grouped into six vowel ratio bands. Clear rate rises from 0% in the lowest band to 100% in the 0.36–0.42 range, falling slightly to 93% in the highest band due to `lfreq_6`.

Figure 36 summarises the relationship from two perspectives. The left figure plots vowel ratio against final score for every grid, with cleared and failed grids distinguished by colour. The right figure groups the 100 grids into six vowel ratio bands and reports the clear rate within each band.

The banded view makes the trend clear. Grids with a vowel ratio below 0.30 were never cleared which means all three grids in that range ended with tiles remaining. Grids in the 0.30–0.33 band were cleared 71% of the time, and once the vowel ratio exceeds 0.36, every grid in the batch was cleared except one. The practical implication is consistent with the finding from Section 6.2.1: a vowel ratio below 0.33 constitutes a meaningful difficulty threshold at which clearing becomes uncertain, while a vowel ratio above 0.36 provides a strong guarantee of clearability for the Hybrid policy.

The single exception in the high-vowel region is `lfreq_6`, visible as the rightmost red cross in the left figure with a vowel ratio of 0.47 (the highest in the entire batch). It is the only grid above 0.42 that Hybrid failed to clear. Its failure is not caused by a shortage of vowels, and it was discussed in full in

Section 6.2.2. The key finding there was that its vowels were spatially clustered rather than evenly distributed, suppressing word availability despite the high vowel count. The `words per vowel unit` ratio for `lfreq_6` placed it at the 3rd percentile of the batch, far below the cleared grid mean, which explains why a grid that looks easy on the vowel ratio metric alone still failed.

Taken together, the evidence from both the start and the end of the game points to the same conclusion. At the start, the vowel ratio of a grid predicts how many valid word paths the player will have access to throughout the game. By the end of the game, grids with low vowel ratios tend to create unsolvable endings, as seen in the 13 failed cases. Grid `lfreq_6` shows the limit of using vowel ratio alone. It helps, but it is not enough to guarantee a clear. Spatial distribution of those vowels matters too, a factor that the aggregate ratio does not capture.

Together, these findings suggest that for a policy to reliably clear the board, the grid must provide not only sufficient vowels in aggregate, but also good spatial distribution of those vowels throughout the letter arrangement.

## 6.4 Efficiency and Trade-Off Analysis

The preceding sections have evaluated each policy primarily in terms of clear rate and raw score. However, these two metrics do not account for the computational cost required to achieve them. A policy that produces high scores or clears the board frequently is only practically useful if it does so within a reasonable amount of time. This section examines the relationship between performance and runtime more directly, using score per second as a combined measure of efficiency.

### 6.4.1 Score per Second

Score per second is defined as the total score earned (excluding the clear bonus) divided by the total runtime for that game. It captures how efficiently a policy converts computation time into points. Table 11 summarises

the mean score per second for each policy, and Figure 37 shows the full distribution as a box plot on a log scale.

Table 11: Mean score per second by policy (100 grids, batch seed 123). Score excludes the 500-point clear bonus.

| Policy | Mean Score/s | Median Score/s |
|---|---|---|
| Random | 4,148 | 4,080 |
| Greedy-1 | 5,888 | 5,526 |
| Greedy-2 | 301 | 286 |
| Greedy-3 | 26 | 23 |
| Hybrid | 119 | 116 |

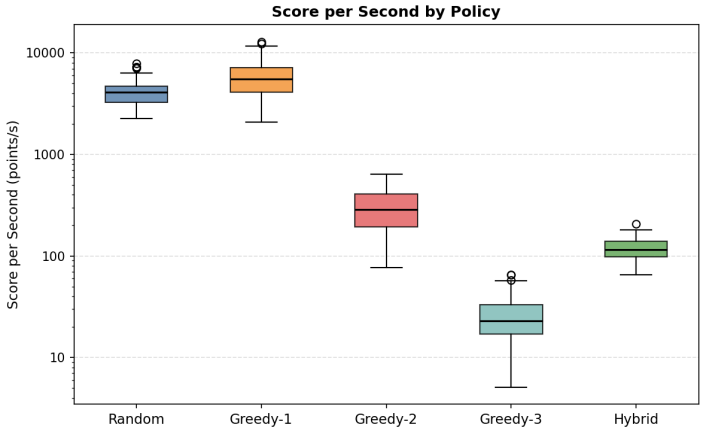



Figure 37: Score per second distribution for all five policies on a log scale (100 grids). Random and Greedy-1 achieve the highest throughput due to their low runtimes but do not account for the clear bonus which substantially benefits Hybrid in practice.

Random and Greedy-1 lead on score per second by a wide margin, achieving means of 4,148 and 5,888 points per second respectively, because both policies complete each game in under one second without performing any lookahead. However, this metric can be misleading as a stand alone measure of efficiency. Neither policy clears the board reliably, so the scores they accumulate are structurally limited. Greedy-2 and Greedy-3 incur substantially

higher runtimes without proportionate gains in clear rate, giving them the lowest score-per-second values of 301 and 26 respectively. The Hybrid policy occupies an intermediate position at 119 points per second. Although this is lower than Random and Greedy-1, the comparison changes materially once the 500-point clear bonus is factored in: at an 87% clear rate, the effective contribution of the bonus adds approximately 435 points per game on average, a gain that no other policy approaches. Score per second therefore understates Hybrid's practical efficiency when board clearing is the primary objective.

## 6.5 Single-Game Case Study: Grid `lfreq_7`

The aggregate results in Sections 6.1– 6.4 establish that the Hybrid policy clears the board significantly more often than any greedy variant while also achieving the highest mean raw score across all 100 grids. To understand why this is the case at the level of individual decisions, this section presents a move-by-move analysis of a single representative grid, `lfreq_7`, on which Hybrid clears the board while every other policy fails. It has a vowel ratio of 0.31 and a rare-letter ratio of 0.03, making it a moderately difficult instance consistent with the grid properties.

Figure 38 shows the initial 12×9 board of 108 tiles presented identically to all five policies.

### Move 1: The Divergence Point

Every policy diverges at the very first move. Table 12 summarises the choice made by each policy and its immediate outcome.

Greedy-1 selects `representments` because it carries the highest immediate score on the board (280 points). Greedy-2 and Greedy-3 both independently select `unsentenced`: their 2-move and 3-move lookahead horizons agree that this word leads to the highest projected score over the next two or three moves. Hybrid selects `presentments` (216 points), which is neither the top immediate scorer nor the greedy pair or trio winner. It chooses this word

| o | r | l | g | t | d | h | s | n |
|---|---|---|---|---|---|---|---|---|
| s | n | s | d | r | r | a | t | t |
| t | o | d | e | g | t | t | p | i |
| p | a | m | u | a | b | e | g | g |
| i | m | e | p | p | p | t | c | a |
| u | t | m | t | e | e | e | n | u |
| s | e | n | n | x | s | x | d | e |
| n | u | c | o | d | c | e | r | j |
| s | s | p | e | l | b | o | r | e |
| i | l | r | s | g | b | t | s | p |
| r | i | a | r | g | y | e | g | e |
| t | l | y | g | e | o | f | e | n |

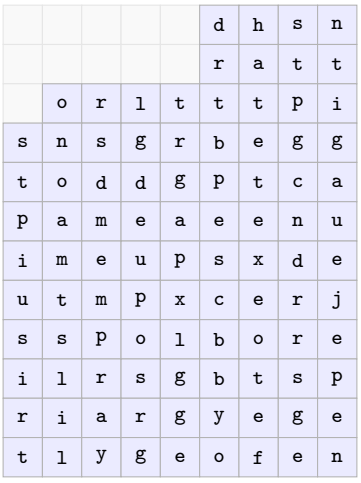
Figure 38: Initial grid for `lfreq_7` (12×9, 108 tiles). This is the identical starting state presented to all five policies. The grid has a vowel ratio of 0.31 and a rare-letter ratio of 0.03, making it a moderately difficult instance.

Table 12: First-move choices for each policy on `lfreq_7`.

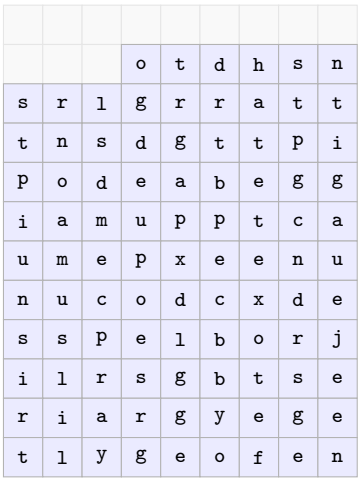
| Policy | Word chosen | Score | Tiles removed |
|---|---|---|---|
| Random | coder | 40 | 5 |
| Greedy-1 | representments | 280 | 14 |
| Greedy-2 | unsentenced | 198 | 11 |
| Greedy-3 | unsentenced | 198 | 11 |
| Hybrid | presentments | 216 | 12 |

because the full-game rollout from the resulting board state is the only one among the top-20 candidates that eventually leads to a clear.

Figure 39 shows the board state after Move 1 for Greedy-3 and Hybrid side-by-side. The grey cells indicate removed tiles, with gravity applied so remaining tiles fall to the bottom. The two boards already have a visibly different letter distribution after just one move, and this difference propagates through the remainder of the game.



Greedy-3: played **unsentenced** (198 pts, 11 tiles removed)



Hybrid: played **presentments** (216 pts, 12 tiles removed)

Figure 39: Board state after Move 1 for Greedy-3 and Hybrid.

### Why Each Greedy Policy Gets Stuck

Despite applying increasingly sophisticated lookahead, all four non-Hybrid policies exhaust their legal moves before clearing the board.

**Greedy-1** (10 tiles remaining): By always picking the highest-scoring available word at each turn, Greedy-1 accumulates score quickly in the early game. Its first three moves alone yield 710 points via `representments`, `superlarge`, and `misemploys`. However, this aggressive tile removal rapidly

depletes the vowels from the board, leaving a consonant-heavy residual that supports very few valid words. By Move 8 the policy is reduced to words such as `jestee` (78 points) and by Move 12 to 3-letter words including `nth`, `brr`, and `dot`. The game ends with 10 tiles remaining that spell no valid word.

**Greedy-2** (12 tiles remaining): Planning in 2-move committed blocks improves average score per move (129 points) relative to Greedy-1 (113 points), but the 2-move horizon is still insufficient to anticipate the board topology several moves ahead. The policy terminates on `brr` with 12 tiles remaining. Notably, Greedy-2's average score per move is higher than Hybrid's (120 points), confirming that it is successfully optimising local score but the board configuration it produces cannot be completed.

**Greedy-3** (6 tiles remaining): With a 3-move block horizon, this policy achieves the highest average score per move (139 points) of all policies and comes closest to clearing, leaving only 6 tiles. However, those 6 tiles are the letters `r g g g t g` (refer Figure 40), a cluster of consonants from which no valid word of length three or more can be formed. The terminal position arises as follows: at Move 13 the only available word is `tay` (18 points, 3 tiles), but after removing it the remaining `r g g g t g` cluster is entirely unplayable. Greedy-3 effectively painted itself into a corner: its 3-move planning window was sufficient to maximise score locally, but not sufficient to foresee that this particular sequence of moves would isolate an unplayable residual cluster.

### Why Hybrid Clears: The Rollout Mechanism

Hybrid evaluates candidate moves differently from all greedy variants. Rather than looking ahead a fixed number of moves, it performs a full-game rollout from each candidate first move, simulating the rest of the game to completion and selecting the candidate whose rollout leads to a board clear. On this grid, the rollout from `presentments` is the only one among the top 20 candidates that terminates in a clear, which is why Hybrid selects it even though 216 points is not the optimal immediate or short-horizon score.

A second important mechanism is Hybrid's *phase-adaptive depth*, which governs which greedy policy is used inside each rollout. The depth is selected based on the fraction of tiles remaining:

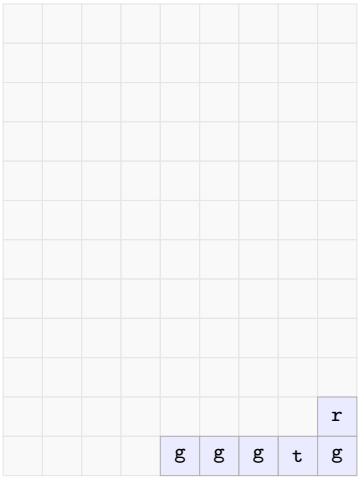
Table 13: Hybrid phase transitions on `lfreq_7`. Depth is selected based on the proportion of tiles remaining relative to the initial 108. Move ranges shown are specific to `lfreq_7` (15 moves total). Phase boundaries are determined by tile fraction, not move count.

| Phase | Tiles remaining | % of board | Rollout depth used |
|---|---|---|---|
| Early | >54 tiles | >50% | Greedy-1 (Moves 1–6) |
| Middle | 22–54 tiles | 20–50% | Greedy-2 (Moves 7–10) |
| Late | <22 tiles | <20% | Greedy-3 (Moves 11–15) |

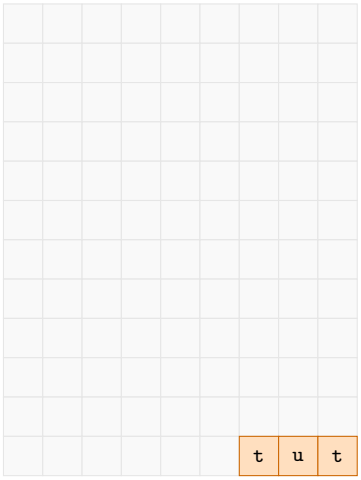
As the board empties, Hybrid deepens its rollout policy automatically. By Move 11 (20 tiles remaining), it switches to Greedy-3-depth evaluation on a board small enough for 3-move planning to be highly effective. From this point, Hybrid navigates the endgame via the sequence `claspt` → `bind` → `jet` → `trog` → `tut`, clearing the board on Move 15.

The final five moves are all low-scoring (84, 40, 30, 20, and 12 points respectively), but each removes tiles in a way that preserves the connectivity and word-formability of the remaining letters. This is precisely the property that fixed-horizon greedy policies cannot guarantee: they have no mechanism to verify that the tiles they leave behind will remain playable

Figure 40 shows the contrast between the two endgame states. Greedy-3 is left with 6 unplayable tiles, while Hybrid reaches a 3-tile state spelling `tut` before clearing the board.




Greedy-3: **6 tiles remain** after Move 13 (`tay`). No valid word can be formed from `r g g g t g`. **Stuck.**




Hybrid: **3 tiles remain** before Move 15. The word `tut` clears the board. +**500 bonus.**

Figure 40: Endgame comparison for `lfreq_7`. Greedy-3 (left) reaches a terminal state with 6 tiles forming no valid word — the isolated consonant cluster `r g g g t g` is unplayable. Hybrid (right) arrives at 3 tiles spelling `tut`, completing the clear and earning the 500-point bonus. This contrast illustrates why full-game rollout evaluation outperforms fixed-depth lookahead on difficult grids.

### Score Trade-off

Table 14 and Figure 41 summarise the final outcomes. Hybrid's cumulative score before the clear bonus (1,798 points) is actually *lower* than Greedy-3's (1,810 points) — a deficit of just 12 points. At no point during the game does Hybrid lead on raw cumulative score; Greedy-3 is consistently ahead in score-per-move terms. However, the 500-point clear bonus gives Hybrid a final total of 2,298 points, a margin of 488 points over Greedy-3.

Table 14: Final outcomes for all five policies on `lfreq_7`.

| Policy | Moves | Score (no bonus) | Clear bonus | Total score | Cleared |
|---|---|---|---|---|---|
| Random | 23 | 859 | — | 859 | No |
| Greedy-1 | 14 | 1,587 | — | 1,587 | No |
| Greedy-2 | 13 | 1,679 | — | 1,679 | No |
| Greedy-3 | 13 | 1,810 | — | 1,810 | No |
| Hybrid | 15 | 1,798 | +500 | 2,298 | Yes |

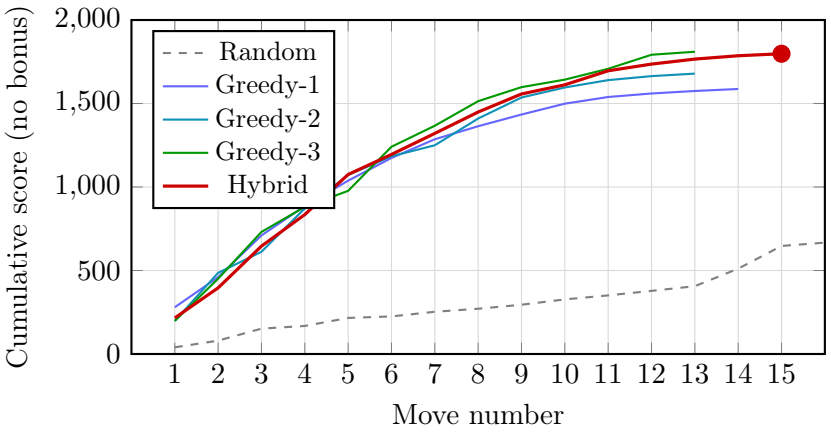



Figure 41: Cumulative score progression (excluding the clear bonus) for each policy on `lfreq_7`. The Hybrid policy (red) does not lead in raw score at any point in the game; Greedy-3 (green) consistently accumulates higher scores per move. However, Hybrid reaches a clearable final state at Move 15, earning the 500-point bonus for a total of 2,298 points (488 points ahead of Greedy-3's final total of 1,810).

This result reflects the central tension identified in Section 6.1.2: greedy policies with higher lookahead depth accumulate more raw score, but they do so by optimising move sequences that do not account for long-term board clearability. Hybrid accepts a small raw-score penalty in exchange for a trajectory that ends in a clear, and the 500 point bonus more than compensates. To put this in perspective, a typical move in this game scores in the range of 100–150 points, so the 500 point bonus is worth roughly three to five additional moves. On this grid, the raw score sacrifice is just 12 points; the net gain from clearing is 488 points, nearly half the bonus itself. This case study provides a concrete, move-level explanation of the aggregate pattern observed across all 100 grids in Section 6.2.

# 7 Discussion

This chapter brings together the findings from Chapter 6 and interprets what they reveal about the relative strengths and limitations of each automated policy. Throughout this chapter, the term *player* refers to an automated policy running without human input.

## 7.1 Interpretation of Results

The experimental results demonstrate that players with more advanced strategies in decision making consistently perform better, although the extent of improvement depends on which performance metric is considered.

For board clearing, the improvement across policies is not gradual but sharply non-linear. The three greedy policies achieve a clear rates of 6%, 8% and 10% respectively, suggesting that increasing ply depth from one to three produces only marginal gains in clearing ability. The hybrid player, by contrast, clears 87% of the grids. This gap cannot be explained by lookahead depth alone. All four non-random policies share the same objective which is clearing the grid, but the greedy policies apply it through fixed-depth search, which is too shallow to anticipate the long chains of moves required to fully empty the board. The Hybrid player's full-game rollout simulation evaluates entire trajectories rather than short sequences, giving it a qualitatively different ability to identify board-clearing paths. The result confirms that the jump from fixed-depth lookahead to rollout-based evaluation is far more consequential for clearing success than any incremental increase in ply depth.

For raw word score, the results are less straightforward. Among the greedy policies, mean score increases with ply depth, which is consistent with the theoretical expectation that deeper lookahead should produce better move selection. However, the Hybrid policy's raw score advantage over the greedy policies is smaller than its total score advantage, because a substantial portion of Hybrid's lead comes from the 500-point clearing bonus rather than from superior word selection alone. This distinction is important: Hybrid is

a stronger clearing strategy, but it does not necessarily select higher-scoring individual words than a well-tuned greedy player, at least in the short term where greedy search remains an effective means of accumulating word score move by move.

The runtime analysis reveals the expected exponential cost of increasing ply depth among the greedy policies. Greedy-3 is substantially slower than Greedy-1 and Greedy-2, while delivering only a marginal improvement in clear rate. This confirms that increasing ply depth beyond a certain point yields diminishing returns: runtime continues to grow while performance gains become smaller. Each greedy policy can be understood as producing a sequence of locally optimal decisions, selecting the best available move over a fixed horizon without reasoning about the global board state. For small ply depths, this is computationally tractable but structurally limited; for larger depths, the exponential branching of the search tree makes the approach infeasible. The Hybrid policy is slower still, but its runtime cost is justified by the large improvement in clearing success. Taken together, these findings suggest that a fixed depth greedy search is not well suited in clearing the board as a long-term objective, and that rollout-based evaluation represents a more appropriate strategy for that objective even at greater computational expense. A natural direction for strengthening the framework would be to explore adaptive depth control, where the search horizon expands selectively in the endgame when clearing becomes critical.

The correlation analysis further reinforces this picture. Vowel ratio emerged as the strongest structural predictor of clearing success, with low vowel density consistently associated with failure across all policies. This suggests that grid difficulty in Word Soup is not primarily determined by the number of words available at the start, but by the letter composition of the grid and its effect on word availability as the board empties. Initial word count showed no meaningful correlation with final score, confirming that a grid that appears rich with starting words at the start does not necessarily produce better outcomes, since early moves change the board in ways that the

initial count cannot anticipate.

### 7.2 Comparison with Human Play

To understand the performance of the Hybrid policy, this section compares it against an expert human player who played ten games of Word Soup under standard conditions. The human data was collected by the project supervisor and is summarised in Table 15. Since the human games used different grids from the 100 grid test set, a direct game comparison is not possible. Furthermore, the two sets of grids were generated by different means and may therefore be drawn from different distributions, which introduces an additional source of incomparability. For these reasons, only aggregate results are compared in this section, with a controlled same-grid comparison presented in Section 7.3. No broader baseline data on typical human performance at beginner or intermediate level was available for this study, so the human player here should be understood as a single expert reference point rather than a representative sample.

Table 15: Aggregate performance comparison between the expert human player and the Hybrid policy.

| Metric | Human (expert, $n = 10$) | Hybrid ($n = 100$) |
|---|---|---|
| Clear rate | 80% | 87% |
| Mean score (no bonus) | 1548.7 | 2391.4 |
| Mean score (with bonus) | 1948.7 | 2826.4 |
| Mean moves per game | 23.2 | 13.9 |

Figure 42 and 43 compare the score distributions and key metrics between the two players. The Hybrid policy achieves a substantially higher mean total score of 2826.4 points compared to the human's 1948.7 points, a difference of approximately 877 points. This gap is driven primarily by the raw score component: the Hybrid policy's mean score without the bonus (2391.4) is 842.7 points higher than the human's (1548.7), despite both players clearing the board at comparable rates.

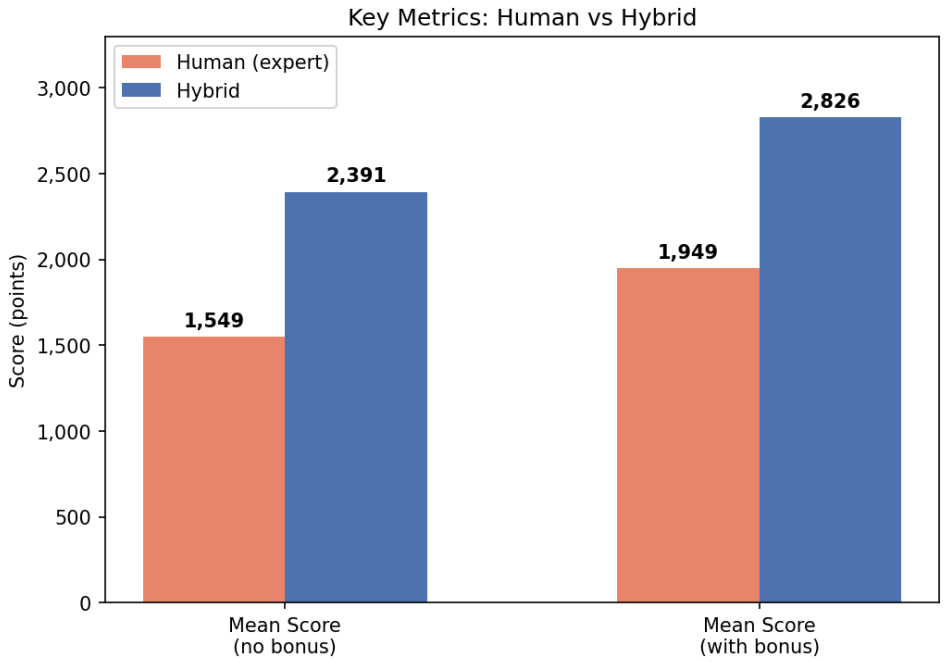



Figure 42: Key metric comparison between the expert human player ($n = 10$ games) and the Hybrid policy ($n = 100$ games) on mean score without the clearing bonus and mean total score with the bonus. The Hybrid policy leads substantially on both measures.

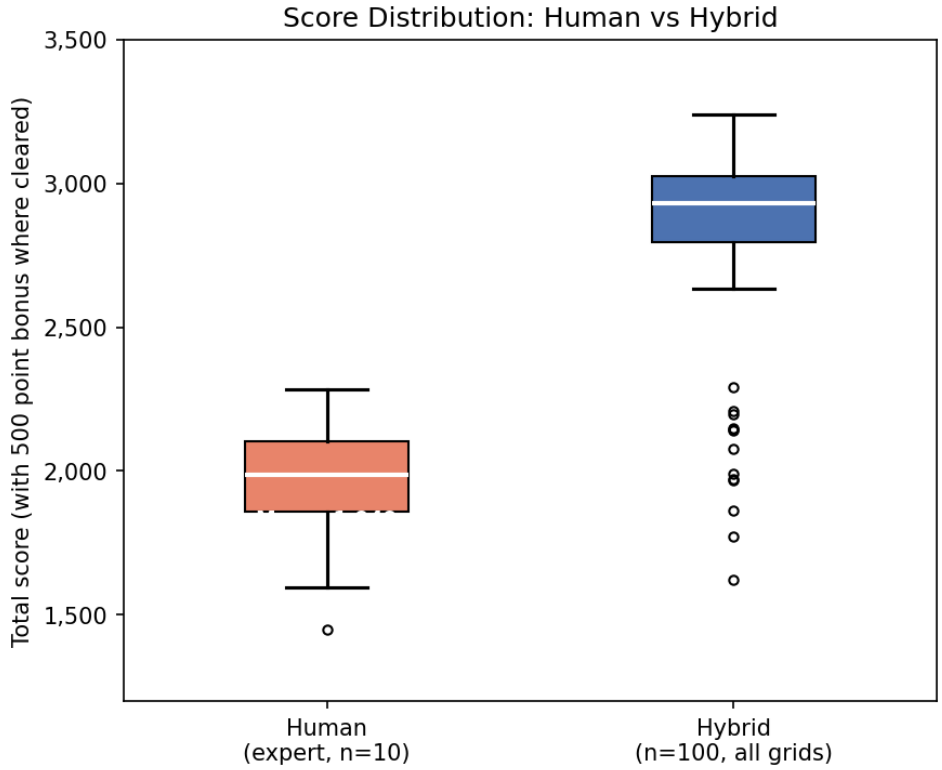



Figure 43: Score distribution comparison between the expert human player ($n = 10$ games) and the Hybrid policy ($n = 100$ games).

A notable difference lies in the number of moves per game. The human player uses a mean of 23.2 moves to complete a game, compared to Hybrid's 13.9 moves. This difference reflects a fundamental distinction in strategy. The human approach involves sacrificing short term score optimality in order to manoeuvre tiles into position for a single very long word, accepting a sequence of low scoring intermediate moves in exchange for a large eventual payoff. This is analogous to a long horizon planning strategy in combinatorial decision problems, where short term local optima are deliberately avoided in order to reach a more valuable global outcome. The Hybrid policy, by contrast, selects fewer but consistently higher scoring words by evaluating move sequences through rollout simulation, without targeting any specific word.

The human's best individual word across all ten games was EXTRATERRITORIALITIES (840 points, 21 letters), one of the highest scoring words achievable on this board size. However, this level of performance is not consistent across moves. The majority of human moves are short words scoring under 50 points, used primarily to set up or clear around the target long word.

The clear rate comparison should be interpreted with caution. The human achieved 8 clearances in 10 games (80%) while Hybrid achieved 87 clearances in 100 games (87%). The sample sizes differ substantially, the grids were not shared, and the two sets of grids may have been drawn from different distributions. For these reasons, the difference in clear rate cannot be taken as a statistically meaningful result. What can be said is that expert human play achieves a comparable clearing rate to Hybrid, but produces substantially lower scores, indicating that the Hybrid policy is more efficient at converting board positions into points.

### 7.3 Same Grid Comparison

The aggregate comparison above is limited by the fact that the human and Hybrid played on different grids and from a potentially different distribution. To address this, the Hybrid policy was run on the same ten grids used by the human player, allowing a direct comparison under identical starting conditions. Table 16 shows the result for each game. The most direct conclusion from the table is that when Hybrid clears the board, it wins; when it does not, the human has a chance of winning provided they can also clear.

Table 16: Game-by-game score comparison between the human player and the Hybrid policy on the same ten grids. Scores include the 500-point clearing bonus where applicable.

| Game | Human Score | Human Cleared | Hybrid Score | Hybrid Cleared | Winner |
|---|---|---|---|---|---|
| 1 | 1960 | Yes | 2514 | Yes | Hybrid |
| 2 | 1910 | Yes | 2438 | Yes | Hybrid |
| 3 | 2285 | Yes | 2420 | Yes | Hybrid |
| 4 | 2012 | Yes | 2291 | Yes | Hybrid |
| 5 | 1594 | No | 1862 | No | Hybrid |
| 6 | 1450 | No | 1897 | No | Hybrid |
| 7 | 2126 | Yes | 2071 | No | Human |
| 8 | 2034 | Yes | 1902 | No | Human |
| 9 | 1842 | Yes | 2400 | Yes | Hybrid |
| 10 | 2274 | Yes | 2621 | Yes | Hybrid |
| Mean | 1949 | 8/10 | 2242 | 6/10 | |

Hybrid wins 8 out of 10 games, with a mean total score of 2242 compared to the human's 1949. However, the human clears more boards (8/10 vs. 6/10), which reveals an important asymmetry in how each player accumulates points.

Figure 44 shows the total score for each game, and Figure 45 shows the same data with the no clearing bonus. On raw word score alone, Hybrid outscores the human in every single game, with a mean raw score of 1942 against the human's 1549. The Hybrid player's word selection advantage is therefore consistent across all ten grids regardless of clearing outcome. However, in Games 7 and 8, Hybrid's raw score margin over the human is less than 500 points, meaning the clearing bonus is sufficient for the human to overturn the deficit and win.

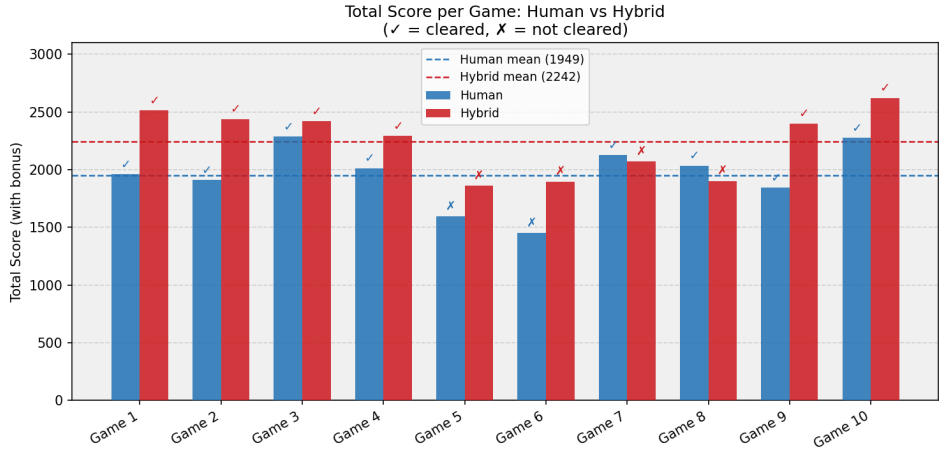



Figure 44: Total score per game including the clearing bonus. Tick marks above bars indicate whether the player cleared the board.

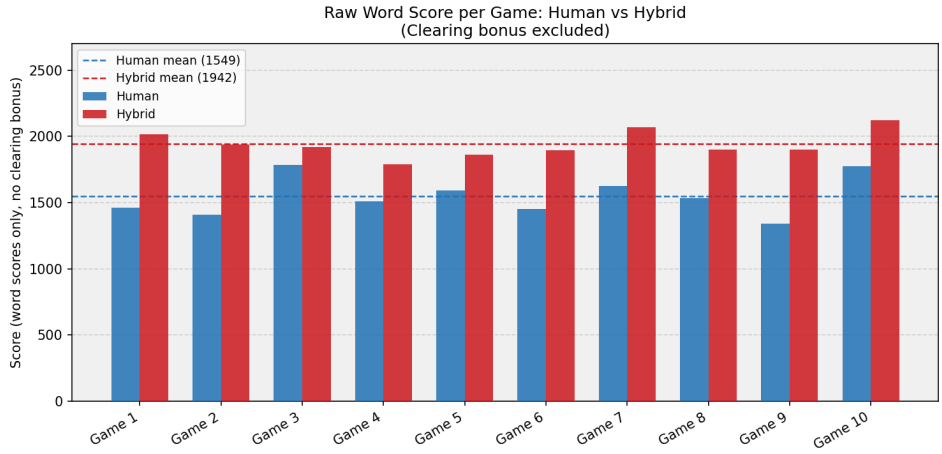



Figure 45: Raw word score per game with the clearing bonus excluded. Hybrid outscores the human on word selection alone in all ten games.

**The role of exceptional words.** A key feature of the human strategy is the objective of deliberately constructing very long words, particularly those including high value letters such as Q, X, and Z, to score large amounts in

a single move. Figure 46 compares the best single word played per game by both players. Human best words include EXTRATERRITORIALITIES (840 points), QUIZZICALITIES (784 points), and INTERNATIONALITIES (594 points). Hybrid's best single words are considerably shorter, typically scoring between 220 and 360 points. Despite this, Hybrid's raw score is higher in all 10 games because it scores consistently across all moves rather than concentrating score in one or two exceptional plays. The human's remaining moves after a long word are often short words scoring under 50 points, used primarily to remove residual tiles.

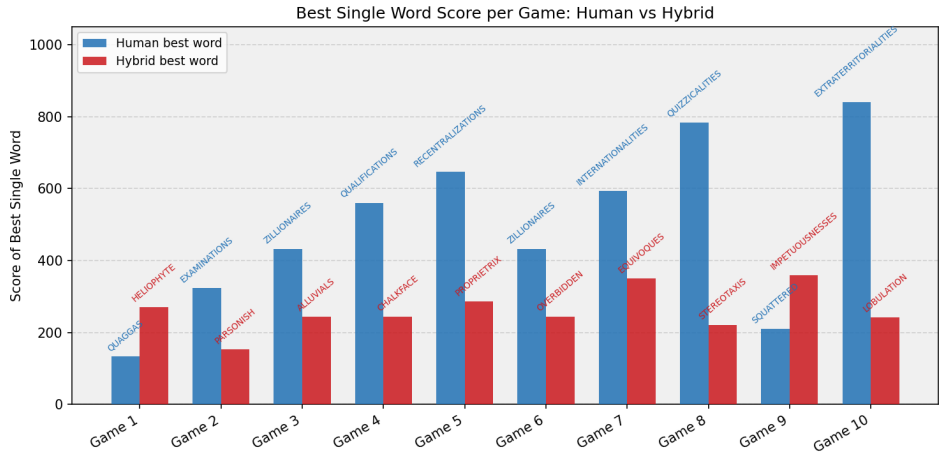



Figure 46: Score of the best single word played per game by each player. Human best words substantially exceed Hybrid's in most games, reflecting a deliberate strategy of building very long words by sacrificing short term score.

**Cumulative score progression.** Figure 47 shows the cumulative score for Game 1, where Hybrid's curve rises steeply and consistently, staying above the human's from the very first move. The human's curve rises gradually and steadily across 22 moves, with the only large jump at the final move when the 500 point clearing bonus is added. Notably, the human did not construct any exceptionally long word in this game; the longest words played were QUAGGAS and CHILDES at seven letters each, meaning the human's

score accumulated through many short to medium words rather than a single high value play. Despite both players clearing the board, Hybrid finishes 554 points ahead, reflecting its higher word scoring efficiency across all moves.

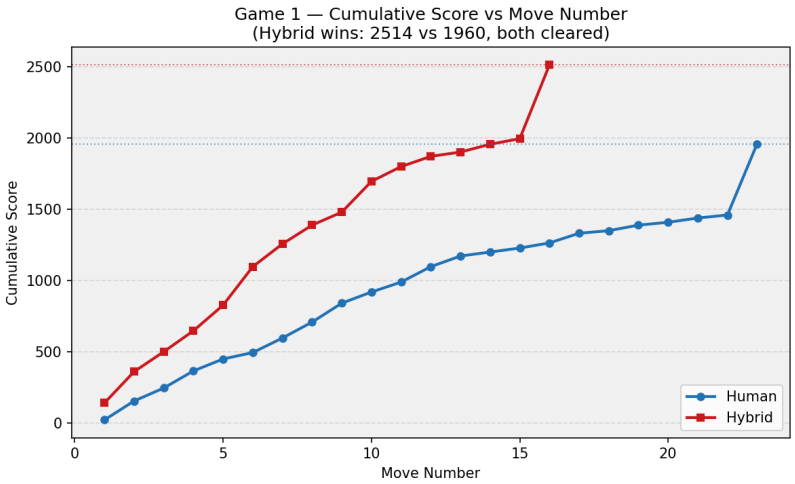



Figure 47: Cumulative score vs. move number for Game 1. Both players cleared the board. Hybrid's consistent word selection produces a steadier accumulation curve.

Games 7 and 8 are the two cases where the human wins. In both games the human played one exceptional word (INTERNATIONALITIES in Game 7, QUIZZICALITIES in Game 8) which produced a large spike in the cumulative curve, followed by smaller moves to clear the board. Figures 48 and 49 illustrate this pattern. Hybrid's raw word score exceeds the human's in both games, but because Hybrid fails to clear, it misses the 500 point bonus. Notably, in Game 7, Hybrid also plays one high value word, EQUIVOQUES (350 points), producing a visible spike in its cumulative curve around move 10; despite this, Hybrid still cannot clear the remaining tiles, suggesting that individual high value plays alone are insufficient to guarantee clearance on low vowel grids. The human's ability to clear the board after a long word move is therefore the decisive factor in both victories.

Table 17 and Table 18 list the full sequence of words played by each player

in Games 7 and 8 respectively.

Table 17: Word sequences for Game 7. Human cleared; Hybrid did not.

| **Human** | | | **Hybrid** | | |
|---|---|---|---|---|---|
| Move | Word | Score | Move | Word | Score |
| 1 | AYES | 28 | 1 | MORALITY | 152 |
| 2 | ILEX | 68 | 2 | TONNEAUX | 144 |
| 3 | VAU | 21 | 3 | VIRUCIDAL | 216 |
| 4 | BOLT | 40 | 4 | COMBATABLE | 220 |
| 5 | COP | 21 | 5 | EXCEPTIVE | 225 |
| 6 | MAXI | 60 | 6 | IMPOLITICAL | 341 |
| 7 | QUITE | 85 | 7 | ZAKATS | 114 |
| 8 | CHARK | 70 | 8 | FORWORN | 98 |
| 9 | REC | 15 | 9 | CHATTY | 84 |
| 10 | MOP | 21 | 10 | EQUIVOQUES | 350 |
| 11 | INTERNATIONALITIES | 594 | 11 | CREDO | 40 |
| 12 | EXCLAVE | 161 | 12 | SOC | 15 |
| 13 | VACS | 36 | 13 | JAG | 33 |
| 14 | GYM | 27 | 14 | CWM | 30 |
| 15 | BOWL | 52 | 15 | TET | 9 |
| 16 | DUCT | 32 | | (did not clear) | |
| 17 | QUOIT | 85 | | | |
| 18 | CAZ | 42 | | | |
| 19 | CWM | 30 | | | |
| 20 | RIFT | 36 | | | |
| 21 | VOX | 39 | | | |
| 22 | TAR | 9 | | | |
| 23 | GET | 12 | | | |
| 24 | ROD | 12 | | | |
| 25 | JOT | 30 | | | |
| | (clear) | 500 | | | |
| | **Total** | **2126** | | **Total** | **2071** |

Table 18: Word sequences for Game 8. Human cleared; Hybrid did not.

| **Human** | | | **Hybrid** | | |
|---|---|---|---|---|---|
| Move | Word | Score | Move | Word | Score |
| 1 | RED | 12 | 1 | NONMORTALS | 180 |
| 2 | TAX | 30 | 2 | RAZZED | 150 |
| 3 | MORN | 28 | 3 | STEREOTAXIS | 220 |
| 4 | SHEET | 40 | 4 | JONCANOES | 180 |
| 5 | JORS | 44 | 5 | PLENTEOUS | 153 |
| 6 | NON | 15 | 6 | CIRRHOPODS | 200 |
| 7 | RIGHT | 55 | 7 | UNJOINT | 133 |
| 8 | QAT | 36 | 8 | GIDDIEST | 120 |
| 9 | POST | 24 | 9 | TENFOLDS | 136 |
| 10 | LEOS | 32 | 10 | SCHERZO | 147 |
| 11 | SCENT | 40 | 11 | QURSH | 90 |
| 12 | FLOWS | 75 | 12 | TRANQ | 75 |
| 13 | PENNE | 45 | 13 | FETT | 28 |
| 14 | DEN | 15 | 14 | HOPPY | 75 |
| 15 | OPS | 15 | 15 | NUR | 15 |
| 16 | ORDS | 20 | | (did not clear) | |
| 17 | JOES | 44 | | | |
| 18 | QUIZZICALITIES | 784 | | | |
| 19 | NTH | 21 | | | |
| 20 | DUO | 15 | | | |
| 21 | PRY | 24 | | | |
| 22 | FRA | 18 | | | |
| 23 | RUN | 15 | | | |
| 24 | DZO | 39 | | | |
| 25 | CUT | 18 | | | |
| 26 | HOT | 18 | | | |
| 27 | RAN | 12 | | | |
| | (clear) | 500 | | | |
| | **Total** | **2034** | | **Total** | **1902** |

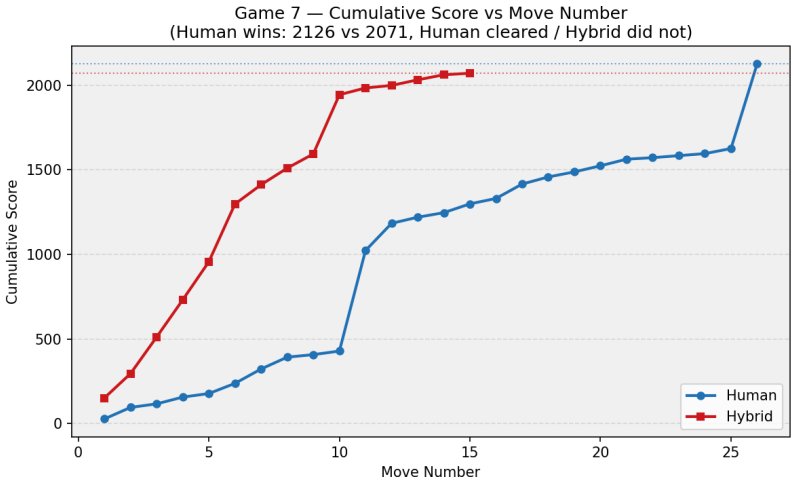



Figure 48: Cumulative score vs. move number for Game 7. The human's INTERNATIONALITIES (594 points) and subsequent board clear produces the winning margin despite Hybrid's higher raw word score.

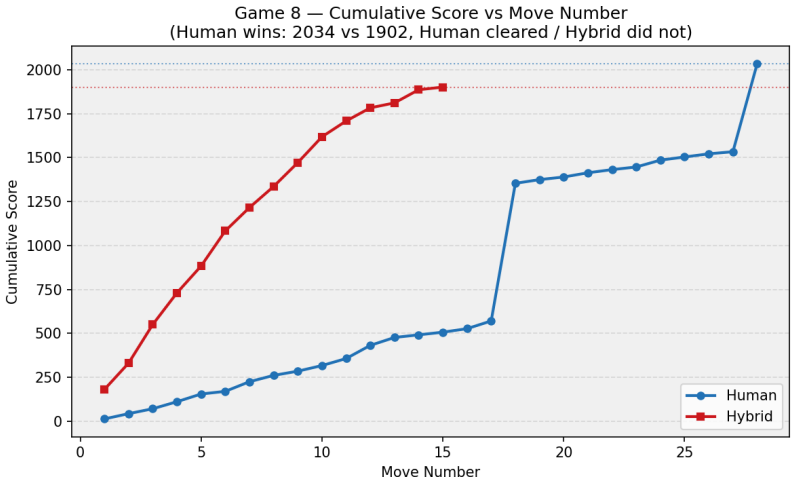



Figure 49: Cumulative score vs. move number for Game 8. The human spends the first 17 moves accumulating a modest score before playing QUIZZICALITIES (784 points) for a decisive spike, then clears with short words. This illustrates the contrast between the human's long-term word construction strategy and Hybrid's consistent but bounded scoring approach.

**Why Hybrid fails to clear in four games.** The four grids where Hybrid fails to clear (Games 5, 6, 7, 8) have among the lowest vowel ratios in the set: 0.315, 0.278, 0.380, and 0.333 respectively. This is consistent with the finding in Section 6.2.1 that vowel ratio is the strongest structural predictor of board clearance. Low vowel density restricts the formation of legal words from the remaining tiles as the board empties, increasing the risk of reaching a dead end with a small number of cells still present. Hybrid's rollout-based clearing strategy is highly effective on letter-frequency-based grids but does not fully compensate for this structural constraint.

The human player, by contrast, removes large numbers of tiles in a single long word, reducing the remaining tile count rapidly and preserving more flexibility in the endgame even under low vowel conditions. Unlike Hybrid, the human may be implicitly responding to low vowel density by targeting

words that use fewer vowels, such as QUIZZICALITIES and CWM, rather than relying on vowel-rich combinations. This is not a deliberate algorithmic response to vowel ratio, but rather a natural consequence of scanning the full board for high value letter combinations regardless of vowel content. This is clearly visible in Game 6, where Hybrid leaves 12 cells remaining despite outscoring the human on raw word score. Neither player clears that grid, which has the lowest vowel ratio in the set (0.278).

Overall, the same grid comparison confirms that the Hybrid policy leads consistently on word scoring efficiency but is more vulnerable to unfavourable grid structure than the human. The two games where the human wins both involve the combination of one very high value word and a successful board clear, a pattern that Hybrid's fixed rollout strategy does not have. This suggests a direction for future improvement: incorporating vowel ratio as an explicit signal during move selection, so that the policy can adjust its strategy on low vowel grids rather than applying the same rollout depth uniformly. A version of the Hybrid policy that targets consonant-compatible words earlier in low vowel games may be better placed to avoid the dead end tile configurations that prevent clearance.

## 7.4 Limitations

Several limitations of this study should be acknowledged before drawing final conclusions.

**Non-comparable to human baseline.** The human gameplay data used in Section 7.2 was collected on a different set of grids from the 100 grids evaluation batch. Since the grids were not shared between human and automated policies, the comparison cannot be controlled for grid difficulty. The human sample also consists of only ten games, which is too small to draw statistically reliable conclusions. The clear rate difference between the human (80%) and Hybrid (87%) should therefore be treated as indicative rather than definitive. Nevertheless, the same-grid comparison was able to

identify specific advantages and vulnerabilities in each strategy, such as Hybrid's consistent word-scoring efficiency and the human's greater resilience on low-vowel grids.

**Long-word blindness.** All five policies are unable to target high scores and long words, although the reason differs by policy. The Random policy selects moves without any evaluation at all. Greedy-1 considers only the current move and cannot plan beyond it. Greedy-2 and Greedy-3 are bounded by their fixed lookahead depths of two and three plies, respectively, meaning they cannot reason about words requiring longer planning plies. Hybrid evaluates moves through full game rollouts, but these rollouts simulate sequences of greedy decisions rather than searching for specific target words. As a result, no policy can construct the path required to play a word such as EXTRATERRITORIALITIES, even if the letters are present on the board. This is consistent with the behaviour of greedy and rollout based methods more generally: each policy converges on a sequence of locally optimal moves, which forecloses the possibility of reserving tiles for a single very high value word later in the game. From the human gameplay evidence, paths to such words do exist in the solution tree, but they require holding back tiles across many moves, a form of planning that is not captured by any fixed depth lookahead or greedy rollout. A policy that could explicitly target long words would require either a stochastic search over a much wider solution space, or an approach closer to the human strategy of deliberately constructing a target word before clearing residual tiles.

**Bonus inflation of Hybrid scores.** The 500-point clearing bonus means that Hybrid's total score advantage over the greedy policies is partly a product of its clearing dominance rather than its raw word-selection quality. When comparing mean scores with the bonus included, Hybrid appears substantially ahead. Comparing scores without the bonus gives a more honest view of word-selection quality, and the gap is narrower in that case. Any interpretation of Hybrid as a stronger scoring player should account for this distinction. This is inherent in the structure of the problem: to achieve the

clearing bonus as a long term objective, the policy must sacrifice some short term scoring opportunities, and this trade off can be understood within the broader framework of sequential decision making under delayed reward.

**Spatial vowel distribution not captured.** The vowel ratio metric used throughout this study is an aggregate figure, computed as the proportion of vowels across all 108 cells at the start of the game. This does not capture where the vowels are located on the board. Grid `lfreq_6` illustrated this limitation: its aggregate vowel ratio was not unusually low, yet Hybrid still failed to clear it. The distribution of vowels across rows and columns, and in particular whether vowels are clustered or spread evenly, may influence playability in ways that the aggregate ratio cannot detect. This spatial dimension was not measured and represents a gap in the current feature analysis.

## 7.5 Future Work

The directions for future work fall into two broad categories. The first concerns tactical improvements that can be made directly within the existing framework, such as refining the planning strategy, tuning algorithm parameters, and enriching the grid features used for difficulty prediction. The second concerns a more substantial algorithmic extension, namely the formalisation of the Hybrid player's rollout mechanism within the Monte Carlo Tree Search (MCTS) framework [3], which connects this project to a wider body of research in sequential decision-making and combinatorial game solving.

### 7.5.1 Implementation Improvements

**Targeted long-word construction.** All policies in this study select moves greedily from the set of currently available words, which means that words are selected purely based on what is reachable at the current board state, with no mechanism to pursue a specific high-scoring word across multiple moves. A more sophisticated or structured approach would work in reverse:

identify a specific high-scoring word, determine which letters are needed and where they are currently on the board, and then plan a sequence of pre-moves that will gradually move those letters into an adjacent path, though doing this optimally would be computationally expensive given the large number of possible move sequences; a stochastic search approach would therefore be more practical than exhaustive planning. Intermediate moves in this sequence can score poorly, but the final payoff from the target word can substantially exceed what a greedy policy would select move-by-move over the same horizon, precisely because a greedy criterion discards low-scoring intermediate moves that are necessary to reach the high-value target. This idea, sometimes described as finding the needle in a haystack, represents a fundamentally different planning paradigm and requires a dedicated search algorithm separate from the greedy framework developed here. It is primarily motivated by taking the one advantage the human player has over Hybrid: the ability to identify a far-field objective and commit to it, while deliberately sacrificing near-field optimality in the intermediate moves. In tree search terms, this corresponds to a different kind of pruning, one that discards branches which cannot lead to a specific target word, rather than branches with low immediate score. This is the opposite of what greedy and rollout based methods do, and formalising it would bring the approach closer to goal search.

One practical consideration that makes this direction more tractable than it might initially appear is the size of the target word set. Of the 278,688 words in the dictionary, only a small fraction score above a high threshold, for example 500 points or more. Many of these are further eliminated during the initial grid filter, since any word requiring letters absent from the current board is immediately ruled out. The remaining set of viable high value targets is therefore small enough to enumerate explicitly at the start of each game. These candidates can then be prioritised further by computing a rough estimate of how many moves would be needed before each target becomes reachable, based on how spatially close its required letters are at game start. Words whose required letters are already adjacent or nearly ad-

jacent rank highest, while words requiring letters that are far apart or buried beneath other tiles are less prioritised. This triage step does not require an exact computation; a simple distance heuristic applied to the initial grid is sufficient to rank candidates in order of difficulty. The result is a short, ordered list of concrete objectives that the targeted construction algorithm can attempt in sequence.

This commitment to a specific target at the cost of intermediate moves is structurally different from rollout planning [1], where there is no fixed goal; instead, the policy improves move by move based on expected future reward. Targeted construction assumes the goal is known in advance, which makes it closer to goal directed search than to approximate dynamic programming.

**Expanding Hybrid parameters.** The Hybrid policy uses a fixed shortlist of the top 20 candidate moves and fixed phase transition thresholds of 50% and 20% of remaining tiles. These values were chosen based on a balance between runtime and performance, but they were not systematically optimised. Increasing the beam width beyond 20 would allow the rollout stage to evaluate a broader set of candidates, potentially identifying clearing paths that the current shortlist misses. Similarly, adjusting the phase thresholds could improve endgame performance on difficult grids. Both changes are small parameter modifications that could be tested directly within the existing framework, making them a natural first step for any follow up study. A further extension would be to make these thresholds adaptive rather than fixed, adjusting the beam width and phase transition points based on the structural properties of the initial grid, such as vowel ratio or branching factor, so that the policy allocates more search effort on grids that are structurally harder to clear. A simpler but complementary approach would be to allow the algorithm to backtrack and retry with adjusted parameters whenever a game ends without clearing the board. Since the 87% clear rate is preserved as a lower bound regardless of what the retry attempt produces, this carries no risk of degrading existing results and could recover at least some of the 13 failed grids at the cost of additional runtime.

**Spatial vowel distribution as a grid feature.** As noted in Section 7.4, the aggregate vowel ratio used throughout this study does not capture where vowels are located on the board. The anomalous behaviour of grid `lfreq_6` suggests that spatial distribution matters. A grid where vowels are clustered in one region and the rest of the board is consonant-heavy may be effectively harder than its aggregate ratio implies, because word formation depends on local adjacency rather than global letter counts. Future work could define spatial vowel features, such as the variance in vowel density across rows and columns or the size of the largest vowel connected region, and test whether these features improve the prediction of clearing success beyond what aggregate ratio alone can achieve. A related direction would be to track *problem letters*: tiles that persist on the board deep into the game and block clearance, typically rare or awkward consonants that are difficult to incorporate into valid words. Identifying these letters early, based on their position and the words they can contribute to, would allow the policy to prioritise removing them before the board reaches a state where no clearing path remains.

### 7.5.2 Monte Carlo Tree Search and Machine Learning

**Formalising the Hybrid rollout as MCTS.** The Hybrid player's evaluation phase; simulating full game trajectories from a shortlisted set of candidate moves, is conceptually close to the simulation phase of Monte Carlo Tree Search [3]. MCTS is a heuristic search algorithm that combines the precision of tree search with the generality of random sampling, making it well suited to problems where the state space is too large for exhaustive enumeration. It consists of four phases that repeat iteratively: selection, where the algorithm traverses the tree using a policy that balances exploration and exploitation; expansion, where a new child node is added; simulation, where the game is played out from the new node using a rollout policy; and back propagation, where the result is used to update scores along the path.

The Hybrid player in its current form implements an informal version of the selection and simulation phases only: beam search shortlists the top

candidates, and rollout simulates full games from each one. However, the Hybrid does not maintain a persistent search tree between moves, meaning that information from previous rollouts is discarded rather than accumulated. A formal MCTS implementation would retain this tree across decisions, allowing the algorithm to refine its estimates incrementally as more rollouts are performed. Replacing the fixed beam shortlist with a UCT (Upper Confidence Bounds for Trees) selection strategy would allow search effort to be directed dynamically toward moves that have shown the most promise in earlier iterations, rather than treating all top 20 candidates equally [7]. MCTS has been extended to practical domains including transportation and scheduling, demonstrating that the framework generalises well beyond classical board games.

From a theoretical standpoint, rollout can be understood as a single step of policy improvement applied to a base heuristic [1]. Bertsekas shows that one rollout step always produces a policy whose cost is no worse than that of the base heuristic. This is known as the cost improvement of the property. The Hybrid player therefore has a formal justification: its rollout simulation is performing one step of improvement over the underlying greedy policy, which is why it consistently outperforms the greedy policies on clearing rate. This framing also clarifies the limitation observed in the 13 remaining failed grids: the rollout step can only improve over what the base greedy policy is capable of, and on grids where even the best greedy moves leave the board in a structurally difficult state, one step of improvement is not enough, particularly given the depth of the game tree. A typical Word Soup game requires 15 or more moves to complete, meaning the search horizon is long and a single rollout step evaluates only one candidate trajectory from each shortlisted move rather than exploring the full tree of consequences. A full MCTS implementation, by repeating this process across many iterations and accumulating information from previous rollouts, would progressively refine these estimates and is likely to recover at least some of those failures.

An analogous situation arises in Wordle, where Bertsimas and Paskov [2]

demonstrate that greedy and approximate reinforcement learning methods consistently fail to recover the optimal policy, whereas exact dynamic programming succeeds. Their analysis shows that even small errors in the estimated value function compound across time steps, degrading the resulting policy substantially. This result is directly relevant to Word Soup: it motivates the use of rollout and MCTS as intermediate approaches that are more principled than greedy lookahead but more tractable than exact DP, and it illustrates why the transition from local to global optimality is non-trivial in sequential word-game settings.

**Combining MCTS with machine learning.** A further extension would incorporate machine learning into the MCTS framework, following the architecture surveyed by Kemmerling et al. [6]. Neural MCTS, in which a learned policy network and value network guide the selection and evaluation phases respectively, has been applied successfully across a wide range of combinatorial and sequential decision problems beyond games, including materials design and network optimisation. The survey identifies that the evaluation phase has the greatest impact on overall performance, and that agents combining a learned value function for node initialisation with rollout based evaluation achieve the most favourable results. This finding suggests that even a relatively simple value network trained on Word Soup game trajectories could substantially improve the quality of the beam search shortlist and the rollout criterion in the Hybrid policy.

The main challenge in pursuing this direction is training data generation. Unlike games such as Go or Chess where self-play is straightforward, Word Soup presents an imbalanced outcome distribution: most grids are cleared by the Hybrid policy but a small fraction are not, and those hard grids are precisely the ones of most interest. One approach to handling this imbalance would be oversampling of failed trajectories, with techniques such as SMOTE (Synthetic Minority Over-sampling Technique) [4] applied to the feature space of grid-level statistics (vowel ratio, branching factor, initial word count) to generate synthetic training examples for structurally difficult

grids. An alternative is to define the objective function not as the binary clear/fail outcome but as the remaining number of the cells metric, which provides a more informative training signal across all grids.

An important practical consideration is computational cost. The Hybrid policy already achieves an 87% clear rate without any machine learning, running on standard hardware without access to high performance computing resources. This is itself a noteworthy result: the rollout framework, a well-established technique in approximate dynamic programming, is competitive with the performance ceiling of this problem using only modest resources [1]. A machine-learning extension would add offline training cost but should not substantially increase the move inference time if the network is small and the MCTS beam width is held constant. Whether the additional clear rate gained over the Hybrid policy's 87% baseline would justify this overhead depends on the difficulty of the 13 remaining failed grids are structurally unusual rather than simply unlucky.

**What would and would not work.** It is worth being clear about the limitations of these proposals. Full MCTS with a persistent search tree would require careful state representation: the Word Soup board after each move is a 12×9 grid with a specific letter arrangement, and two boards that look similar overall may behave very differently in terms of which words are locally reachable. Storing and reusing these states efficiently across moves is a non-trivial implementation challenge. The neural MCTS approach would also require enough training data to generalise across different grids, and it is not clear that the 100 grids used in this study would be sufficient; a larger and more varied set of training grids would need to be generated first. It is also worth noting that any extension built on top of the Hybrid policy cannot reduce the 87% clear rate already achieved. The 87 grids that the Hybrid successfully clears can be retained as-is; the extended method only needs to be applied to the 13 remaining failed grids. This means the current result acts as a guaranteed lower bound, and any improvement, however small, is strictly additive.

Finally, it is important to acknowledge that adding MCTS and machine learning on top of the Hybrid policy does not guarantee better results on every grid. However, this is not necessarily a drawback. If the additional search complexity produces no improvement on the grids that the Hybrid currently fails, that outcome would itself be informative: it would confirm that those grids are hard because of how their letters are arranged spatially, not because the search depth is insufficient. This would shift the focus toward a more targeted and tractable direction which is designing better spatial grid features, rather than continuing to invest in deeper search. In this sense, a negative result narrows the problem and points clearly to where future effort should go, which is a valuable contribution in its own right.

# 8 Conclusion

This project investigated whether algorithmic players with increasing levels of lookahead could solve the Word Soup puzzle more effectively, measured by two criteria: the total score achieved and the rate at which the board is fully cleared. Five players were built and tested: a random baseline, three greedy players with 1-ply, 2-ply, and 3-ply lookahead, and a Hybrid player using beam-search shortlisting combined with full-game rollout simulation. All five were tested on 100 grids generated using letter frequency distributions under the same conditions, providing a fair and reproducible basis for comparison. The problem is framed as a sequential decision problem in which each move prunes the solution tree irreversibly, connecting the design of the players to broader ideas in dynamic programming, heuristic search, and parameter optimisation.

The most striking finding is that fixed-depth greedy search, regardless of how many plies are added, is fundamentally limited as a board-clearing strategy. Going from 1-ply to 3-ply raised the clear rate from 6% to only 10%, while the Hybrid player cleared 87% of grids by simulating entire game trajectories rather than short sequences. This gap suggests that the outcome is highly sensitive to early decisions: only a small proportion of all solution branches lead to a cleared board, and fixed depth search is not deep enough to distinguish them reliably. Beyond clearing, the analysis of grid structure revealed that vowel ratio is the single most useful predictor of whether a grid can be cleared, and that the number of words available at the start of a game tells us very little about the final score. A direct head-to-head comparison with an expert human player showed that Hybrid outscored the human in 8 out of 10 games, and on raw word score in all 10, though the human's ability to construct exceptionally long words, thereby accessing regions of the solution tree that are structurally unreachable by a greedy or rollout policy, proved decisive in the two games the human won. The results suggest that this gap could be narrowed by adjusting the Hybrid's parameters, such as increasing the beam width or raising the minimum word length threshold, without re-

quiring a fundamentally different algorithm.

The study has a number of limitations. The human comparison is based on a small sample of ten games played on different grids, so the clear rate difference between the human and Hybrid cannot be treated as a statistically reliable result. The vowel ratio metric is a single aggregate number and does not capture where vowels are located on the board, which appears to matter in some failure cases. Additionally, none of the five policies can deliberately plan toward a specific long word, since all strategies work with whatever words happen to be available at each step. The presence of rare letters, specifically Q, Z, X, and J, poses a further challenge: these tiles carry high individual letter values but appear in very few valid words, meaning they are difficult to incorporate and tend to persist on the board late into the game, where they can obstruct the remaining adjacency paths needed for clearance.

Future work could pursue several directions. A targeted long-word planning algorithm, one that works backwards from a chosen high-value word to engineer the sequence of moves needed to make it available, would go beyond what any of the current policies can achieve. Across the full dictionary of 278,688 words, the mean word score is approximately 200 points and the top 5% of words score 448 points or above, meaning that a relatively small set of high-value targets can be identified at the start of each game and ranked by how reachable their required letters are on the initial board. The Hybrid player's parameters, such as beam width and phase thresholds, were not optimised and could be tuned further. Spatial vowel features that capture the distribution of vowels across the board, rather than just the overall proportion, could improve the prediction of grid difficulty and help identify in advance which grids are likely to resist clearing even by the strongest policy.

# 9 Python Code for Players

## 9.1 Greedy Game Loop: `play_one_greedy_run`

This is the main game loop for all greedy-based policies (Greedy-1, Greedy-2, and Greedy-3). It runs a single complete game from start to finish. On each turn, it finds all legal words on the current board, selects the best move according to the chosen ply depth, removes the word's tiles, and applies gravity. The loop also records game statistics at each step, including the branching factor (number of available words), the top move scores, and the points per move. The game ends when no legal moves remain or the board is cleared.

```
def play_one_greedy_run(
    G: List[List[Optional[str]]],
    trie: "base.Trie",
    *,
    min_len: int = 3,
    ply: int = 1,
    max_candidates: int = DEFAULT_MAX_CANDIDATES,
) -> Tuple[int, bool, List[str], Dict[str, float]]:

    total = 0
    chosen: List[str] = []
    branching_factors: List[int] = []
    top1_scores: List[int] = []
    top3_means: List[float] = []
    tie_events = 0
    multi_path_events = 0
    move_scores: List[int] = []
    tiles_removed = 0
    pending_plan: List[Tuple[str, Tuple[Coord, ...]]] = []
    while True:
        playable_all = words_map_all_paths_local(G, trie, min_len
    =min_len)
        # branching factor and top scores
        bf = len(playable_all)
```

```
        branching_factors.append(bf)
        all_scores_sorted = sorted((base.score_word(w) for w in playable_all.keys()), reverse=True)
        if all_scores_sorted:
            top1_scores.append(all_scores_sorted[0])
            top3_means.append(sum(all_scores_sorted[:3]) / min(3, len(all_scores_sorted)))
        else:
            top1_scores.append(0)
            top3_means.append(0.0)
        if not playable_all:
            metrics = {
                "n_moves": float(len(chosen)),
                "branching_factor_mean": float(statistics.mean(branching_factors)) if branching_factors else 0.0,
                "branching_factor_median": float(statistics.median(branching_factors)) if branching_factors else 0.0,
                "top1_score_mean": float(statistics.mean(top1_scores)) if top1_scores else 0.0,
                "top3_mean": float(statistics.mean(top3_means)) if top3_means else 0.0,
                "tie_rate": float(tie_events / len(branching_factors)) if branching_factors else 0.0,
                "multi_path_rate": float(multi_path_events / max(1, len(chosen))),
                "avg_points_per_move": float(total / max(1, len(chosen))),
                "max_move_score": float(max(move_scores) if move_scores else 0),
                "avg_word_len": float(statistics.mean([len(w) for w in chosen])) if chosen else 0.0,
                "score_per_removed_cell": float(total / max(1, tiles_removed)),
                "final_cells_remaining": float(_count_nonempty(G)),
            }
            # if no moves and grid empty, then CLEARED
```

```
48            if _grid_is_empty(G):
49                return total + CLEAR_BONUS, True, chosen, metrics
50            return total, False, chosen, metrics
51        if ply in (2, 3):
52            if not pending_plan:
53                if ply == 2:
54                    pending_plan = _best_two_move_plan(G, trie,
    min_len=min_len, max_candidates=max_candidates)
55                else:
56                    pending_plan = _best_three_move_plan(G, trie,
     min_len=min_len, max_candidates=max_candidates)
57
58            if not pending_plan:
59                return total, False, chosen, {
60                    "n_moves": float(len(chosen)),
61                    "branching_factor_mean": float(statistics.
    mean(branching_factors)) if branching_factors else 0.0,
62                    "branching_factor_median": float(statistics.
    median(branching_factors)) if branching_factors else 0.0,
63                    "top1_score_mean": float(statistics.mean(
    top1_scores)) if top1_scores else 0.0,
64                    "top3_mean": float(statistics.mean(top3_means
    )) if top3_means else 0.0,
65                    "tie_rate": float(tie_events / len(
    branching_factors)) if branching_factors else 0.0,
66                    "multi_path_rate": float(multi_path_events /
    max(1, len(chosen))),
67                    "avg_points_per_move": float(total / max(1,
    len(chosen))),
68                    "max_move_score": float(max(move_scores) if
    move_scores else 0),
69                    "avg_word_len": float(statistics.mean([len(w)
     for w in chosen])) if chosen else 0.0,
70                    "score_per_removed_cell": float(total / max
    (1, tiles_removed)),
71                    "final_cells_remaining": float(
    _count_nonempty(G)),
```

```
                }
            w, path = pending_plan.pop(0)

            top_score = max(base.score_word(x) for x in playable_all.keys()) if playable_all else 0
            ties = sum(1 for x in playable_all.keys() if base.score_word(x) == top_score)
            is_tie = ties > 1

            is_multi = (w in playable_all) and (len(playable_all[w]) > 1)

            move_s = base.score_word(w)
            tiles_inc = len(path)

            base.physics.remove_path(G, path)
            base.physics.apply_gravity(G)
        else:
            w, move_s, tiles_inc, is_multi, is_tie = _apply_one_greedy_move(
                G,
                trie,
                playable_all,
                min_len=min_len,
                ply=ply,
                max_candidates=max_candidates,
            )
        if is_tie:
            tie_events += 1
        if is_multi:
            multi_path_events += 1

        move_scores.append(move_s)
        chosen.append(w)
        total += move_s
        tiles_removed += tiles_inc
        if _grid_is_empty(G):
```

```
105             total_with_bonus = total + CLEAR_BONUS
106             return total_with_bonus, True, chosen, {
107                 "n_moves": float(len(chosen)),
108                 "branching_factor_mean": float(statistics.mean(
    branching_factors)) if branching_factors else 0.0,
109                 "branching_factor_median": float(statistics.
    median(branching_factors)) if branching_factors else 0.0,
110                 "top1_score_mean": float(statistics.mean(
    top1_scores)) if top1_scores else 0.0,
111                 "top3_mean": float(statistics.mean(top3_means))
    if top3_means else 0.0,
112                 "tie_rate": float(tie_events / len(
    branching_factors)) if branching_factors else 0.0,
113                 "multi_path_rate": float(multi_path_events / max
    (1, len(chosen))),
114                 "avg_points_per_move": float(total / max(1, len(
    chosen))),
115                 "max_move_score": float(max(move_scores) if
    move_scores else 0),
116                 "avg_word_len": float(statistics.mean([len(w) for
     w in chosen])) if chosen else 0.0,
117                 "score_per_removed_cell": float(total / max(1,
    tiles_removed)),
118                 "final_cells_remaining": float(_count_nonempty(G)
    ),
119             }
```

Listing 1: Main Game Loop for All Greedy Policies: `play_one_greedy_run`

### 9.2 Greedy-1: `_greedy_best_choice`

This function implements the Greedy-1 decision rule. It scans all legal words available on the current board and selects the one with the highest immediate letter value score. If two words have the same score, the longer word is preferred; ties are then broken by alphabetical order to ensure consistency. This is a 1-ply decision: it only looks at the current move and makes no attempt to evaluate future states.

```
def _greedy_best_choice(
    G: List[List[Optional[str]]],
    trie: "base.Trie",
    playable: Dict[str, List[Tuple[Coord, ...]]],
    *,
    min_len: int,
) -> Tuple[str, Tuple[Coord, ...]]:

    best_word: Optional[str] = None
    best_score = -1
    best_len = -1

    # step 1: choose the best word
    for w in playable.keys():
        s = base.score_word(w)
        L = len(w)
        if (
            s > best_score
            or (s == best_score and L > best_len)
            or (s == best_score and L == best_len and (best_word
    is None or w < best_word))
        ):
            best_score = s
            best_len = L
            best_word = w

    assert best_word is not None

    # step 2: choose the best path for that word
    paths = playable[best_word]
    if len(paths) == 1:
        return best_word, paths[0]

    def path_quality(path: Tuple[Coord, ...]) -> Tuple[int, float
    , int, Tuple[Coord, ...]]:

```

```
        G_after = _simulate_move(G, path)
        nxt = words_map_all_paths_local(G_after, trie, min_len=
    min_len)
        if not nxt:
            return (0, 0.0, 0, path)
        scores = sorted((base.score_word(x) for x in nxt.keys()),
     reverse=True)
        next_top1 = scores[0] if scores else 0
        next_top3 = sum(scores[:3]) / min(3, len(scores)) if
    scores else 0.0
        next_branching = len(nxt)
        return (next_top1, next_top3, next_branching, path)

    # choose path that maximise the tuple value = best board
    best_path = max(paths, key=path_quality)
    return best_word, best_path
```

Listing 2: Greedy-1 move selection: `_greedy_best_choice`

### 9.3 Greedy-2: `_best_two_move_plan`

This function implements 2-ply lookahead. It first ranks all legal first moves by their immediate score and prunes the list to the top 20 candidates (controlled by `max_candidates`). For each candidate first move, it simulates the resulting board state and evaluates the best available second move. The function selects the first move that maximises the combined score of both moves. Board-clearing is given strict priority: a sequence that clears the board is always preferred over a higher-scoring sequence that does not.

```
def _best_two_move_plan(
    G: List[List[Optional[str]]],
    trie: "base.Trie",
    *,
    min_len: int,
    max_candidates: int = DEFAULT_MAX_CANDIDATES,
) -> List[Tuple[str, Tuple[Coord, ...]]]:
    playable = words_map_all_paths_local(G, trie, min_len=min_len
    )
    if not playable:
        return []

    best_total = -1
    best_plan: List[Tuple[str, Tuple[Coord, ...]]] = []

    # rank all possible first moves (word, path)
    ranked = sorted(
        playable.items(),
        key=lambda kv: base.score_word(kv[0]),
        reverse=True,
    )
    # beam prune: keep only top 20 words by immediate score
    if max_candidates is not None and max_candidates > 0:
        ranked = ranked[:max_candidates]
    for w1, paths1 in ranked:
        s1 = base.score_word(w1)
        for p1 in paths1:
```

```
            G1 = _simulate_move(G, p1)

            playable1 = words_map_all_paths_local(G1, trie,
    min_len=min_len)
            if not playable1:
                total = s1
                if total > best_total:
                    best_total = total
                    best_plan = [(w1, p1)]
                continue

            # planning the 2nd move
            w2, p2 = _greedy_best_choice(G1, trie, playable1,
    min_len=min_len)
            s2 = base.score_word(w2)
            total = s1 + s2
            if total > best_total:
                best_total = total
                best_plan = [(w1, p1), (w2, p2)]

    return best_plan
```

Listing 3: Greedy-2 move selection: `_best_two_move_plan`

## 9.4 Greedy-3: `_best_three_move_plan`

This function extends the lookahead to 3 moves. It works by iterating over three planning horizons (3, 2, 1) on a simulated copy of the board. At each horizon, it evaluates all candidate first moves and their downstream sequences. As with Greedy-2, the top 20 candidates by immediate score are kept at each step to keep the search tractable. The function returns the first move from the best 3-move sequence found, again prioritising board-clearing above raw score.

```
def _best_three_move_plan(
    G: List[List[Optional[str]]],
    trie: "base.Trie",
    *,
    min_len: int,
    max_candidates: int = DEFAULT_MAX_CANDIDATES,
) -> List[Tuple[str, Tuple[Coord, ...]]]:

    plan: List[Tuple[str, Tuple[Coord, ...]]] = []

    # work on a copied grid
    G_sim: List[List[Optional[str]]] = [row[:] for row in G]

    for horizon in (3, 2, 1):
        playable = words_map_all_paths_local(G_sim, trie, min_len
    =min_len)
        if not playable:
            break
        w, path = _choose_move_with_ply(
            G_sim,
            trie,
            playable,
            min_len=min_len,
            ply=horizon,
            max_candidates=max_candidates,
        )
        plan.append((w, path))
```

```
        # apply to the simulated grid for the next planned step
        base.physics.remove_path(G_sim, path)
        base.physics.apply_gravity(G_sim)

    return plan
```

Listing 4: Greedy-3 move selection: `_best_three_move_plan`

## 9.5 Hybrid Stage 1: `_select_hybrid_policy_ply`

This function decide which greedy depth (1, 2, or 3) the Hybrid player should use at any given moment in the game, based on how full the board currently is.

```
def _select_hybrid_policy_ply(
    G: List[List[Optional[str]]],
    *,
    rows: int,
    cols: int,
    early_threshold: float = HYBRID_EARLY_THRESHOLD,
    late_threshold: float = HYBRID_LATE_THRESHOLD,
) -> int:
    initial_tile_count = max(1, rows * cols)
    remaining_ratio = _tiles_left(G) / initial_tile_count
    # preset: early_threshold=0.5, latethreshold=0.2
    if remaining_ratio > early_threshold:
        return 1
    if remaining_ratio > late_threshold:
        return 2
    return 3
```

Listing 5: Greedy Decision: `_select_hybrid_policy_ply`

## 9.6 Hybrid move chooser: `_choose_move_hybrid`

This function decides which word to play next in a game of Word Soup. It is the brain of the Hybrid player. Instead of just picking the highest score word, it looks ahead to simulate the rest of the game might play out, and picks the word that gives the best overall outcome (with board clearing as the top priority).

```
def _choose_move_hybrid(
    G: List[List[Optional[str]]],
    trie: "base.Trie",
    playable: Dict[str, List[Tuple[Coord, ...]]],
    *,
    min_len: int,
    rows: int,
    cols: int,
    top_k: int = HYBRID_TOP_K,
    early_threshold: float = HYBRID_EARLY_THRESHOLD,
    late_threshold: float = HYBRID_LATE_THRESHOLD,
    max_candidates: int = DEFAULT_MAX_CANDIDATES,
) -> Tuple[str, Tuple[Coord, ...]]:
    deep_ply = _select_hybrid_policy_ply(
        G,
        rows=rows,
        cols=cols,
        early_threshold=early_threshold,
        late_threshold=late_threshold,
    )

    shallow_ranked: List[Tuple[int, str, Tuple[Coord, ...]]] = []
    for w, paths in playable.items():
        move_score = base.score_word(w)
        for path in paths:
            G_after = _simulate_move(G, path)
            shallow_score = move_score + (CLEAR_BONUS if
    _grid_is_empty(G_after) else 0)
            shallow_ranked.append((shallow_score, w, path))

```

```
    shallow_ranked.sort(key=lambda item: item[0], reverse=True)
    k = max(1, min(top_k, len(shallow_ranked)))
    shortlist = shallow_ranked[:k]

    best_word: Optional[str] = None
    best_path: Optional[Tuple[Coord, ...]] = None
    best_value: Tuple[int, int] = (-1, -10**9)

    for _, w, path in shortlist:
        move_score = base.score_word(w)
        G_after = _simulate_move(G, path)

        if _grid_is_empty(G_after):
            total_score = move_score + CLEAR_BONUS
            compare_value = (1, total_score)
        else:
            continuation_score, cleared = _rollout_with_policy(
                G_after,
                trie,
                min_len=min_len,
                ply=deep_ply,
                max_candidates=max_candidates,
            )
            total_score = move_score + continuation_score
            compare_value = (int(cleared), total_score)

        if compare_value > best_value:
            best_value = compare_value
            best_word = w
            best_path = path

    assert best_word is not None and best_path is not None
    return best_word, best_path
```

Listing 6: Greedy Decision: `_choose_move_hybrid`

## 9.7 Hybrid Game Loop: `play_one_hybrid_run`

It plays one complete game of Word Soup using the Hybrid policy. On each turn it finds all valid words on the current grid, picks the best one using the hybrid decision logic, removes those letters, and applies gravity. It keeps going until no more words can be formed. At the end, it returns the final score, whether the board was cleared, the list of words played, and a dictionary of performance metrics.

```
def play_one_hybrid_run(
    G: List[List[Optional[str]]],
    trie: "base.Trie",
    *,
    min_len: int = 3,
    rows: int = DEFAULT_ROWS,
    cols: int = DEFAULT_COLS,
    top_k: int = HYBRID_TOP_K,
    early_threshold: float = HYBRID_EARLY_THRESHOLD,
    late_threshold: float = HYBRID_LATE_THRESHOLD,
    max_candidates: int = DEFAULT_MAX_CANDIDATES,
) -> Tuple[int, bool, List[str], Dict[str, float]]:
    """
    Play one full game using the hybrid player.

    Main priority order:
      1. Clear more boards.
      2. Then maximise score.
    """
    total = 0
    chosen: List[str] = []
    branching_factors: List[int] = []
    top1_scores: List[int] = []
    top3_means: List[float] = []
    tie_events = 0
    multi_path_events = 0
    move_scores: List[int] = []
    tiles_removed = 0
```

```
29
30      while True:
31          playable_all = words_map_all_paths_local(G, trie, min_len
    =min_len)
32          bf = len(playable_all)
33          branching_factors.append(bf)
34          all_scores_sorted = sorted((base.score_word(w) for w in
    playable_all.keys()), reverse=True)
35          if all_scores_sorted:
36              top1_scores.append(all_scores_sorted[0])
37              top3_means.append(sum(all_scores_sorted[:3]) / min(3,
     len(all_scores_sorted)))
38          else:
39              top1_scores.append(0)
40              top3_means.append(0.0)
41
42          if not playable_all:
43              metrics = {
44                  "n_moves": float(len(chosen)),
45                  "branching_factor_mean": float(statistics.mean(
    branching_factors)) if branching_factors else 0.0,
46                  "branching_factor_median": float(statistics.
    median(branching_factors)) if branching_factors else 0.0,
47                  "top1_score_mean": float(statistics.mean(
    top1_scores)) if top1_scores else 0.0,
48                  "top3_mean": float(statistics.mean(top3_means))
    if top3_means else 0.0,
49                  "tie_rate": float(tie_events / len(
    branching_factors)) if branching_factors else 0.0,
50                  "multi_path_rate": float(multi_path_events / max
    (1, len(chosen))),
51                  "avg_points_per_move": float(total / max(1, len(
    chosen))),
52                  "max_move_score": float(max(move_scores) if
    move_scores else 0),
53                  "avg_word_len": float(statistics.mean([len(w) for
     w in chosen])) if chosen else 0.0,
```

```
                "score_per_removed_cell": float(total / max(1, tiles_removed)),
                "final_cells_remaining": float(_count_nonempty(G)),
            }
            if _grid_is_empty(G):
                return total + CLEAR_BONUS, True, chosen, metrics
            return total, False, chosen, metrics

        w, move_s, tiles_inc, is_multi, is_tie = _apply_one_hybrid_move(
            G,
            trie,
            playable_all,
            min_len=min_len,
            rows=rows,
            cols=cols,
            top_k=top_k,
            early_threshold=early_threshold,
            late_threshold=late_threshold,
            max_candidates=max_candidates,
        )

        if is_tie:
            tie_events += 1
        if is_multi:
            multi_path_events += 1

        move_scores.append(move_s)
        chosen.append(w)
        total += move_s
        tiles_removed += tiles_inc

        if _grid_is_empty(G):
            total_with_bonus = total + CLEAR_BONUS
            return total_with_bonus, True, chosen, {
                "n_moves": float(len(chosen)),
```

```
            "branching_factor_mean": float(statistics.mean(branching_factors)) if branching_factors else 0.0,
            "branching_factor_median": float(statistics.median(branching_factors)) if branching_factors else 0.0,
            "top1_score_mean": float(statistics.mean(top1_scores)) if top1_scores else 0.0,
            "top3_mean": float(statistics.mean(top3_means)) if top3_means else 0.0,
            "tie_rate": float(tie_events / len(branching_factors)) if branching_factors else 0.0,
            "multi_path_rate": float(multi_path_events / max(1, len(chosen))),
            "avg_points_per_move": float(total / max(1, len(chosen))),
            "max_move_score": float(max(move_scores) if move_scores else 0),
            "avg_word_len": float(statistics.mean([len(w) for w in chosen])) if chosen else 0.0,
            "score_per_removed_cell": float(total / max(1, tiles_removed)),
            "final_cells_remaining": float(_count_nonempty(G)),
        }
```

Listing 7: Hybrid Game Loop: `play_one_hybrid_run`

## References


[1] Dimitri P. Bertsekas. *Abstract Dynamic Programming*. Athena Scientific, 3rd edition, 2022. https://www.mit.edu/~dimitrib/AbstractDP_ED3_TEXT_2021.pdf.

[2] Dimitris Bertsimas and Alex Paskov. An exact and interpretable solution to wordle. *Operations Research*, 73(3):1384–1394, 2025. https://auction-upload-files.s3.amazonaws.com/Wordle_Paper_Final.pdf, 2022.

[3] Cameron B. Browne, Edward Powley, Daniel Whitehouse, Simon M. Lucas, Peter I. Cowling, Philipp Rohlfshagen, Stephen Tavener, Diego Perez, Spyridon Samothrakis, and Simon Colton. A survey of monte carlo tree search methods. *IEEE Transactions on Computational Intelligence and AI in Games*, 4(1):1–43, 2012. DOI: https://doi.org/10.1109/TCIAIG.2012.2186810.

[4] Nitesh V. Chawla, Kevin W. Bowyer, Lawrence O. Hall, and W. Philip Kegelmeyer. SMOTE: Synthetic minority over-sampling technique. *Journal of Artificial Intelligence Research*, 16:321–357, 2002. https://doi.org/10.1613/jair.953.

[5] Edward Fredkin. Trie memory. *Communications of the ACM*, 3(9):490–499, 1960. https://doi.org/10.1145/367390.367400.

[6] Marco Kemmerling, Daniel Lütticke, and Robert H. Schmitt. Beyond games: A systematic review of neural Monte Carlo tree search applications. *Applied Intelligence*, 54:1020–1046, 2024. https://doi.org/10.48550/arXiv.2303.08060.

[7] Maciej Świechowski, Konrad Godlewski, Bartosz Sawicki, and Jacek Mańdziuk. Monte Carlo tree search: A review of recent modifications and applications. *Artificial Intelligence Review*, 56:2497–2562, 2023. https://doi.org/10.1007/s10462-022-10228-y.

[8] The Spruce Crafts. Scrabble tile distribution and point values, n.d. Accessed: 7 January 2026, Available: https://www.thesprucecrafts.com/scrabble-tile-distribution-and-point-values-412402.